

本讲内容

- 基于神经网络的分类

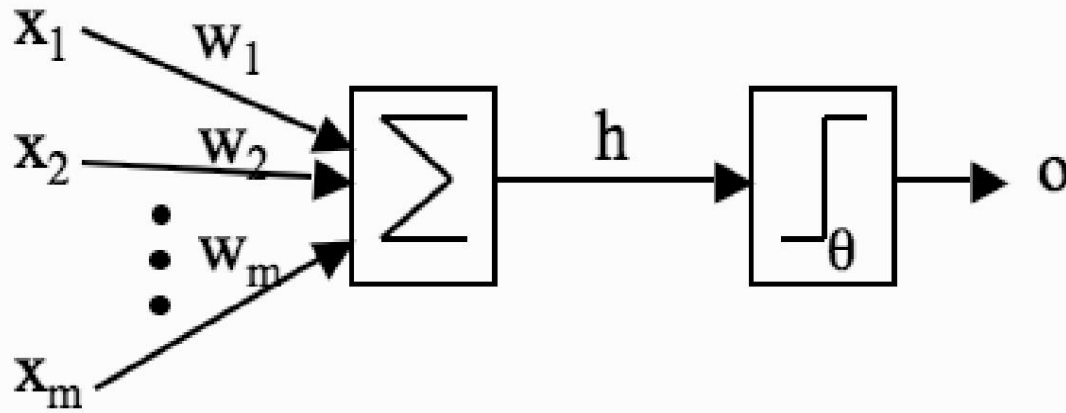
神经网络

- 神经网络最早是由心理学家和神经学家提出的，旨在寻求开发和测试神经的计算模拟。
- 粗略地说，神经网络是一组连接的输入/输出单元，其中每个连接都与一个权相关联。在学习阶段，通过调整权值，使得神经网络的预测准确性逐步提高。由于单元之间的连接，神经网络学习又称**连接者学习**。

神经网络

- 神经网络是以模拟人脑神经元的数学模型为基础而建立的，它由一系列神经元组成，单元之间彼此连接。
- 从信息处理角度看，神经元可以看作是一个多输入单输出的信息处理单元，根据神经元的特性和功能，可以把神经元抽象成一个简单的数学模型。

神经元



- 输入值 x_i 与对应的权重 w_i 相乘，然后神经元会将这些乘积的结果加总。如果这个总和大于阈值 θ ，那么神经元就会被激活；否则，神经元不被激活。

- 设： $x_1=1, x_2=0, x_3=0.5$; $w_1=1, w_2=-0.5, w_3=-1$

则： $h=1 \times 1 + 0 \times -0.5 + 0.5 \times -1 = 1 + 0 + (-0.5) = 0.5$

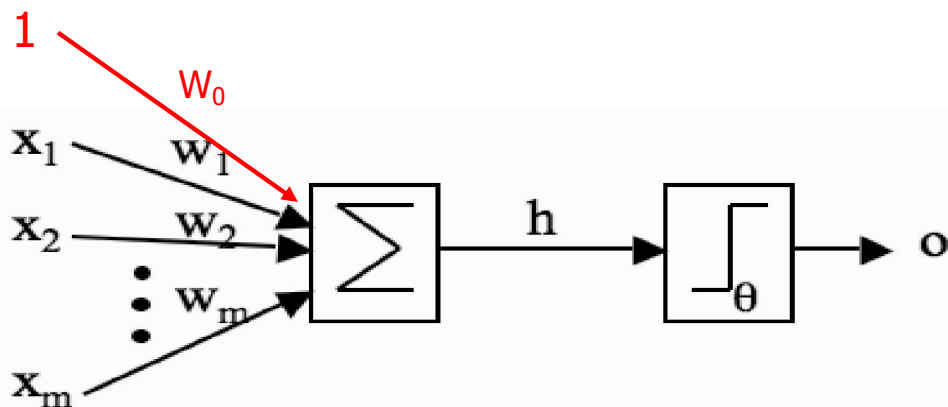
如果设定阈值 $\theta=0$ ，因为 $h=0.5>0$ ，所以神经元会被激活，输出为1

偏置 (bias)

- 激活阈值 θ 决定了神经元的输入累计到多少才会“激活”
- 假设所有输入都是零。这时，无论权重 (weights) 是多少都没有意义（因为0乘以任何数都还是0）。这时候唯一能决定神经元是否被激活的就是阈值。
- 如果阈值是固定的，想让某一个神经元在所有输入都是零的情况下被激活，而另一个神经元却不被激活，就会遇到麻烦。因为既然所有神经元的输入相同、阈值也一样，那它们的行为也应该一样。无论怎么设置权重，都无法做出不同的反应。
- 如果阈值是动态的，需额外增加一个参数来表示它，写代码时也得专门处理这个参数（比如需要更新该参数）

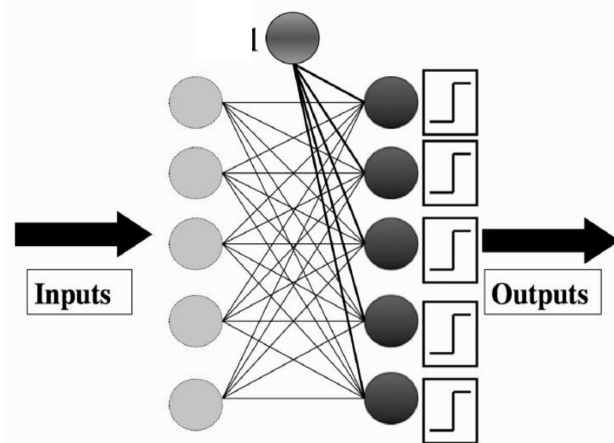
偏置 (bias)

- 解决方法：阈值仍然固定为0。增加一个额外的输入，其值始终固定（通常设为1，但其实任何非零值都可以），其对应的权重 w_0 可调。
- 因为这个额外输入的值始终是1，即使其他所有输入都是0，也仍然有一个固定的输入值在起作用。这样，权重 w_0 的值会在训练过程中自动调整，使得神经元能在需要的时候被激活，或者不被激活
- 这个额外的输入与其权重的乘积($1*w_0$)被称为偏置 (bias)



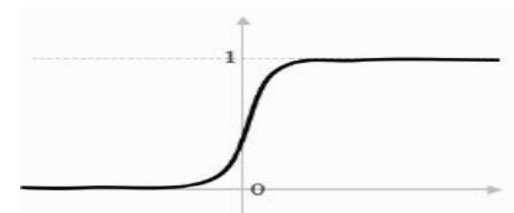
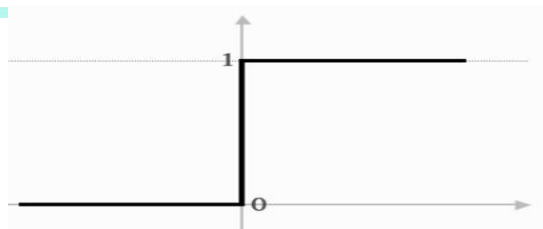
感知机 (Perceptron)

- **感知机**是由一组**神经元**组成的集合，加上一些输入节点和将输入连接到神经元的权重，如右图
- 图的左侧用**浅灰色**阴影表示的是**输入节点**。这些**并不是神经元**，而只是用来直观表示网络是如何接收输入数值的，以及输入值有多少个（也就是输入向量的**维度**或元素个数）。这些输入节点几乎总是画成圆形，看起来像神经元一样，容易让人混淆，这里用不同的颜色以示区分
- 图的右侧是神经元，可以看到它们的**加法部分**（用一个圆圈表示）以及**阈值判断部分**。不过在实际使用中，通常不会把阈值器单独画出。只需要记住它是神经元的一部分就可以了



激活函数

- 前面提到的激活函数是阈值激活函数。其问题是：不连续，有突变，无法求导（无法求梯度）
- 可以使用一种看起来像阈值函数、但可导的激活函数来替代（如果眯着眼看阈值函数的图像，会发现它看起来有点像S形曲线）。这种S形曲线在数学上确实是有对应的表达形式，叫做S型函数（sigmoid functions）。这类函数中最常用的形式如右图（其中 β 是一个正的参数）。它的导数形式很优美



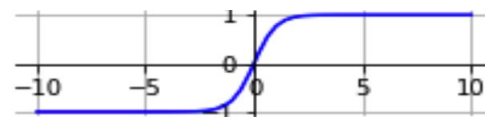
$$a = g(h) = \frac{1}{1 + \exp(-\beta h)}.$$

$$\begin{aligned} g'(h) = \frac{dg}{dh} &= \frac{d}{dh}(1 + e^{-\beta h})^{-1} \\ &= -1(1 + e^{-\beta h})^{-2} \frac{de^{-\beta h}}{dh} \\ &= -1(1 + e^{-\beta h})^{-2} (-\beta e^{-\beta h}) \\ &= \frac{\beta e^{-\beta h}}{(1 + e^{-\beta h})^2} \\ &= \beta g(h)(1 - g(h)) \\ &= \beta a(1 - a) \end{aligned}$$

激活函数

- 另一种激活函数是双曲正切函数 (hyperbolic tangent function) 。它是一种与sigmoid函数不同但类似的函数；它依然是一个S型函数，但它在 ± 1 处“饱和”（也就是说，函数值趋近于某个固定值），而不是在 0 和 1 之间饱和，这在某些情况下会更有用

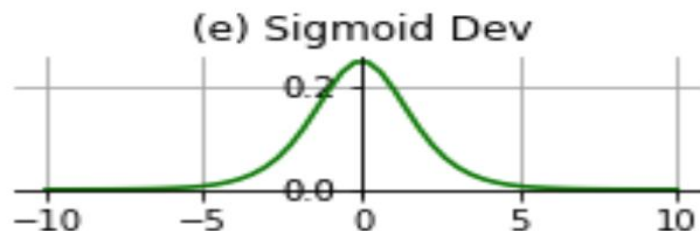
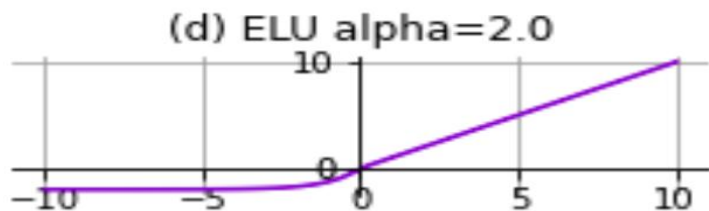
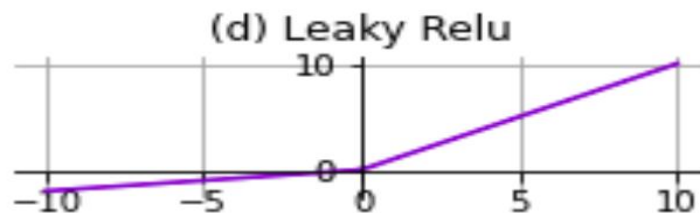
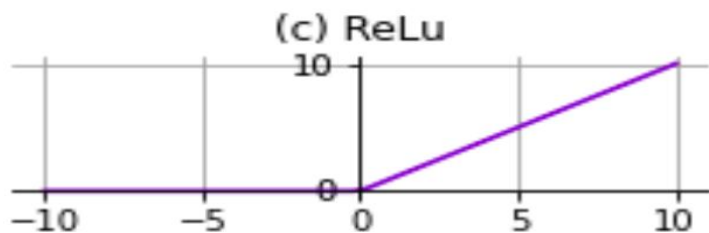
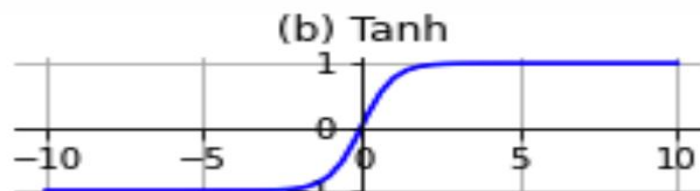
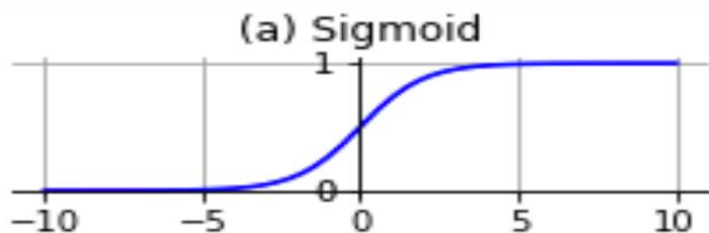
$$a = g(h) = \tanh(h) = \frac{\exp(h) - \exp(-h)}{\exp(h) + \exp(-h)},$$



- tanh函数的导数形式也很优美： $\frac{d}{dx} \tanh x = (1 - \tanh^2(x))$
- 可以很容易地在sigmoid函数和tanh函数的形式之间进行转换：因为如果一个函数的饱和值是 ± 1 ，那么可以通过 $0.5 \times (x+1)$ 将其转换为区间(0,1)内的形式

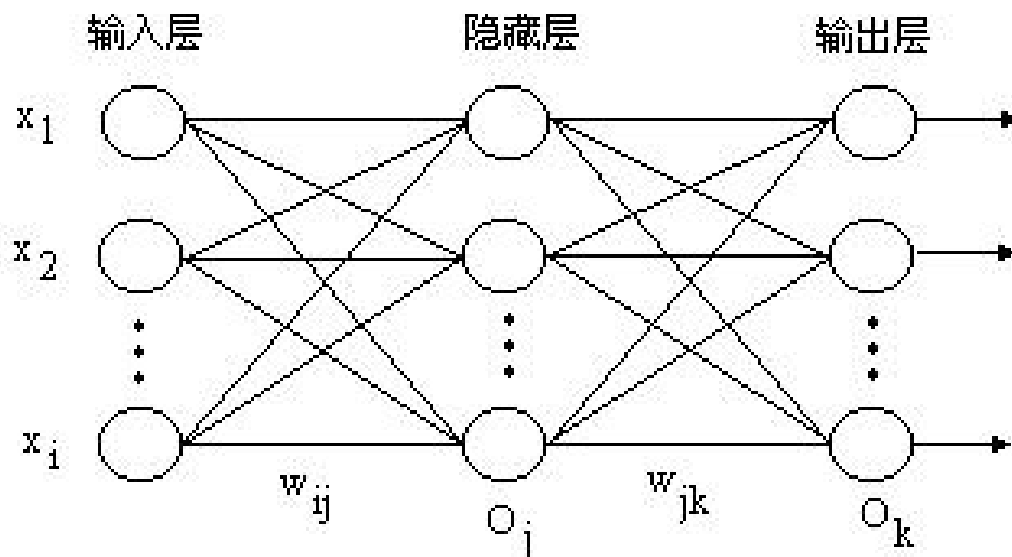
激活函数

- 更多激活函数:



神经网络

- 神经网络有三个要素：拓扑结构、连接方式、学习规则



- 可以从这三方面对神经网络进行分类

神经网络

- **神经网络的拓扑结构**：神经网络的单元通常按照层次排列，根据网络的层数，可以将神经网络分为**单层神经网络**、**两层神经网络**、**三层神经网络**等
- 结构简单的神经网络，在学习时收敛的速度快，但准确度低。
- 神经网络的层数和每层的单元数由问题的复杂程度而定。问题越复杂，神经网络的层数就越多。
 - 例如，两层神经网络常用来解决线性问题，而多层网络就可以解决多元非线性问题

神经网络

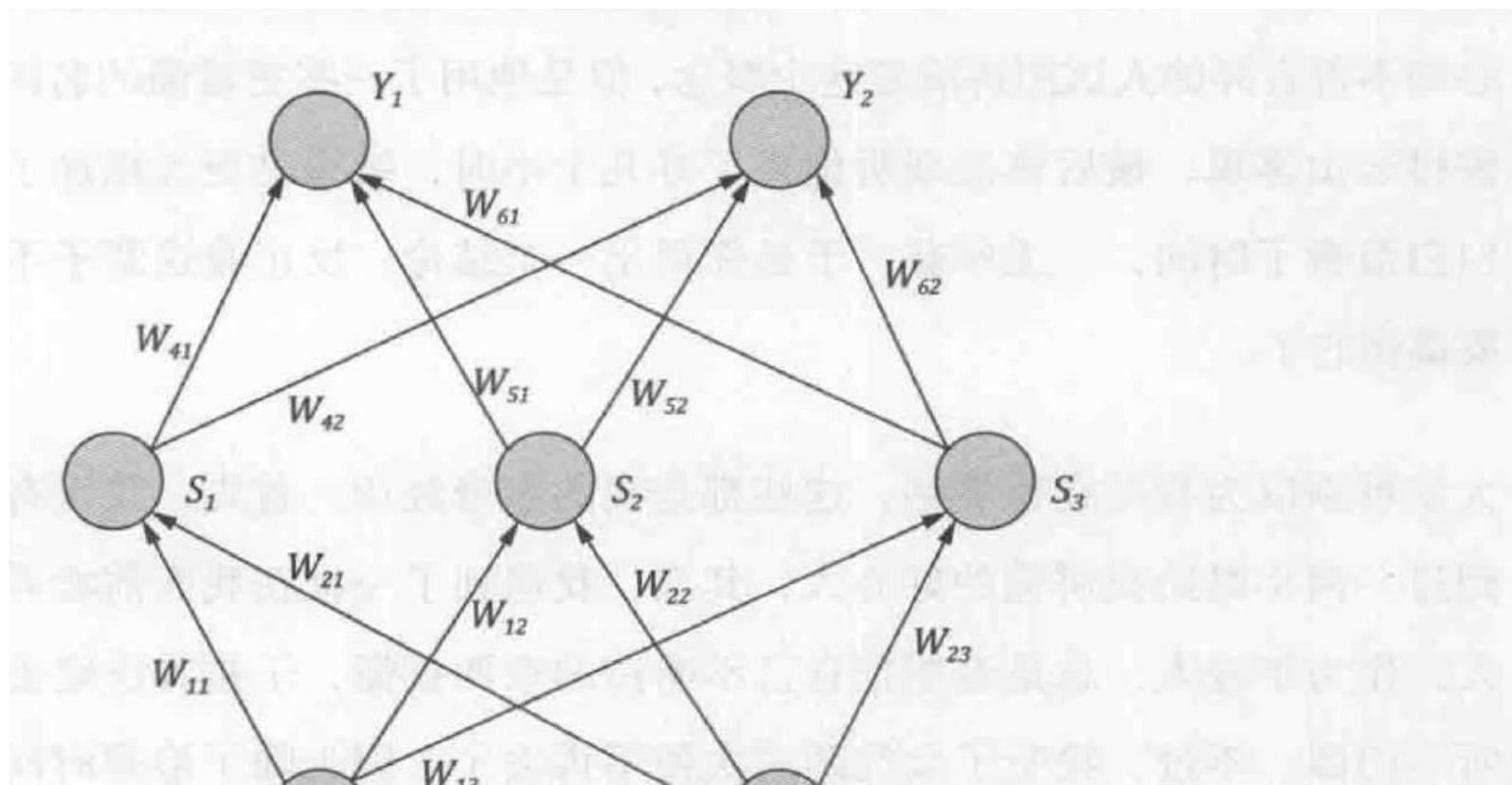
- **神经网络的连接**：包括层次之间的连接和每一层内部的连接，连接的强度用权来表示。
- 根据层次之间的连接方式，分为：
 - 前馈式网络：连接是单向的，上层单元的输出是下层单元的输入，如反向传播网络，Kohonen网络
 - 反馈式网络：除了单项的连接外，还把最后一层单元的输出作为第一层单元的输入，如Hopfield网络
- 根据连接的范围，分为：
 - 全连接神经网络：每个单元和相邻层上的所有单元相连
 - 局部连接网络：每个单元只和相邻层上的部分单元相连

神经网络

■ 神经网络的学习

- 根据学习方法分：
 - 感知器：有监督的学习方法，训练样本的类别是已知的，并在学习的过程中指导模型的训练
 - 认知器：无监督的学习方法，训练样本类别未知，各单元通过竞争学习。
- 根据学习时间分：
 - 离线网络：学习过程和使用过程是独立的
 - 在线网络：学习过程和使用过程是同时进行的
- 根据学习规则分：
 - 相关学习网络：根据连接间的激活水平改变权系数
 - 纠错学习网络：根据输出单元的外部反馈改变权系数
 - 自组织学习网络：对输入进行自适应地学习

人工神经网络这个词儿听起来像是用人工的办法模拟人脑，加上它使用了一些与生物有关的名词比如“神经元”等等，让人感觉很神秘，并且可能会联想到仿生学或认知科学等一辈子都搞不懂的东西。其实除了借用了生物学上的一些名词，并且做了一些形象的比喻外，人工神经网络和人脑没有半点关系，它本质上是我们前面介绍过的一种有向图，只不过它是一种特殊的有向图。我们已经知道有向图包括节点和连接这些节点的有向弧，而人工神经网络也具有节点，只是它使用了一个新的名词——神经元。而它的有向弧则被看成是连接神经元的神经。当然，它是一种特殊的有向图，其特殊性可以概括如下。



常用的神经网络

- 神经网络种类很多，常用的有如下四种
 - **Hopfield**网络，典型的反馈网络，结构单层，有相同的单元组成
 - 反向传播网络，前馈网络，结构多层，采用最小均方差的纠错学习规则，常用于语言识别和分类等问题
 - **Kohonen**网络：典型的自组织网络，由输入层和输出层构成，全连接
 - **ART**网络：自组织网络

深度神经网络

- Convolutional Neural Networks(CNN)卷积神经网络
- Recurrent neural Network(RNN) 循环神经网络
- Deep Belief Networks(DBN)深度信念网络

深度学习

- 深度学习是指多层神经网络上运用各种机器学习算法解决图像、文本等各种问题的算法集合
- 深度学习从大类上可以归入神经网络，不过在具体实现上有许多变化
- 学习的核心是特征学习，旨在通过分层网络获取分层次的特征信息，从而解决以往需要人工设计特征的重要难题

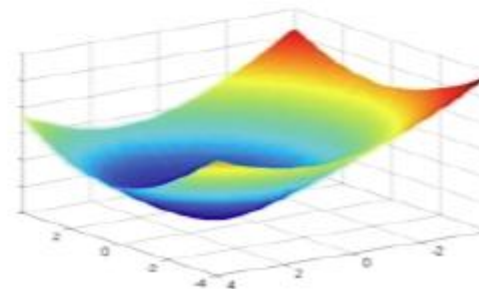
深度学习

- **Deep learning** is a subfield of **machine learning**

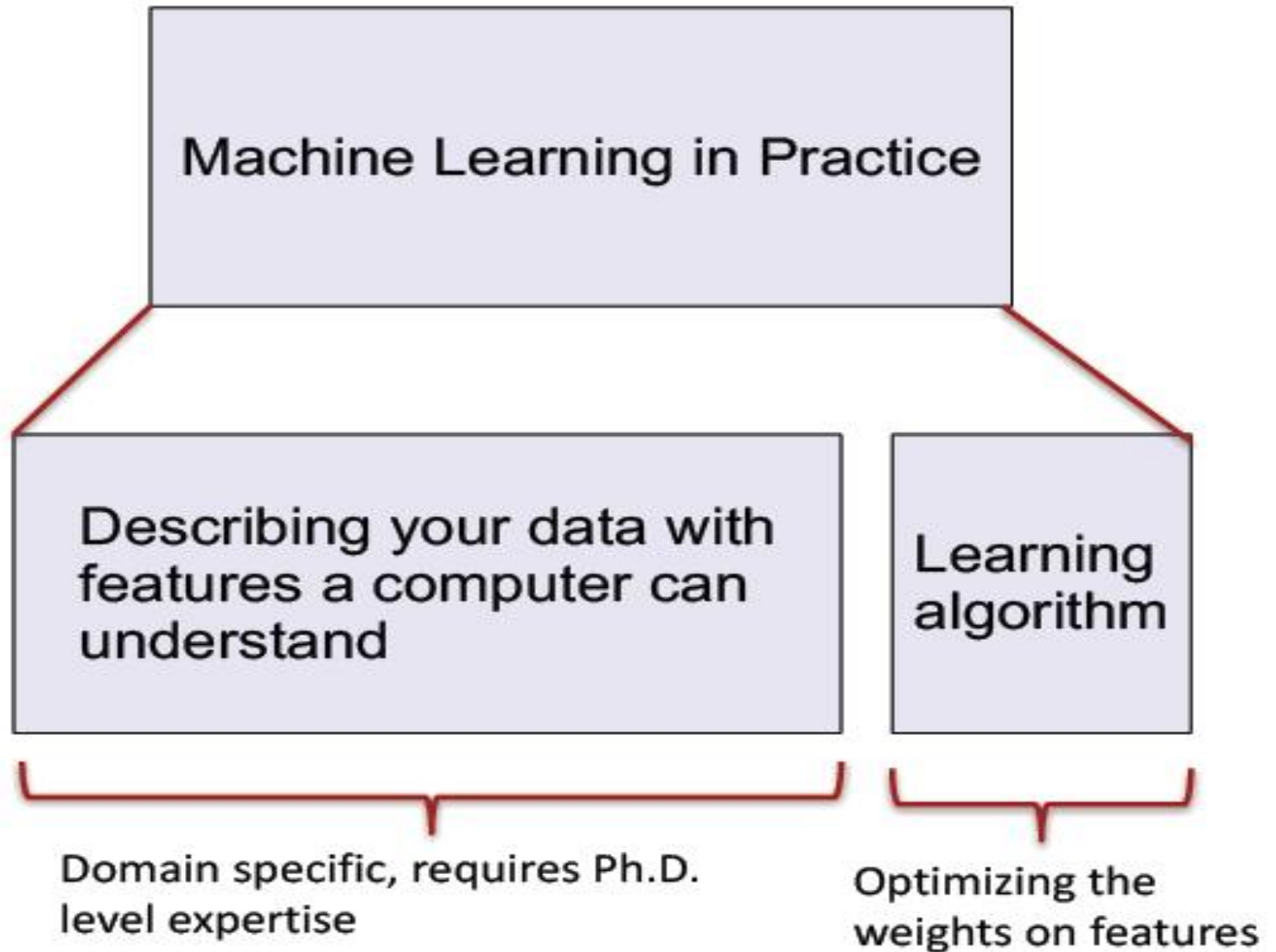
- Most machine learning methods work well because of **human-designed representations** and **input features**
 - For example: features for finding named entities like locations or organization names (Finkel et al., 2010):

Feature	NER
Current Word	✓
Previous Word	✓
Next Word	✓
Current Word Character n-gram	all
Current POS Tag	✓
Surrounding POS Tag Sequence	✓
Current Word Shape	✓
Surrounding Word Shape Sequence	✓
Presence of Word in Left Window	size 4
Presence of Word in Right Window	size 4

- Machine learning becomes just optimizing weights to best make a final prediction

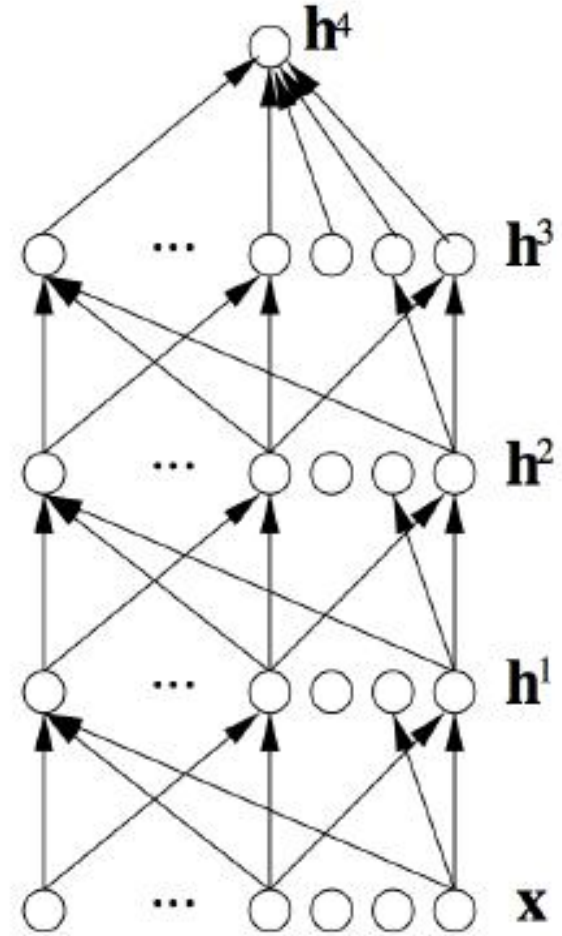


Machine Learning vs. Deep Learning



What's Deep Learning (DL)?

- In contrast to standard machine learning,
- Representation learning attempts to automatically learn good features or representations
- Deep learning algorithms attempt to learn (multiple levels of) representations (here: h^1, h^2, h^3) and an output (h^4)
- From “raw” inputs $\mathbf{x}$ (e.g. sound, pixels, characters, or words)



Reasons for Exploring Deep Learning

- Manually designed features are often over-specified, incomplete and take a long time to design and validate
- **Learned Features** are easy to adapt, fast to learn
- Deep learning provides a very flexible, (almost?) universal, learnable framework for **representing** world, visual and linguistic information.
- Deep learning can learn **unsupervised** (from raw text) and **supervised** (with specific labels like positive/negative)

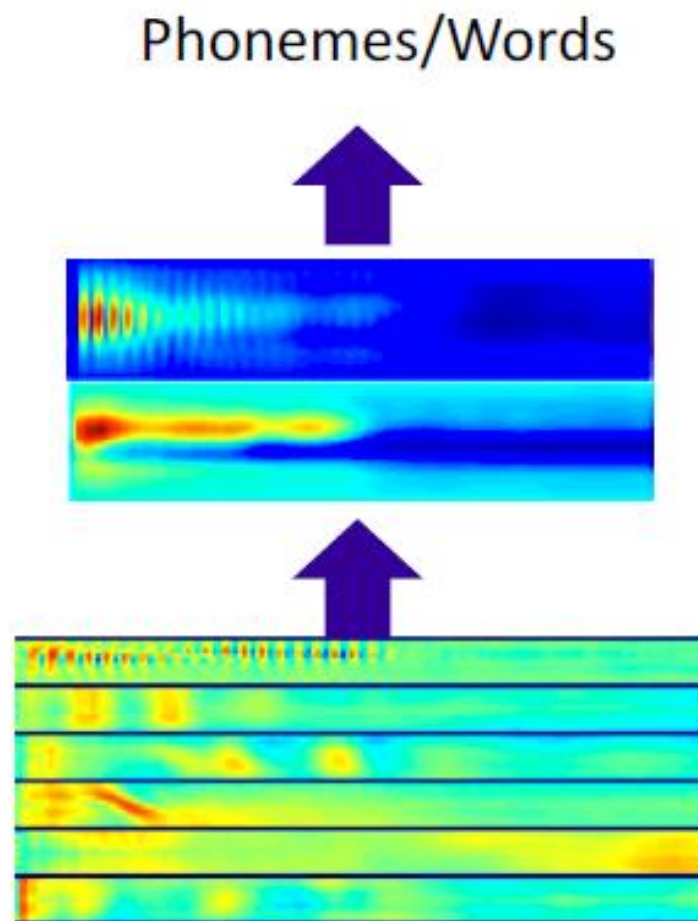
Reasons for Exploring Deep Learning

- In ~2010 **deep** learning techniques started outperforming other machine learning techniques. Why this decade?
 - Large amounts of training data favor deep learning
 - Faster machines and multicore CPU/GPUs favor Deep Learning
 - New models, algorithms, ideas
 - Better, more flexible learning of intermediate representations
 - Effective end-to-end joint system learning
 - Effective learning methods for using contexts and transferring between tasks
 - Better regularization and optimization methods
- **Improved performance** (first in speech and vision, then NLP)

Deep Learning for Speech

- The first breakthrough results of “deep learning” on large datasets happened in speech recognition
- Context-Dependent Pre-trained Deep Neural Networks for Large Vocabulary Speech Recognition
Dahl et al. (2010)

Acoustic model and WER	RT03S FSH	Hub5 SWB
Traditional features	27.4	23.6
Deep Learning	18.5 (-33%)	16.1 (-32%)



Deep Learning for Computer Vision

First major focus of deep learning groups was computer vision

The breakthrough DL paper: ImageNet Classification with Deep Convolutional Neural Networks by Krizhevsky, Sutskever, & Hinton, 2012, U. Toronto. 37% error red.



NeurIPS 2022 会议上，图灵奖得主、深度学习先驱 **Geoffrey Hinton** 被邀请发表演讲，以表彰他十年前与其研究生 Alex Krizhevsky 和 Ilya Sutskever 共同撰写的论文《ImageNet Classification with Deep Convolutional Neural Networks》，该论文因对该领域的「巨大影响」而被授予**时间检验奖**。这项工作发表于 2012 年，是卷积神经网络首次在 ImageNet 图像识别竞赛中表现人类水平的能力，它是启动第三次人工智能浪潮的关键事件。

基于神经网络的分类方法

- 神经网络（主要是感知器）经常用于分类
- 神经网络的分类知识体现在网络连接上，被隐式地存储在连接的权值中。
- 神经网络的学习就是通过迭代算法，对权值逐步修改的优化过程，学习的目标就是通过改变权值使训练集的样本都能被正确分类。

基于神经网络的分类方法

- 神经网络特别适用于下列情况的分类问题：
 - 数据量比较小，缺少足够的样本建立模型
 - 数据的结构难以用传统的统计方法来描述
 - 分类模型难以表示为传统的统计模型

基于神经网络的分类方法

■ 缺点:

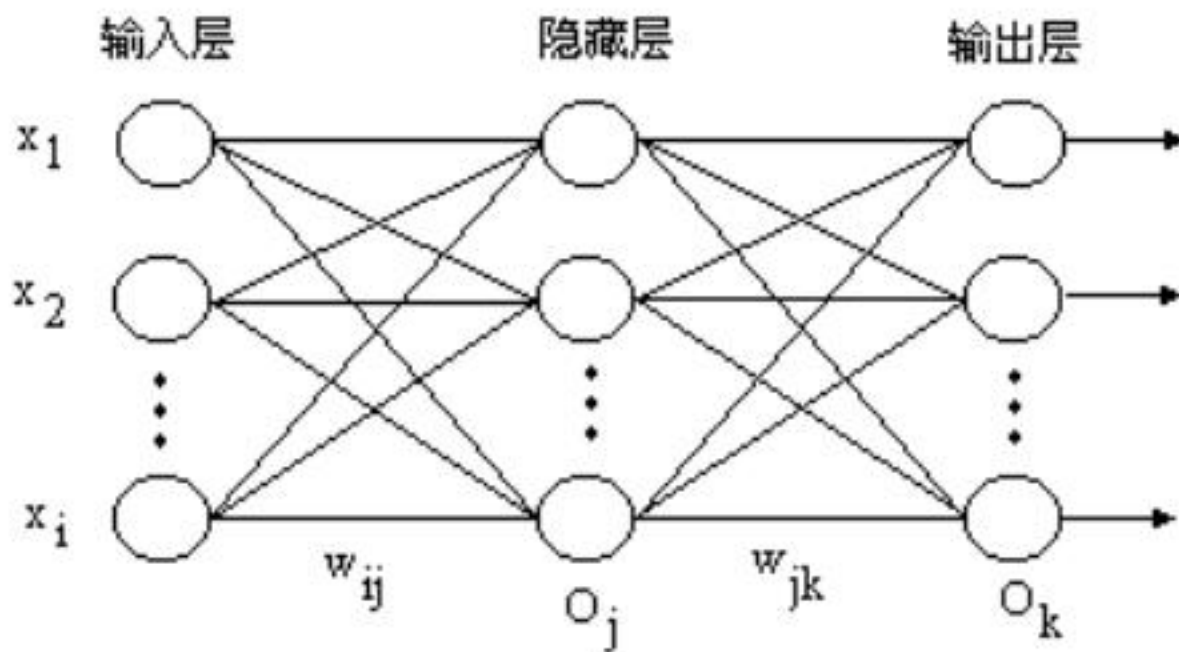
- 需要很长的训练时间，因而对于有足够长训练时间的应用更合适。
- 需要大量的参数，这些通常主要靠经验确定，如网络拓扑或“结构”。
- 可解释性差。该特点使得神经网络在数据挖掘的初期并不看好。

基于神经网络的分类方法

- 优点：
 - 分类的准确度高
 - 并行分布处理能力强
 - 分布存储及学习能力高
 - 对噪音数据有很强的鲁棒性和容错能力
- 最流行的基于神经网络的分类算法是80年代提出的后向传播算法。

后向传播算法

- 后向传播算法在**多路前馈**神经网络上学习。



定义网络拓扑

- 在开始训练之前，用户必须说明输入层的单元数、隐藏层数（如果多于一层）、每一隐藏层的单元数和输出层的单元数，以确定网络拓扑。

定义网络拓扑

- 对训练样本中每个属性的值进行规格化将有助于加快学习过程。通常，对输入值规格化，使得它们落入0.0和1.0之间。
 - 离散值属性可以重新编码，使得每个域值一个输入单元。例如，如果属性A的定义域为 (a_0, a_1, a_2) ，则可以分配三个输入单元表示A。即，我们可以用 I_0, I_1, I_2 作为输入单元。每个单元初始化为0。如果 $A = a_0$ ，则 I_0 置为1；如果 $A = a_1$ ， I_1 置1；如此下去。

定义网络拓扑

- 一个输出单元可以用来表示两个类（值1代表一个类，而值0代表另一个）。
- 如果多于两个类，则每个类使用一个输出单元。

定义网络拓扑

- 隐藏层单元数设多少个“最好”，没有明确的规则。
- 网络设计是一个实验过程，并可能影响准确性。权的初值也可能影响准确性。如果某个经过训练的网络的准确率太低，则通常需要采用不同的网络拓扑或使用不同的初始权值，重复进行训练。

后向传播算法

- 后向传播算法学习过程：
 - 迭代地处理一组训练样本，将每个样本的网络预测与实际类标号比较。
 - 每次迭代后，修改权值，使得网络预测和实际类之间的均方差最小。
- 这种修改“后向”进行。即，由输出层，经由每个隐藏层，到第一个隐藏层（因此称作后向传播）。尽管不能保证，一般地，权将最终收敛，学习过程停止。
- 算法终止条件：训练集中被正确分类的样本达到一定的比例，或者权系数趋近稳定。

后向传播算法

- 后向传播算法分为如下几步：
 - 初始化权
 - 向前传播输入
 - 向后传播误差

后向传播算法

■ 初始化权

- 网络的权通常被初始化为很小的随机数（例如，范围从-1.0到1.0，或从-0.5到0.5）。
- 每个单元都设有一个偏置 (bias)，偏置也被初始化为小随机数。

后向传播算法

- 对于每一个样本 X , 重复下面两步:
 - 向前传播输入
 - 向后传播误差

向前传播输入

- 计算各层每个单元的输入和输出。

- 输入层:

- 输出=输入=样本X的属性;

- 即, 对于单元 j , $O_j = I_j = X_j$

- 隐藏层和输出层:

- 输入=前一层的输出的线性组合,

- 即, 对于单元 j , $I_j = \sum w_{ij} O_i + \theta_j$

- 输出 $= O_j = \frac{1}{1 + e^{-I_j}}$

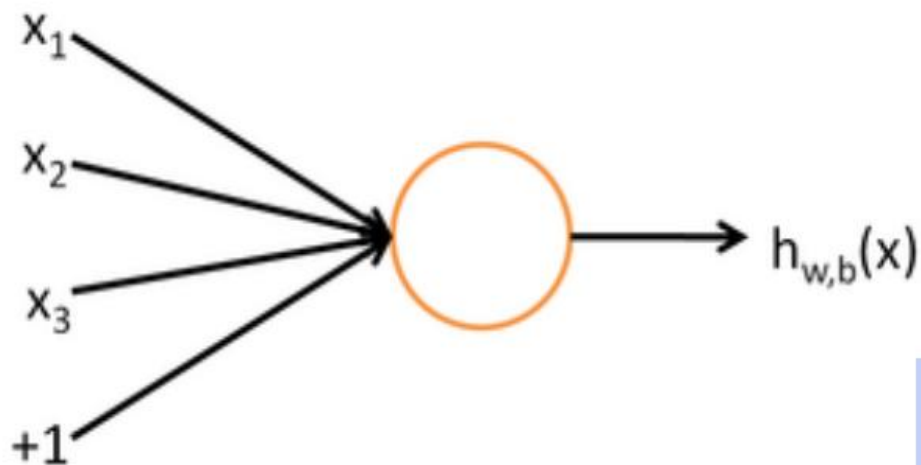
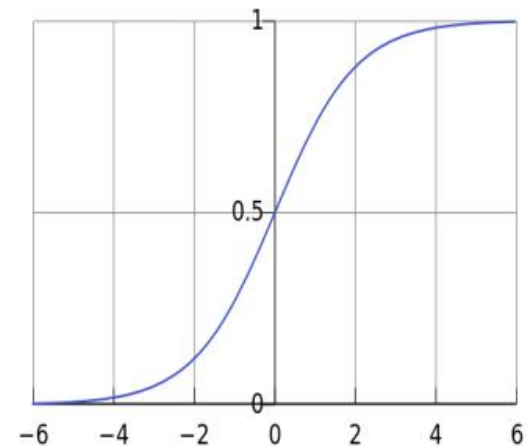
A neuron is essentially a binary logistic regression unit

f = nonlinear activation fct. (e.g. sigmoid), w = weights, b = bias, h = hidden, x = inputs

$$h_{w,b}(x) = f(w^T x + b)$$

b : We can have an “always on” feature, which gives a class prior, or separate it out, as a bias term

$$f(z) = \frac{1}{1 + e^{-z}}$$

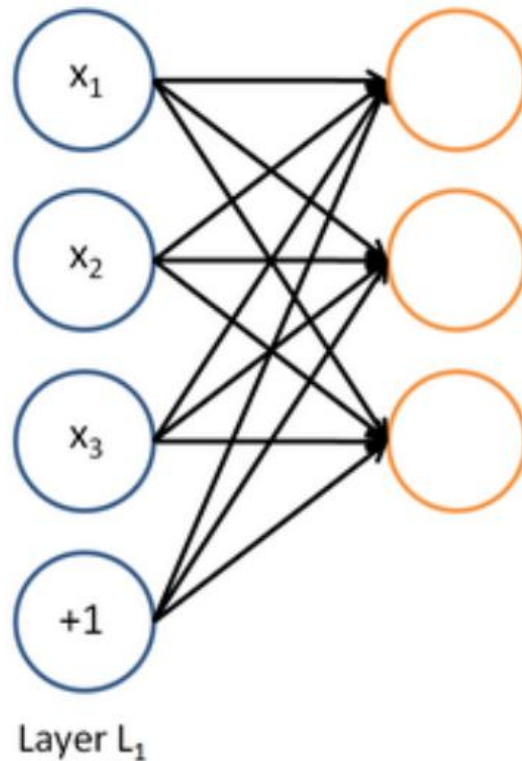


w, b are the parameters of this neuron
i.e., this logistic regression model

A neural network

= running several logistic regressions at the same time

If we feed a vector of inputs through a bunch of logistic regression functions, then we get a vector of outputs ...

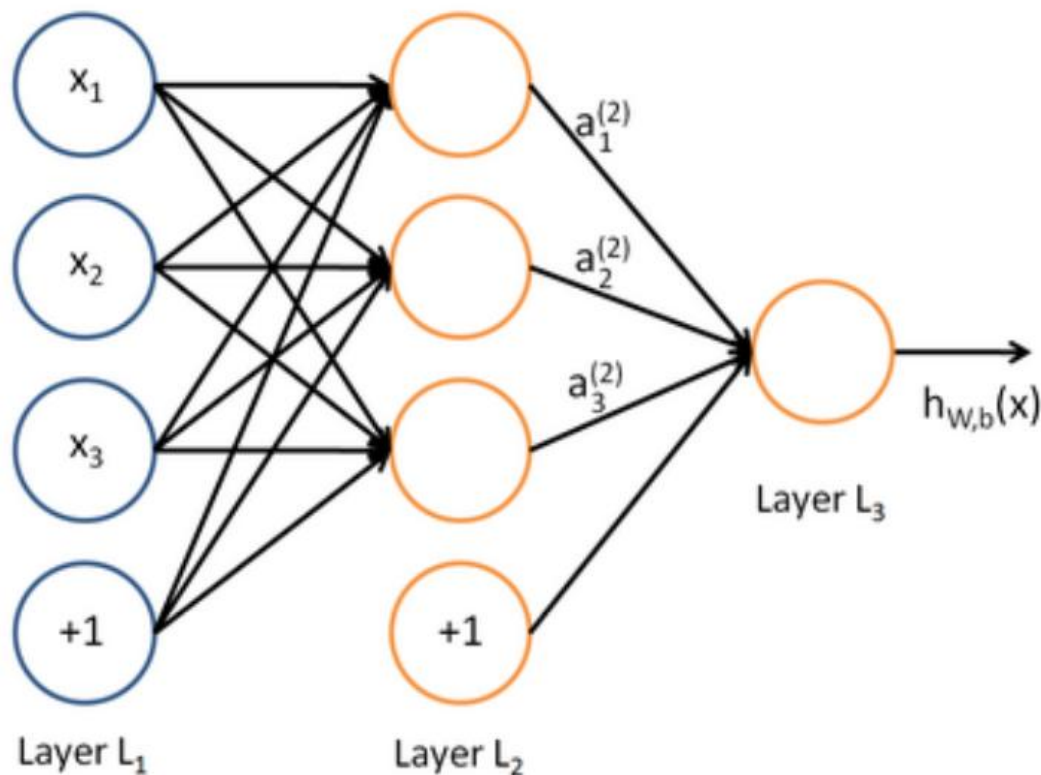


But we don't have to decide ahead of time what variables these logistic regressions are trying to predict!

A neural network

= running several logistic regressions at the same time

... which we can feed into another logistic regression function

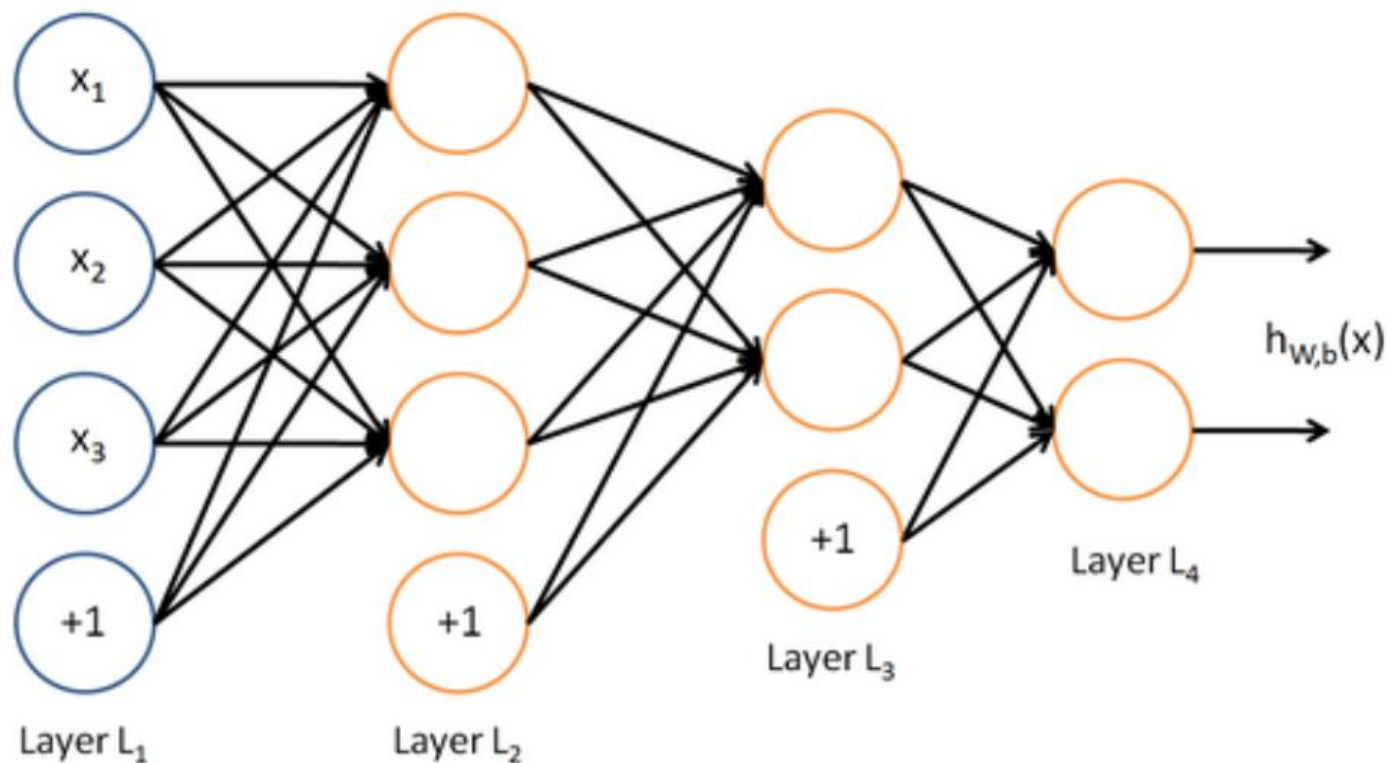


It is the loss function that will direct what the intermediate hidden variables should be, so as to do a good job at predicting the targets for the next layer, etc.

A neural network

= running several logistic regressions at the same time

Before we know it, we have a multilayer neural network....



Matrix notation for a layer

We have

$$a_1 = f(W_{11}x_1 + W_{12}x_2 + W_{13}x_3 + b_1)$$

$$a_2 = f(W_{21}x_1 + W_{22}x_2 + W_{23}x_3 + b_2)$$

etc.

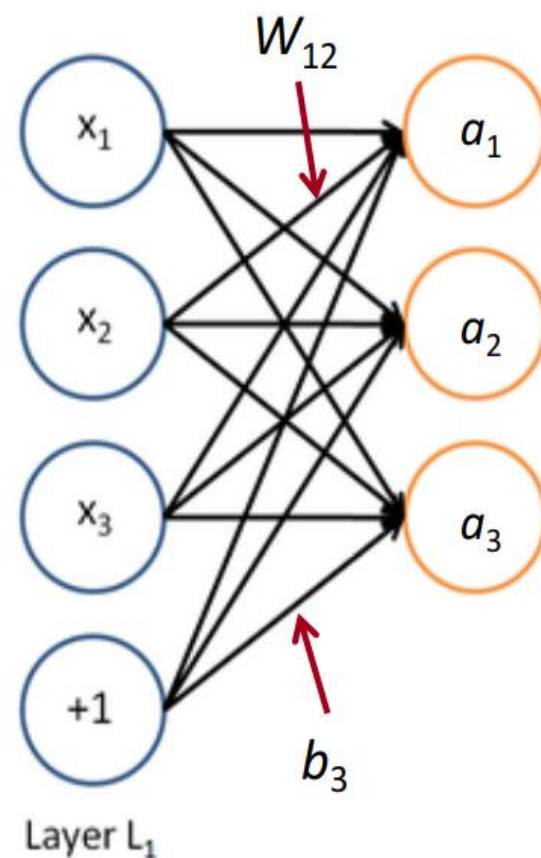
In matrix notation

$$z = Wx + b$$

$$a = f(z)$$

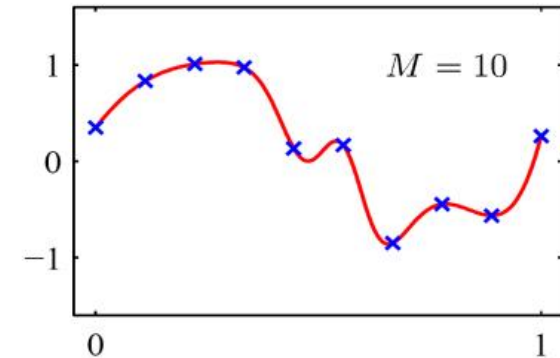
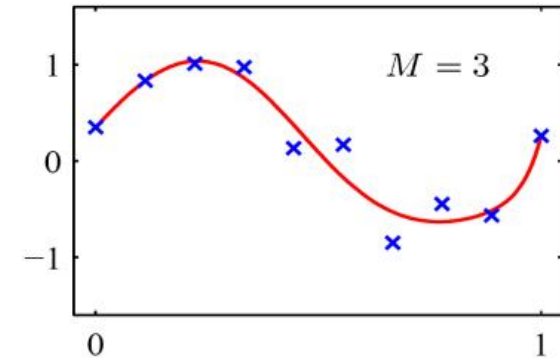
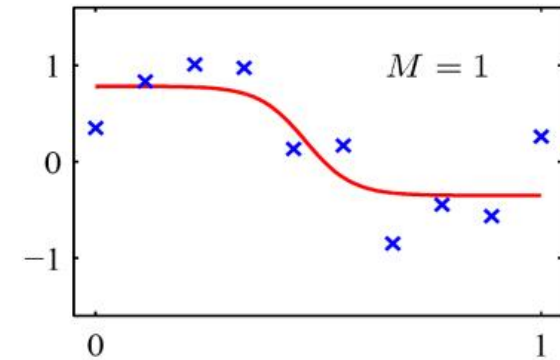
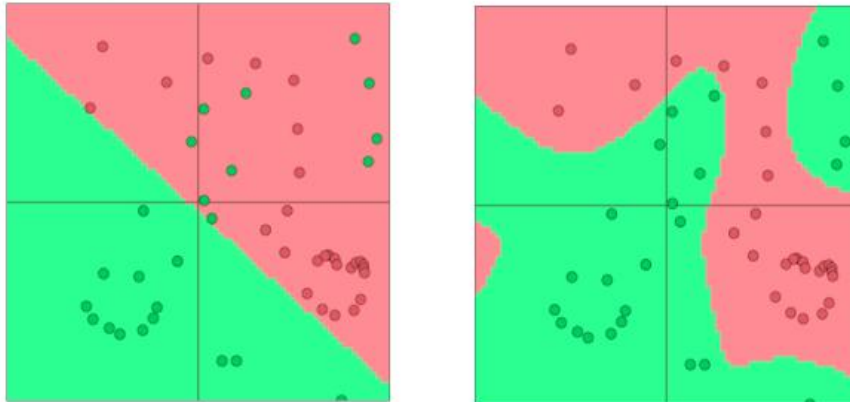
where f is applied element-wise:

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)]$$



Non-linearities (aka “f”): Why they’re needed

- Example: function approximation, e.g., regression or classification
 - Without non-linearities, deep neural networks can’t do anything more than a linear transform
 - Extra layers could just be compiled down into a single linear transform:
$$W_1 W_2 x = Wx$$
 - With more layers, they can approximate more complex functions!



向后传播误差

- 计算各层每个单元的误差。
- 向后传播误差，并更新权和偏置

向后传播误差

- 计算各层每个单元的误差。
 - 输出层单元j, 误差 $Err_j = O_j(1 - O_j)(T_j - O_j)$
 - O_j 是单元j的实际输出, 而 T_j 是j的真正输出。
 - 隐藏层单元j, 误差 $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$
 - w_{jk} 是由j到下一层中单元k的连接^k的权
 - Err_k 是单元k的误差

向后传播误差

更新权和偏差，以反映传播的误差。

权由下式更新： $\Delta w_{ij} = (l) E r_j O_i$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

其中， Δw_{ij} 是权 w_{ij} 的改变。 l 是学习率，通常取0和1之间的值。

偏置由下式更新： $\Delta \theta_j = (l) E r_j$

$$\theta_j = \theta_j + \Delta \theta_j$$

其中， $\Delta \theta_j$ 是偏置 θ_j 的改变。



后向传播算法

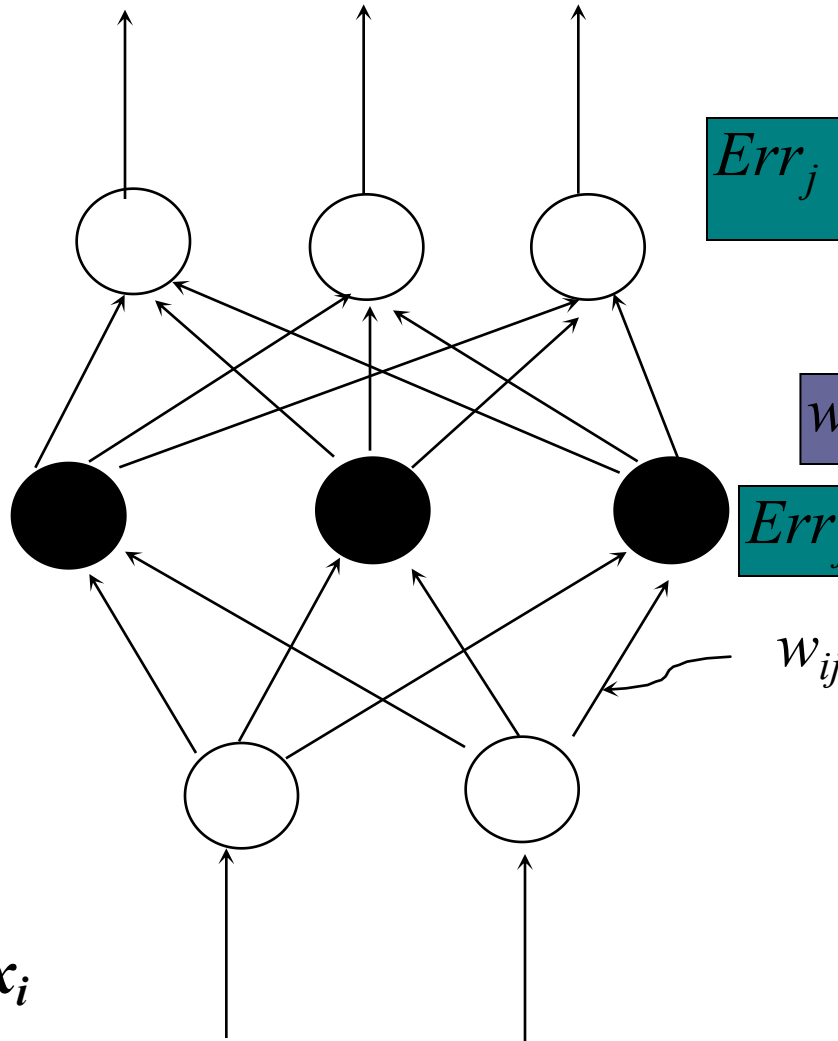
Output vector

Output nodes

Hidden nodes

Input nodes

Input vector: x_i



$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (l) Err_j$$

$$w_{ij} = w_{ij} + (l) Err_j O_i$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$O_j = \frac{1}{1 + e^{-I_j}}$$

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

反向传播

- 利用梯度下降法优化损失函数，迭代修改参数

用一个合适的损失函数来度量训练样本的输出和真正输出之间的差别，接着对这个损失函数进行优化求最小值，对应的一系列线性系数矩阵 W ,偏置向量 b 即为要求的最终参数。

在神经网络中，损失函数最小值的求解过程最常见的一般是通过梯度下降法（**Gradient Descent, GD**）一步步迭代完成，当然也可以是其他的迭代方法

损失函数 (Loss function)

- ⑩ 衡量预测结果 (prediction) 和实际结果 (ground truth, target) 间的差异
- ⑩ 网络训练的目标: 使损失 (loss) 尽可能小
- ⑩ 一个好的的损失函数的要求
 - 预测结果越好, loss 越小
 - 考虑到数据的分布、难易程度等多个方面-->例如交叉熵损失在极度不均衡数据上表现不佳
 -

损失函数

⑩ 均以二分类为例

- n 为样本个数
- y_i 为ground truth, $y_i \in \{0, 1\}$
- x_i 为prediction, $x_i \in [0, 1]$ (在MSE中, x_i 取值可以为实数)

交叉熵 (Cross entropy)

$$CE = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot \log(x_i) + (1 - y_i) \cdot \log(1 - x_i)]$$

均方误差 (Mean squared error)

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2$$

骰子系数损失 (Dice loss)

$$DL = 1 - \frac{2 \cdot \sum_{i=1}^n x_i \cdot y_i}{\sum_{i=1}^n x_i^2 + \sum_{i=1}^n y_i^2}$$

Focal loss

$$FL = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot (1 - x_i)^{\gamma} \cdot \log(x_i) + (1 - y_i) \cdot (x_i)^{\gamma} \cdot \log(1 - x_i)]$$

最优化 (Optimize)

⑩ 有了损失函数，目标是找到合适的网络权重，使得（验证集/测试集）损失函数最小。

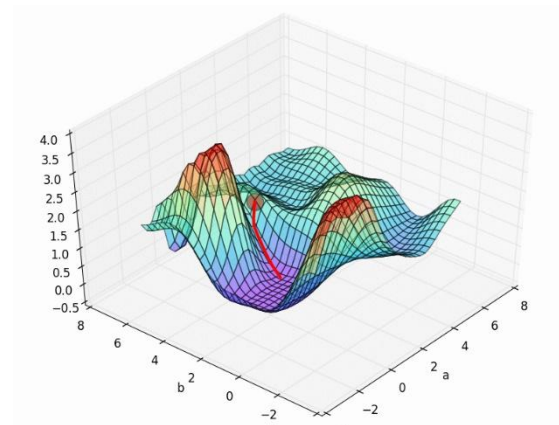
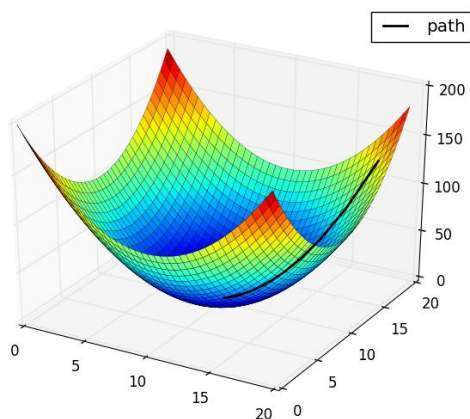
⑩ 优化的难度：

(1) 网络参数数量大都超过百万级，搜索的空间巨大

(2) 损失函数和参数的关系及其复杂

(3) 计算能力的限制，不可能一次考虑到所有的训练数据

(4) 由于过拟合的问题，即使在训练集上损失函数最小，在验证集/测试集并不一定理想

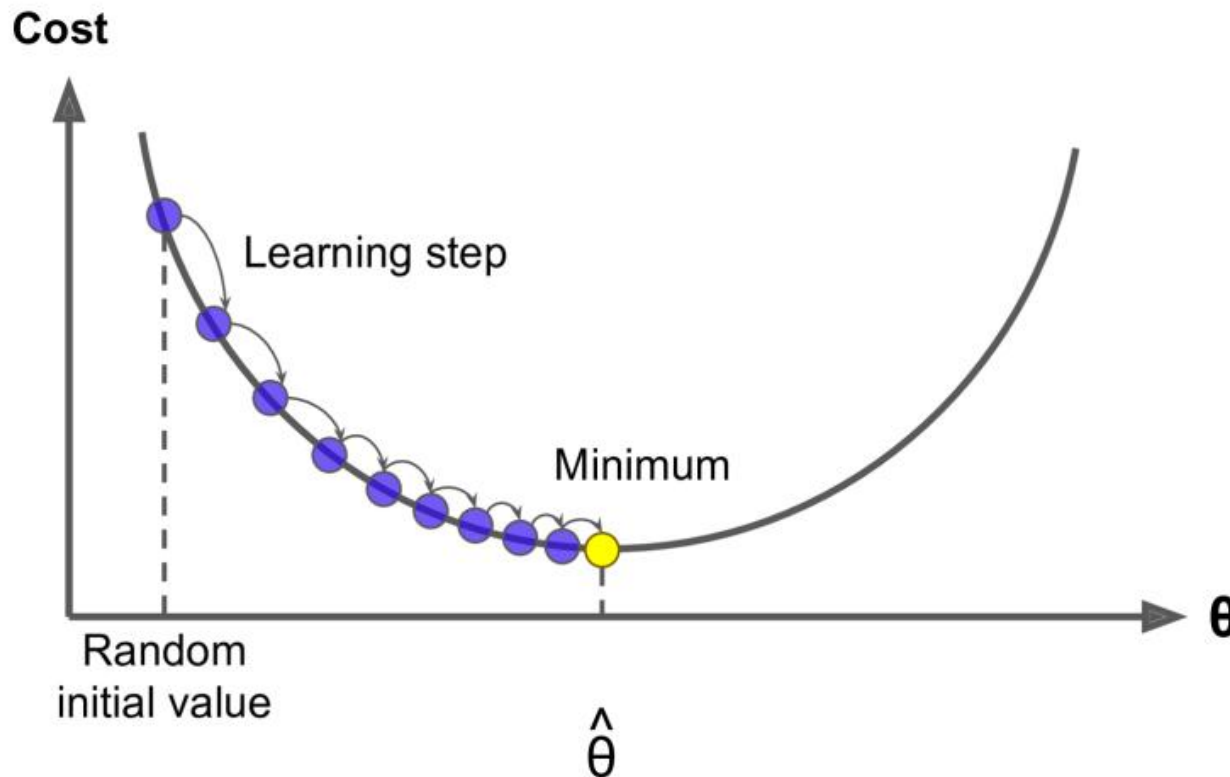


梯度下降

- 基本思想
 - 对于每一个样本，进行一个网络权重的更新
 - 更新的方向：“使loss下降最快的方向”->梯度的反方向->loss对权重求导
 - 更新的步长：学习率->超参数

Gradient Descent

- We have a cost function $J(\theta)$ we want to minimize
- **Gradient Descent** is an algorithm to minimize $J(\theta)$
- Idea: for current value of θ , calculate gradient of $J(\theta)$, then take **small step in direction of negative gradient**. Repeat.

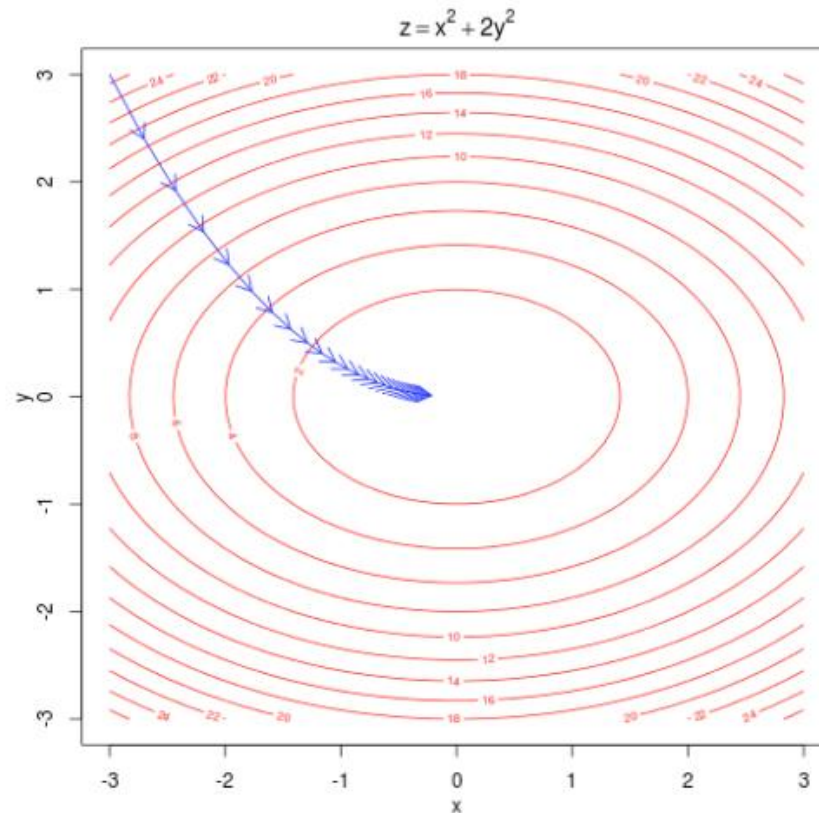


Note: Our objectives are not convex like this :(

Intuition

For a simple convex function over two parameters.

Contour lines show levels of objective function



Gradient Descent

- Update equation (in matrix notation):

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

α = *step size* or *learning rate*

- Update equation (for single parameter):

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial \theta_j^{old}} J(\theta)$$

- Algorithm:

```
while True:
    theta_grad = evaluate_gradient(J, corpus, theta)
    theta = theta - alpha * theta_grad
```

梯度下降 (GD)

<https://www.cnblogs.com/pinard/p/5970503.html>

- 批量梯度下降法 (Batch Gradient Descent)
- 随机梯度下降法 (Stochastic Gradient Descent)
- 小批量梯度下降法 (Mini-batch Gradient Descent)

梯度下降 (GD)

- <https://www.cnblogs.com/pinard/p/5970503.html>
- 批量梯度下降法 (Batch Gradient Descent)：是梯度下降法最常用的形式，具体做法也就是在更新参数时使用所有的样本来进行更新。对应的更新公式是：

$$\theta_i^{new} = \theta_i^{old} - \alpha \sum_{j=1}^m \nabla_{\theta_i} J(\theta)$$

由于有m个样本，这里求梯度的时候就用了所有m个样本的梯度数据

梯度下降 (GD)

- <https://www.cnblogs.com/pinard/p/5970503.html>
- 随机梯度下降法 (Stochastic Gradient Descent) : 其实和批量梯度下降法原理类似, 区别在与求梯度时没有用所有的 m 个样本的数据, 而是仅仅选取一个样本 j 来求梯度。对应的更新公式是:

$$\theta_i^{new} = \theta_i^{old} - \alpha \nabla_{\theta_i} J(\theta)$$

- 随机梯度下降法和批量梯度下降法是两个极端, 一个采用所有数据来梯度下降, 一个用一个样本来梯度下降。自然各自的优缺点都非常突出。对于训练速度来说, 随机梯度下降法由于每次仅仅采用一个样本来迭代, 训练速度很快, 而批量梯度下降法在样本量很大的时候, 训练速度不能让人满意。对于准确度来说, 随机梯度下降法仅仅用一个样本决定梯度方向, 导致解很有可能不是最优。对于收敛速度来说, 由于随机梯度下降法一次迭代一个样本, 导致迭代方向变化很大, 不能很快的收敛到局部最优解。
- 那么, 有没有一个中庸的办法能够结合两种方法的优点呢? 有! 这就是小批量梯度下降法

梯度下降 (GD)

- <https://www.cnblogs.com/pinard/p/5970503.html>
- 小批量梯度下降法 (Mini-batch Gradient Descent) : 小批量梯度下降法是批量梯度下降法和随机梯度下降法的折衷, 也就是对于 m 个样本, 我们采用 x 个样子来迭代, $1 < x < m$ 。一般可以取 $x=10$, 当然根据样本的数据, 可以调整这个 x 的值。对应的更新公式是:

$$\theta_i^{new} = \theta_i^{old} - \alpha \sum_{j=t}^{t+x-1} \nabla_{\theta_i} J(\theta)$$

简单导数计算

■ 常用函数的导数和链式求导法则

$$(a \cdot x^n)' = a \cdot n \cdot x^{n-1}$$

$$(e^x)' = e^x$$

$$\left(\frac{v}{u}\right)' = \frac{uv' - u'v}{u^2}$$

$$(c)' = 0$$

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

Sigmoid激活函数的求导

$$\begin{aligned}\left(\frac{1}{1+e^{f(x)}}\right)' &= \frac{0 \cdot (1+e^{f(x)}) - 1 \cdot (0+e^{f(x)})f'(x)}{(1+e^{f(x)})^2} \\&= \frac{-e^{f(x)}}{(1+e^{f(x)})^2} \cdot f'(x) \\&= -\frac{1}{1+e^{f(x)}} \cdot \frac{e^{f(x)}}{1+e^{f(x)}} \cdot f'(x) \\&= -\frac{1}{1+e^{f(x)}} \cdot \left(1 - \frac{1}{1+e^{f(x)}}\right) \cdot f'(x)\end{aligned}$$

后向传播修改权值和偏置

$$J(\theta) = \frac{1}{2}(T_6 - O_6)^2$$

$$\begin{aligned}\nabla_{\theta_6} &= \frac{\partial J(\theta)}{\partial \theta_6} = \frac{1}{2} \cdot 2 \cdot (T_6 - O_6) \cdot (T_6 - O_6)' \\ &= (T_6 - O_6) \cdot \left(T_6 - \frac{1}{1 + e^{-I_6}} \right)' \\ &= (T_6 - O_6) \cdot \left(0 + \frac{1}{1 + e^{-I_6}} \cdot \left(1 - \frac{1}{1 + e^{-I_6}} \right) \cdot (-I_6)' \right) \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot (-W_{46}O_4 - W_{56}O_5 - \theta_6)' \\ &= -(T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \\ &= -\text{err}_6\end{aligned}$$

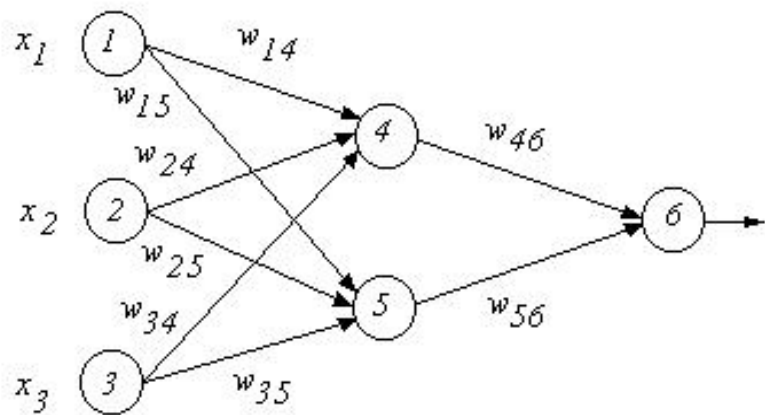
$$\theta_6' = \theta_6 - \ell \nabla \theta_6 = \theta_6 + \ell \cdot \text{err}_6$$

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (\ell) Err_j$$

$$w_{ij} = w_{ij} + (\ell) Err_j O_i$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$



$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (l)Err_j$$

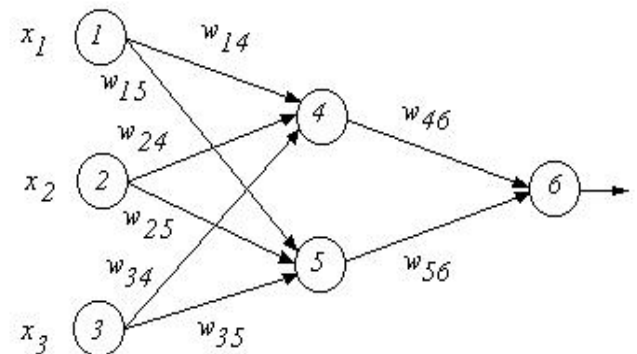
$$w_{ij} = w_{ij} + (l)Err_j O_i$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$J(\theta) = \frac{1}{2}(T_6 - O_6)^2$$

$$\begin{aligned} \nabla_{W_{46}} &= \frac{\partial J(\theta)}{\partial W_{46}} = \frac{1}{2} \cdot 2 \cdot (T_6 - O_6) \cdot (T_6 - O_6)' \\ &= (T_6 - O_6) \cdot \left(T_6 - \frac{1}{1 + e^{-I_6}} \right)' \\ &= (T_6 - O_6) \cdot \left(0 + \frac{1}{1 + e^{-I_6}} \cdot \left(1 - \frac{1}{1 + e^{-I_6}} \right) \cdot (-I_6)' \right) \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot (-W_{46}O_4 - W_{56}O_5 - \theta_6)' \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot (-O_4) \\ &= -err_6 \cdot O_4 \end{aligned}$$

$$W'_{46} = W_{46} - \ell \cdot \nabla W_{46} = W_{46} + \ell \cdot err_6 \cdot O_4$$



$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (l)Err_j$$

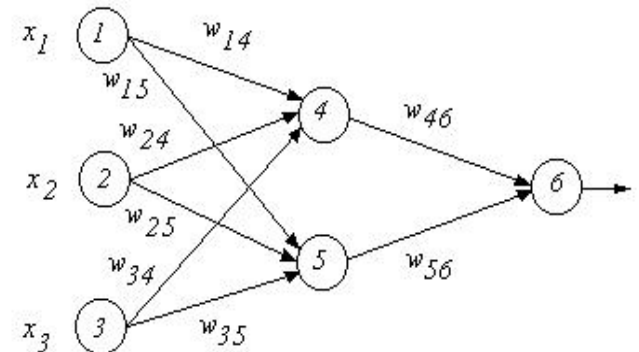
$$w_{ij} = w_{ij} + (l)Err_j O_i$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$J(\theta) = \frac{1}{2}(T_6 - O_6)^2$$

$$\begin{aligned} \nabla_{W_{14}} = \frac{\partial J(\theta)}{\partial W_{14}} &= \frac{1}{2} \cdot 2 \cdot (T_6 - O_6) \cdot (T_6 - O_6)' \\ &= (T_6 - O_6) \cdot \left(T_6 - \frac{1}{1 + e^{-I_6}} \right)' \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot (-I_6)' \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot \left(-W_{46} \cdot \frac{1}{1 + e^{-I_4}} - W_{56}O_5 - \theta_6 \right)' \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot \left(-W_{46} \cdot \frac{1}{1 + e^{-I_4}} \right)' \\ &= (T_6 - O_6) \cdot O_6 \cdot (1 - O_6) \cdot (-W_{46}) \cdot \left(-\frac{1}{1 + e^{-I_4}} \cdot \left(1 - \frac{1}{1 + e^{-I_4}} \right) \cdot (-I_4)' \right) \\ &= err_6 \cdot (-W_{46}) \cdot (-O_4) \cdot (1 - O_4) \cdot (-W_{14}O_1 - W_{24}O_2 - W_{34}O_3 - \theta_4)' \\ &= O_4 \cdot (1 - O_4) \cdot W_{46} \cdot err_6 \cdot (-O_1) \\ &= err_4 \cdot (-O_1) \end{aligned}$$

$$W'_{14} = W_{14} - \ell \cdot \nabla W_{14} = W_{14} + \ell \cdot err_4 \cdot O_1$$



Example

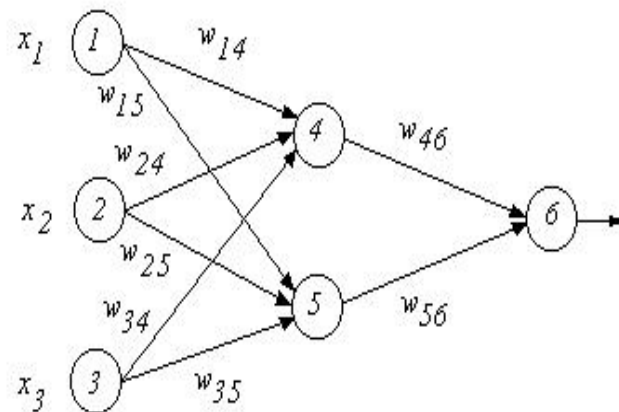
- 设学习率为0.9。训练样本 $X = \{1,0,1\}$ 类标号为1

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

单元j	净输入I _j	输出O _j
4	$0.2+0-0.5-0.4 = -0.7$	$1/(1+e^{0.7}) = 0.332$
5	$-0.3+0+0.2+0.2 = 0.1$	$1/(1+e^{-0.1}) = 0.525$
6	$(-0.3)(0.332)-(0.2)(0.525)+0.1 = -0.105$	$1/(1+e^{0.105}) = 0.474$

单元j	Err _j
6	$(0.474)(1-0.474)(1-0.474) = 0.1311$
5	$(0.525)(1-0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1-0.332)(0.1311)(-0.3) = -0.02087$

权或偏差	新值
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
θ_6	$0.1 + (0.9)(0.1311) = 0.218$
θ_5	$0.2 + (0.9)(-0.0065) = 0.194$
θ_4	$-0.4 + (0.9)(-0.0087) = -0.408$



$$I_j = \sum_i w_{ij} O_i + \theta_j$$

$$O_j = \frac{1}{1 + e^{-I_j}}$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$w_{ij} = w_{ij} + (l) Err_j O_i$$

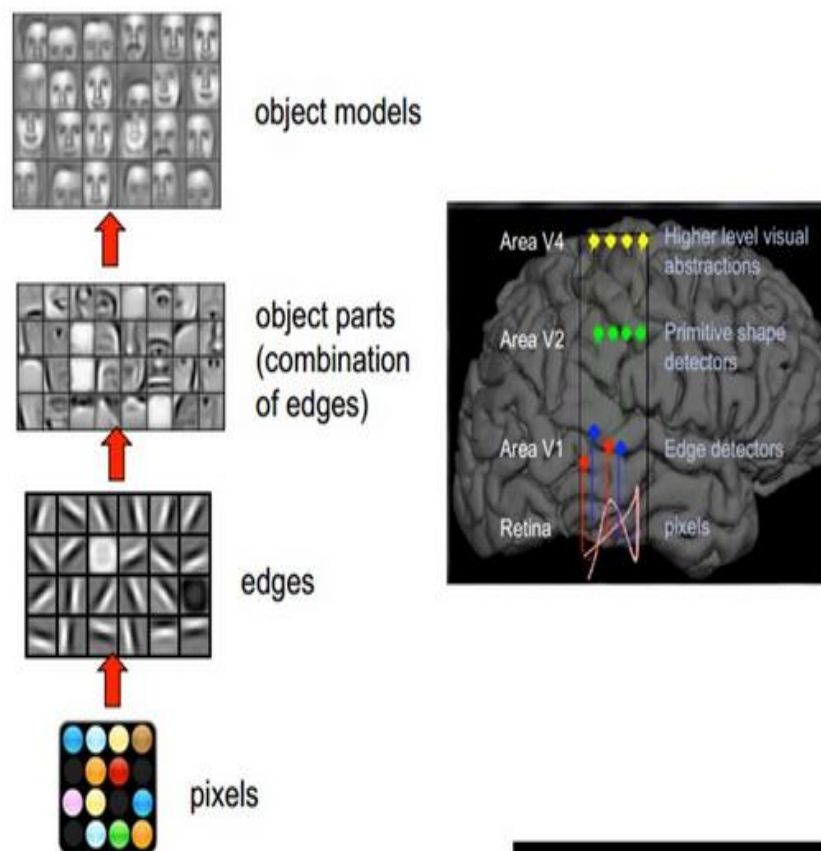
$$\theta_j = \theta_j + (l) Err_j$$

人类视觉原理

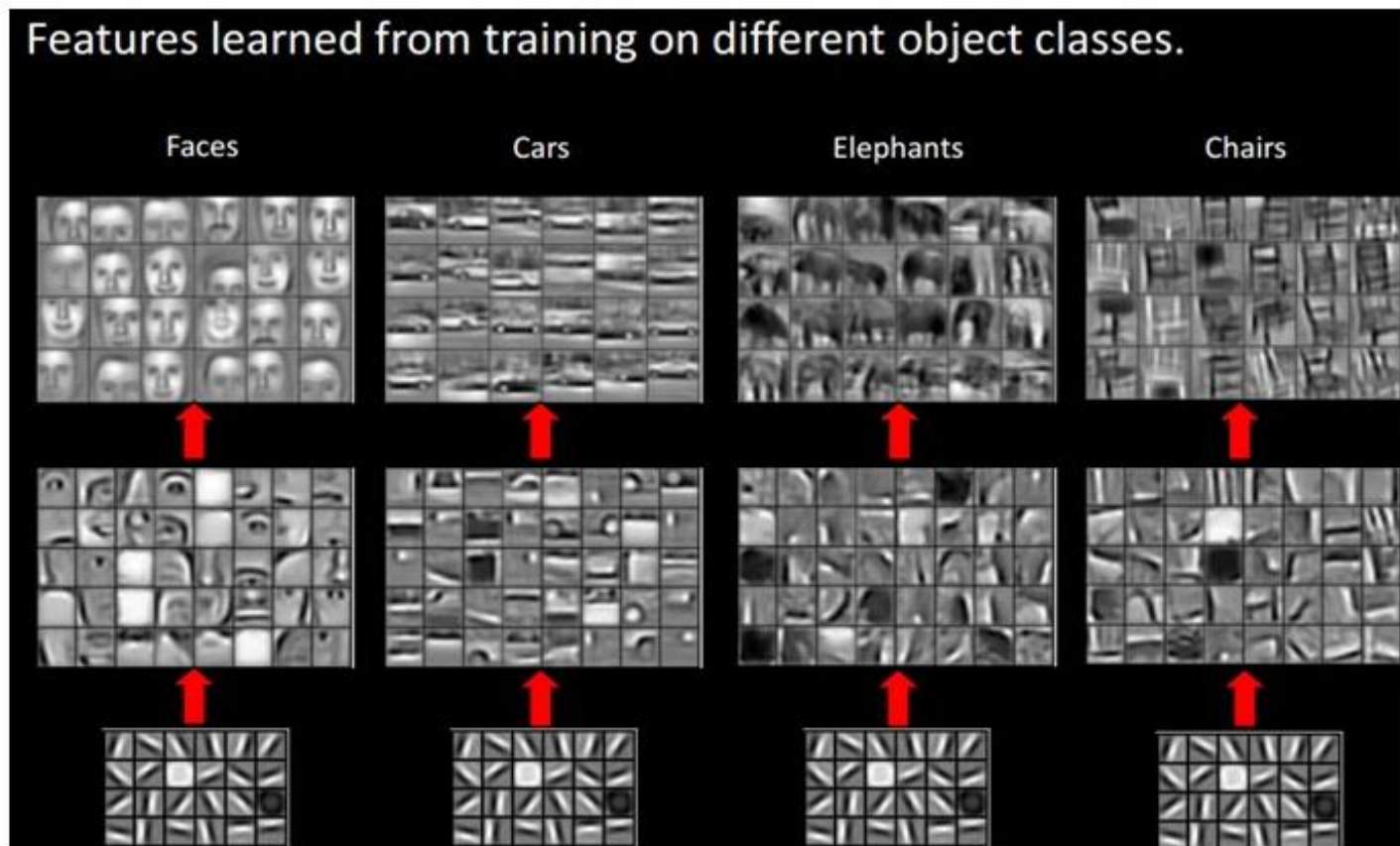
- 深度学习的许多研究成果，离不开对大脑认知原理的研究，尤其是视觉原理的研究
- 1981 年的诺贝尔医学奖，颁发给了 David Hubel（出生于加拿大的美国神经生物学家）和Torsten Wiesel，以及 Roger Sperry。前两位的主要贡献，是“发现了视觉系统的信息处理”，**可视皮层是分级的。**

CNN

- 人类的视觉原理如下：
从原始信号摄入开始（瞳孔摄入像素 **Pixels**），接着做初步处理（大脑皮层某些细胞发现边缘和方向），然后抽象（大脑判定，眼前的物体的形状，是圆形的），然后进一步抽象（大脑进一步判定该物体是只气球）。



- 对于不同的物体，人类视觉也是通过这样逐层分级，来进行认知的：



- 可以看到，在最底层特征基本上是类似的，就是各种边缘，越往上，越能提取出此类物体的一些特征（轮子、眼睛、躯干等），到最上层，不同的高级特征最终组合成相应的图像，从而能够让人类准确的区分不同的物体。

CNN

- 可以很自然的想到：可以不可以模仿人类大脑的这个特点，构造多层的神经网络，较低层的识别初级的图像特征，若干底层特征组成更上一层特征，最终通过多个层级的组合，最终在顶层做出分类呢？答案是肯定的，这也是许多深度学习算法（包括CNN）的灵感来源。

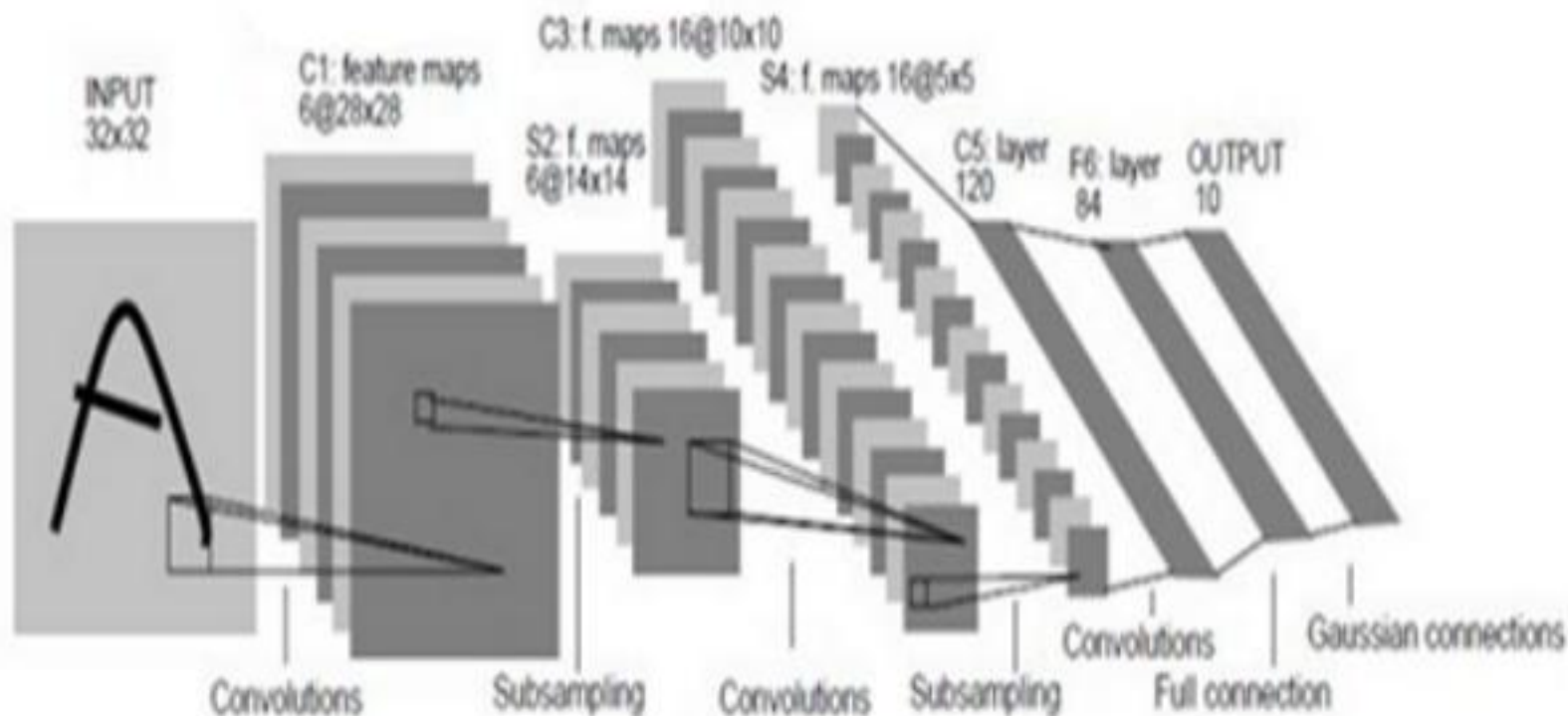
What is CNN?

- CNN is a specialized kind of neural network for processing data that has a known, grid-like topology.
- CNN is usually used to process 1D time-series and 2D image data.
- CNN is the first really successful deep network!
- CNN is simply a neural network that replaces general matrix multiplication with convolution.

CNN

- 卷积神经网络是一种多层神经网络，擅长处理图像特别是大图像的相关机器学习问题。
- 卷积网络通过一系列方法，成功将数据量庞大的图像识别问题不断降维，最终使其能够被训练。
- CNN最早由Yann LeCun提出并应用在手写字体识别上。

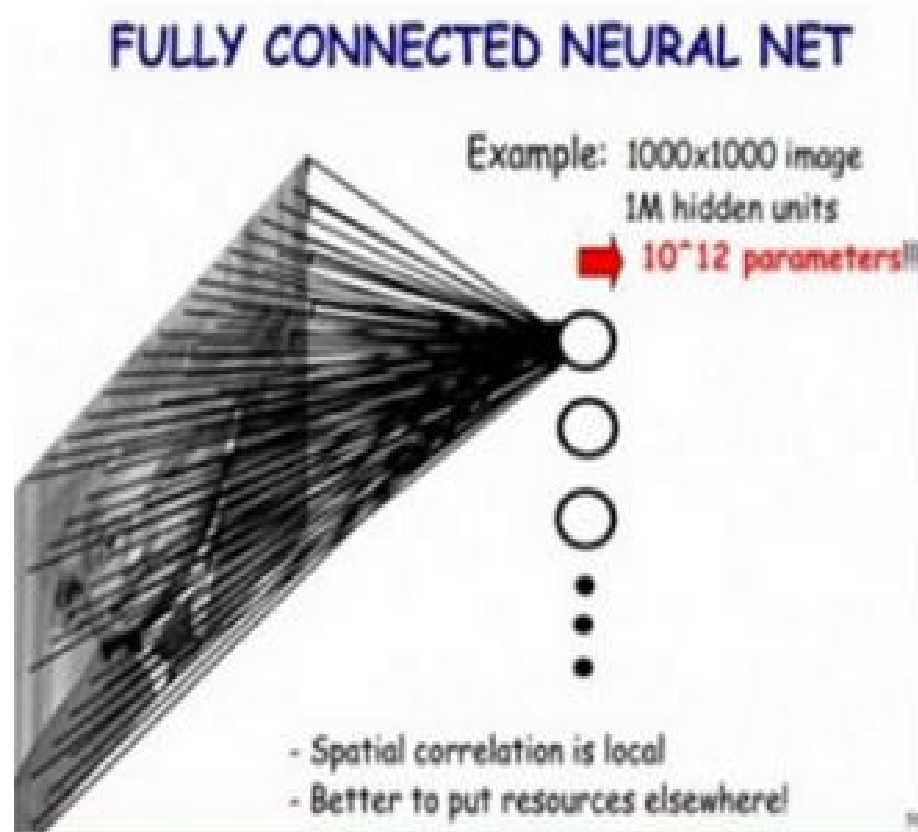
- LeCun提出的网络称为LeNet，其网络结构如下：



- 这是一个最典型的卷积网络，由卷积层、池化层、全连接层组成。其中卷积层与池化层配合，组成多个卷积组，逐层提取特征，最终通过若干个全连接层完成分类。
- CNN通过卷积来模拟特征区分，并且通过卷积的权值共享及池化，来降低网络参数的数量级，最后通过传统神经网络完成分类等任务。

为什么要降低参数量级？

- 如果使用传统神经网络方式，对一张图片进行分类，那么，把图片的每个像素都连接到隐藏层节点上，对于一张1000x1000像素的图片，如果有1M隐藏层单元，一共有 10^{12} 个参数，这显然是不能接受的。



CNN

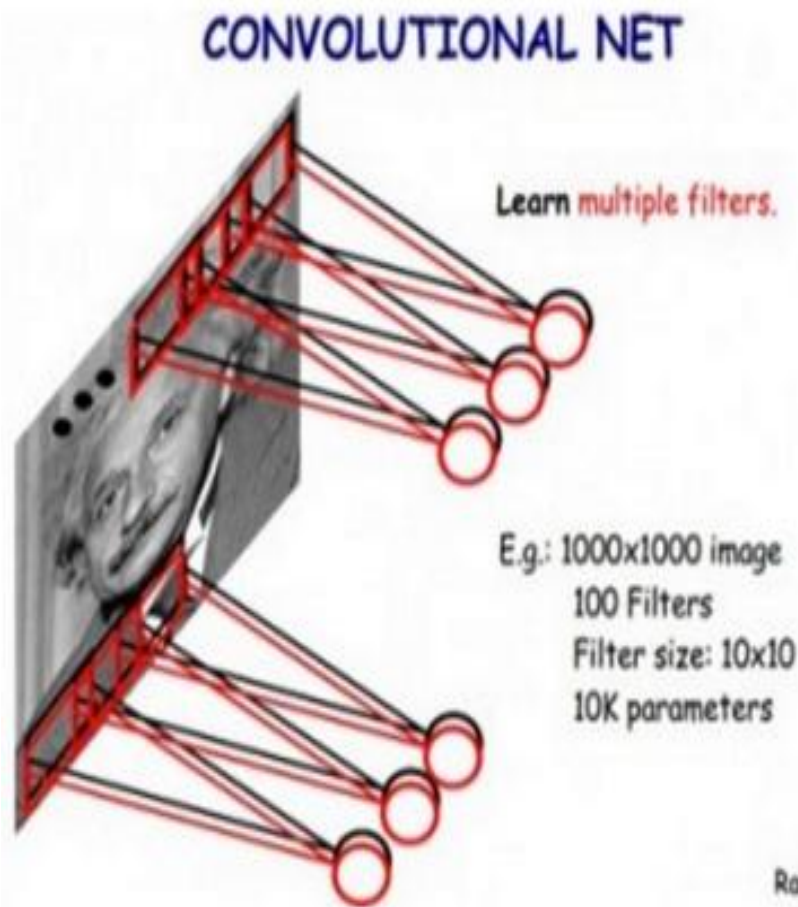
- 但是在CNN里，可以大大减少参数个数，基于以下两个假设：

- 1) 最底层特征都是局部性的，也就是说，用 10×10 这样大小的过滤器就能表示边缘等底层特征

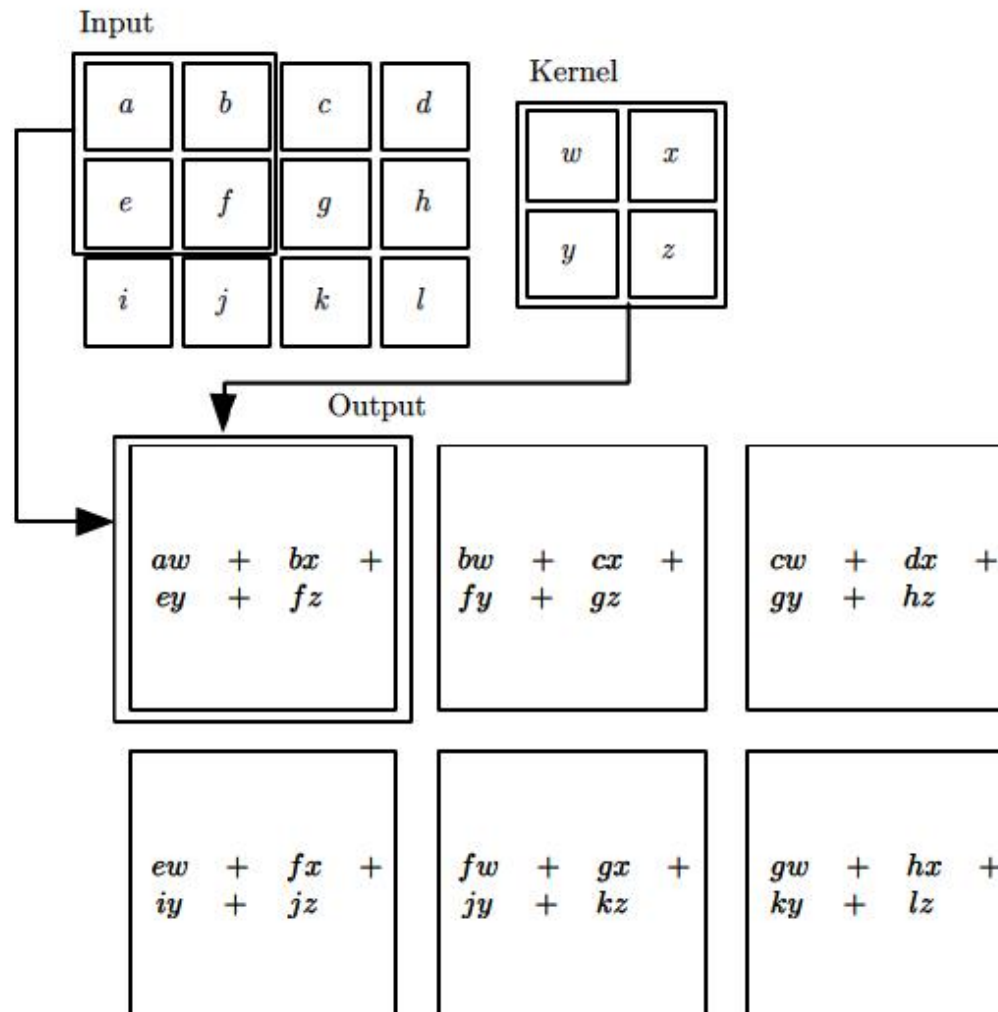
- 2) 图像上不同小片段，以及不同图像上的小片段的特征是类似的，也就是说，能用同样的一组分类器来描述各种各样不同的图像

- 基于以上两个假设，就能把第一层网络结构简化

- 用100个 10×10 的小过滤器，就能够描述整幅图片上的底层特征。

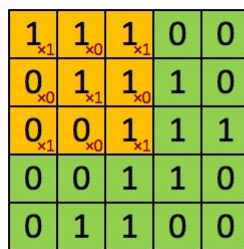
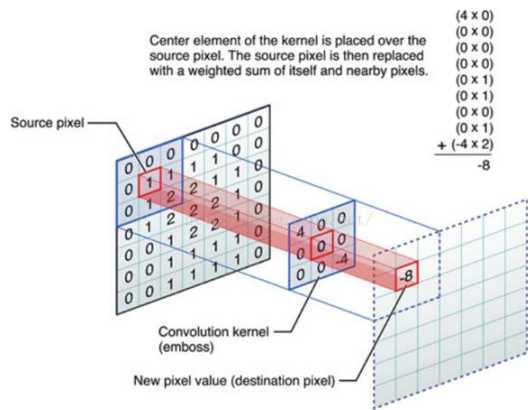


An Example of 2-D Convolution

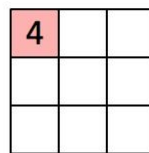


10 卷积运算

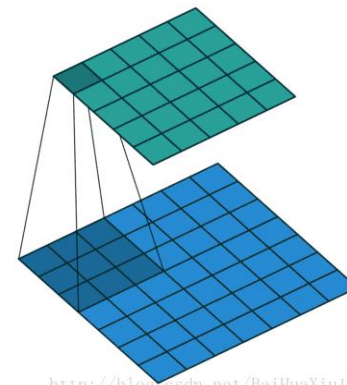
- 用一个固定大小的卷积核扫过输入图，将输入特征图（feature map）上对应位置和卷积核上进行内积运算（element-wise相乘再求和）



Image



Convolved Feature



<http://blog.csdn.net/BaiHuaXiul23>

- 目前有很多人为设计好的卷积核

0	0	0
0	1	0
0	0	0

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

卷积 (Convolution)

- 卷积运算的定义如下图所示：

1	1	1	0	0
0	1	1	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0 _{x0}	0 _{x1}	1 _{x0}	1	0
0 _{x1}	1 _{x0}	1 _{x1}	0	0

Image

4	3	4
2	4	3
2		

Convolved
Feature

卷积 (Convolution)

- 如上图所示，一个5x5的图像，用一个3x3的卷积核：

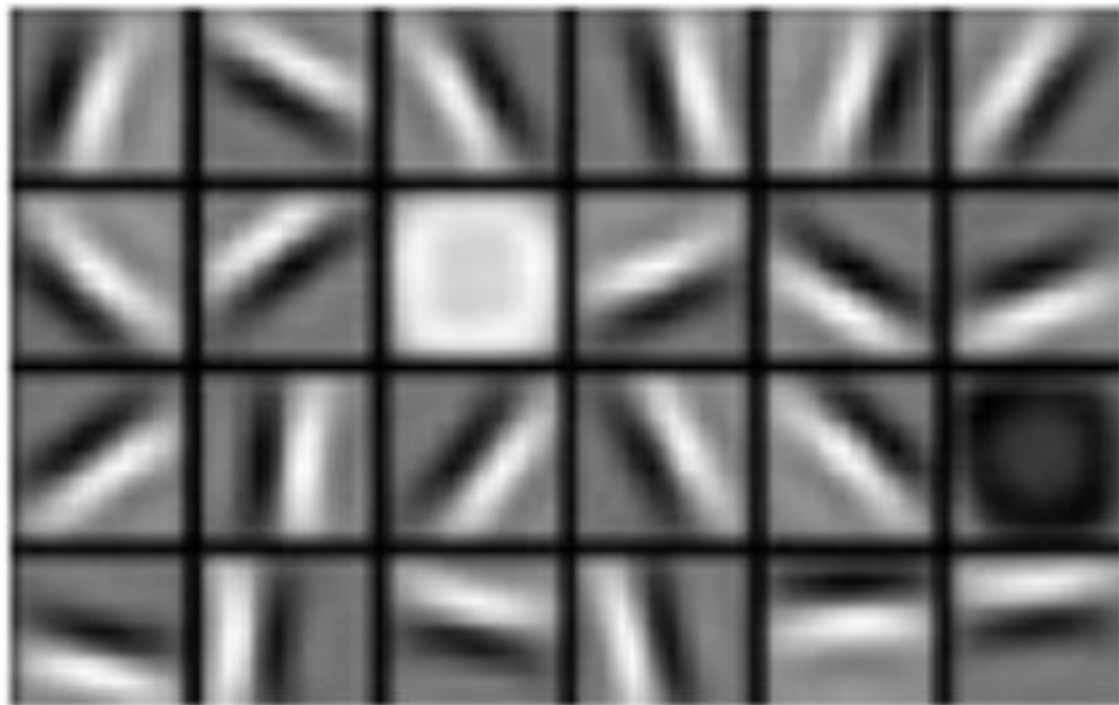
1	0	1
0	1	0
1	0	1

来对图像进行卷积操作（可以理解为有一个滑动窗口，把卷积核与对应的图像像素做乘积然后求和），得到了3x3的卷积结果。

卷积 (Convolution)

- 这个过程可以理解为使用一个过滤器（卷积核）来过滤图像的各个小区域，从而得到这些小区域的特征值
- 在实际训练过程中，卷积核的值是在学习过程中学到的
- 在具体应用中，往往有多个卷积核，可以认为，每个卷积核代表了一种图像模式，如果某个图像块与此卷积核卷积出的值大，则认为此图像块十分接近于此卷积核。如果设计了6个卷积核，可以理解为这个图像上有6种底层纹理模式，也就是用6种基础模式就能描绘出一副图像

-
- 以下就是24种不同的卷积核的示例：



⑩ 卷积核大小 (kernel size)

- 决定一个点的输出值与多大的范围的输入有关->感受野 (respective field)
- 通常是奇数，因为奇数大小的核，才能保证中心点四周的长度是相等的。
- 通常卷积核越大效果越好，但增大卷积核伴随着参数个数平方增长。

⑩ 填充 (padding)

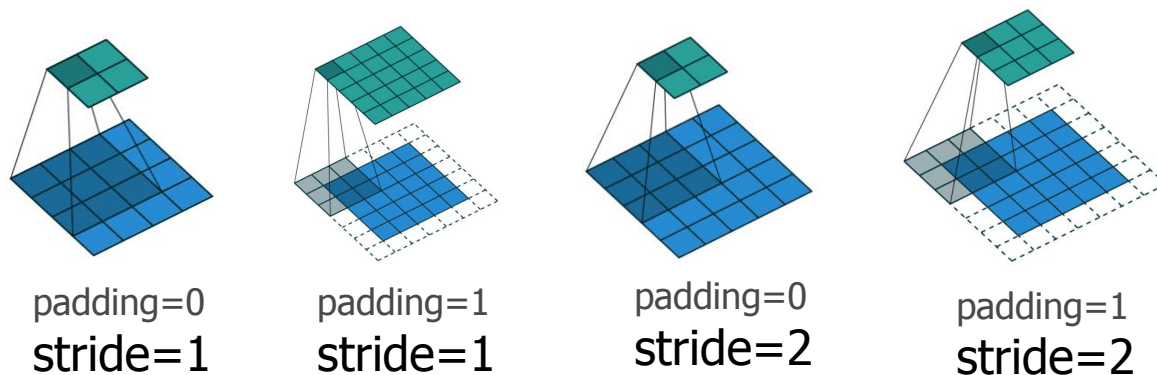
- 目的是不减小feature map的大小
- 常见策略：周边补0；向外对称。

⑩ 步长 (stride)

- 用于降低feature maps的尺寸

⑩ 卷积中的常用参数

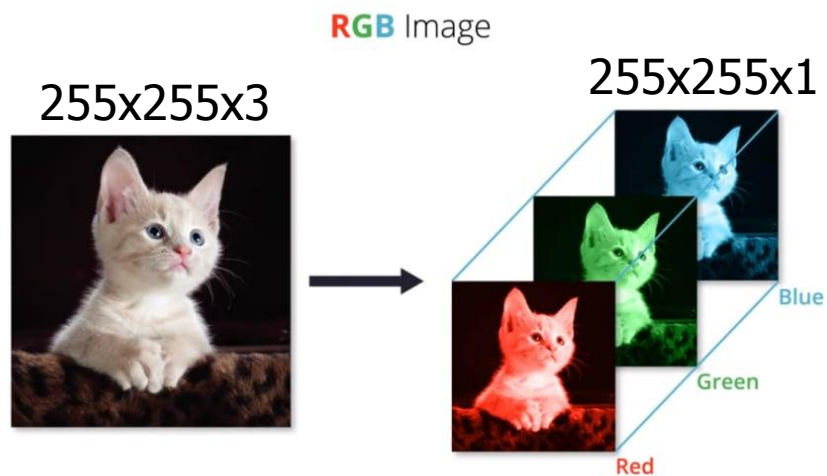
- 输入输出通道数：输入通道数通常固定，就是上一层输出的通道数
- 卷积核大小 (kernel size)
- 填充 (padding)
- 步长 (stride)



各种卷积操作的示意

https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md

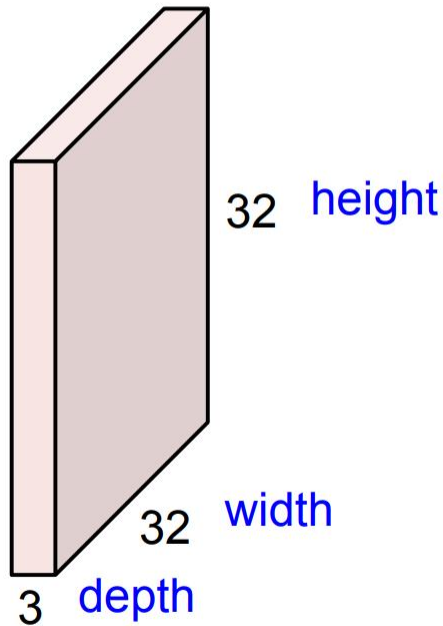
图像表示



数字图像的RGB表示

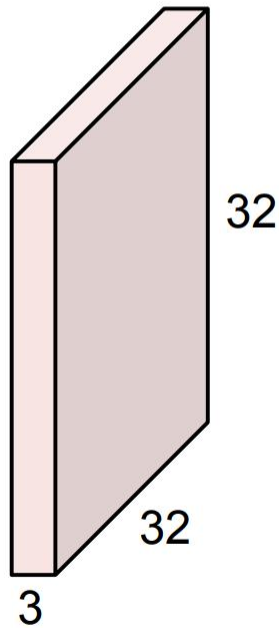
Convolution Layer

32x32x3 image -> preserve spatial structure



Convolution Layer

32x32x3 image



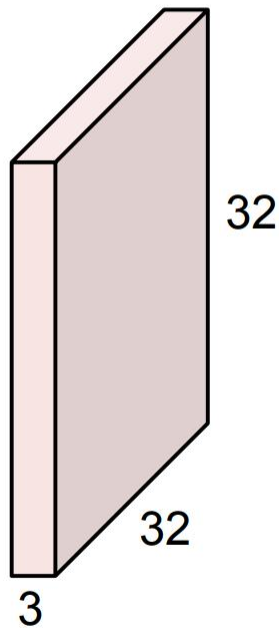
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

32x32x3 image



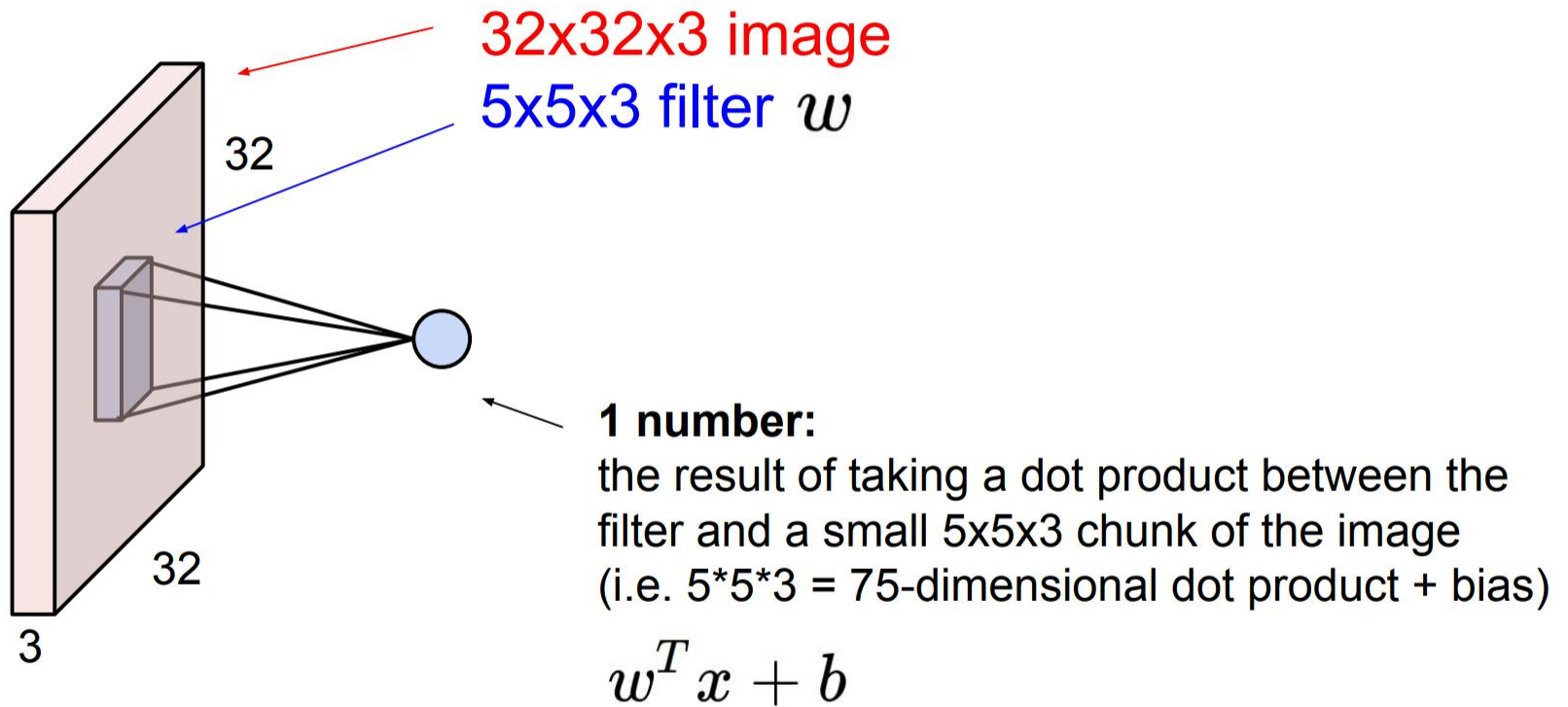
Filters always extend the full depth of the input volume

5x5x3 filter

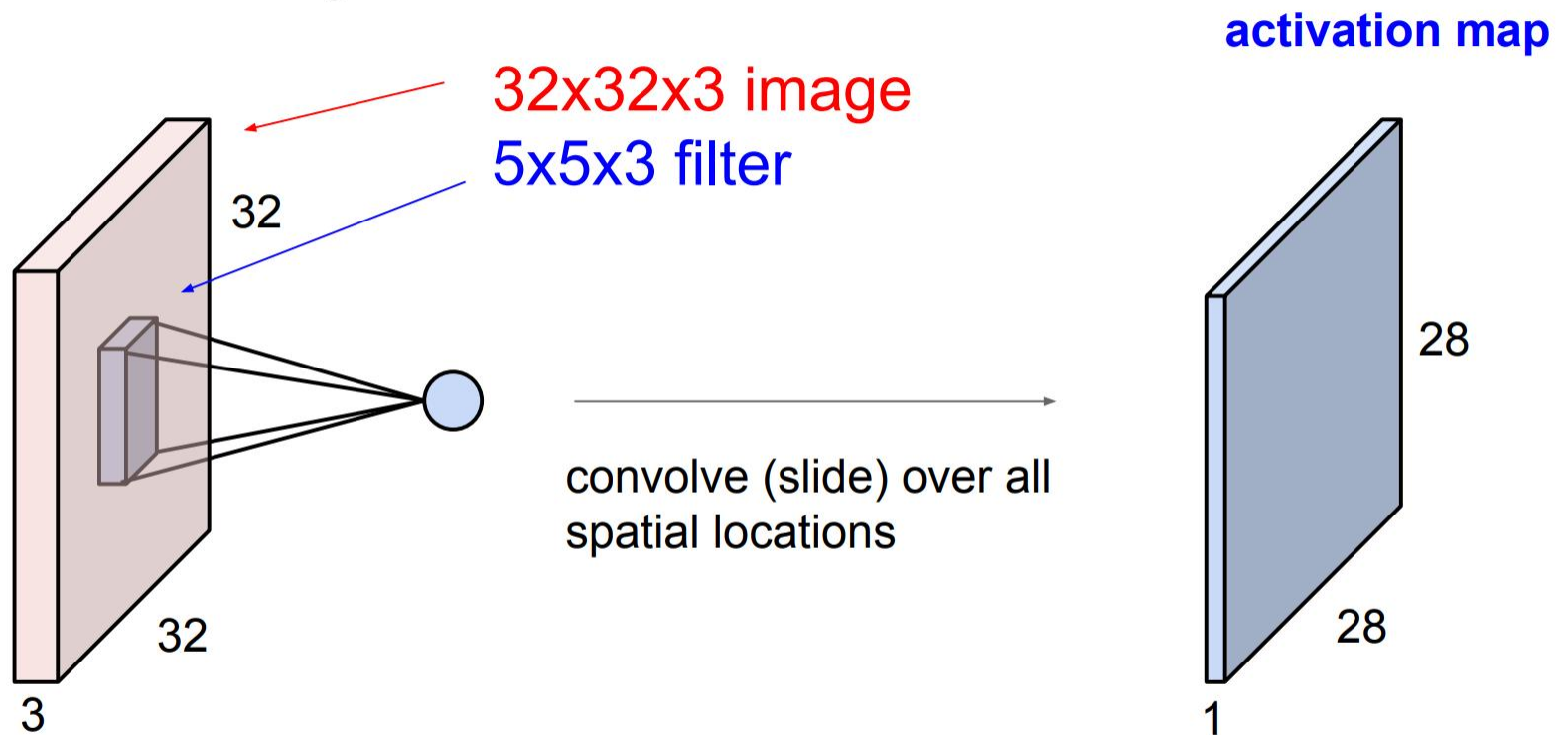


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

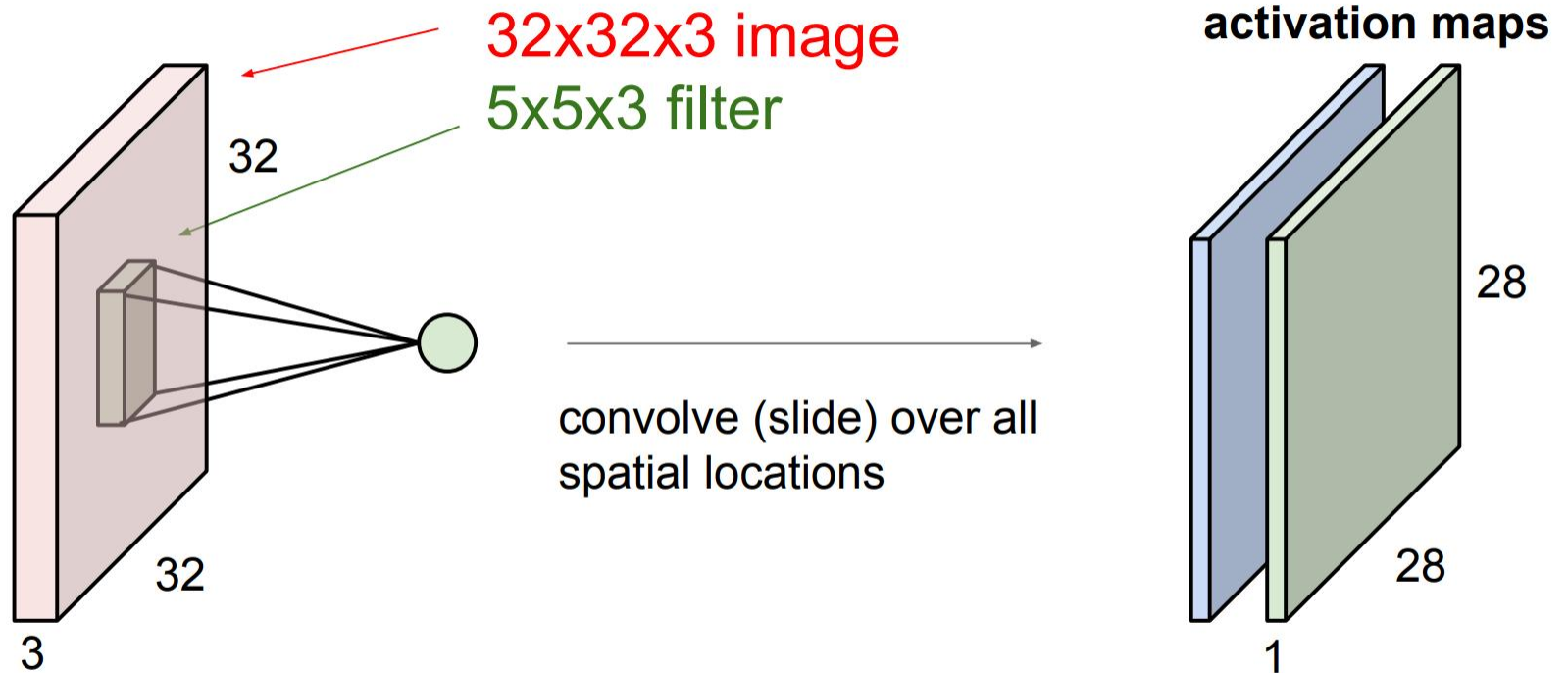


Convolution Layer

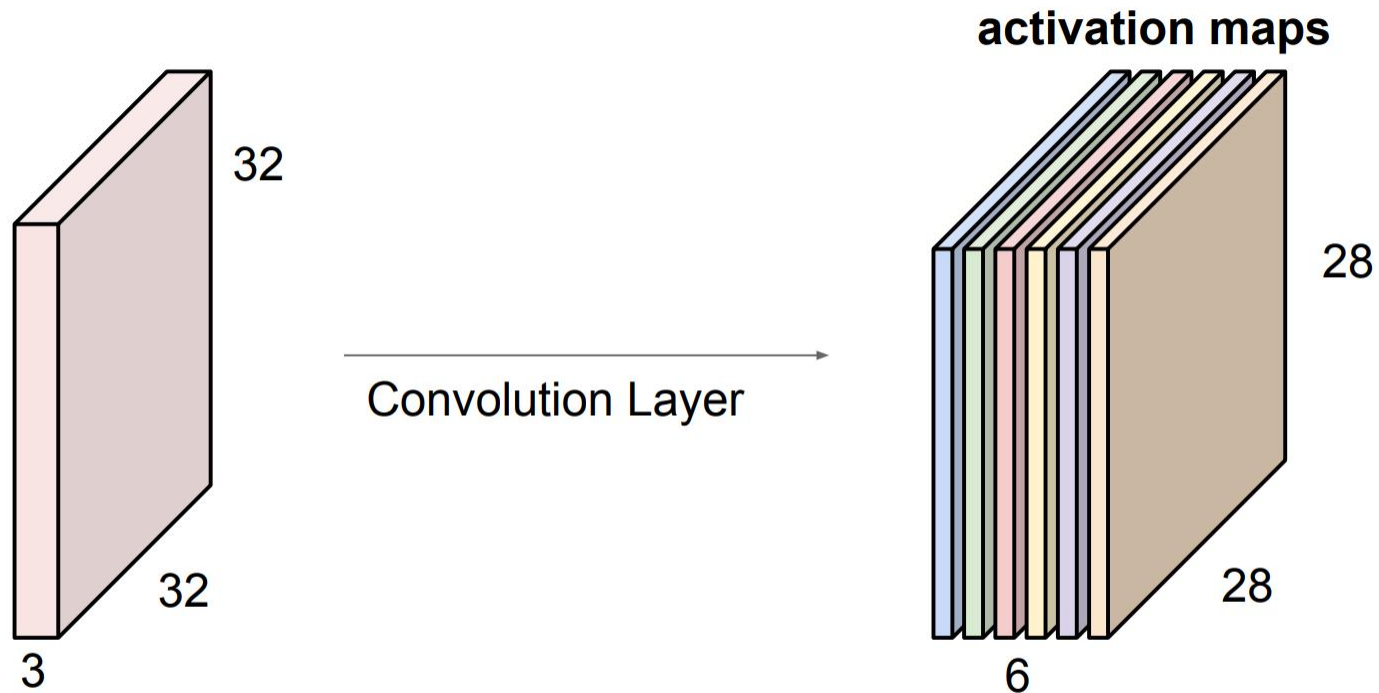


Convolution Layer

consider a second, **green** filter

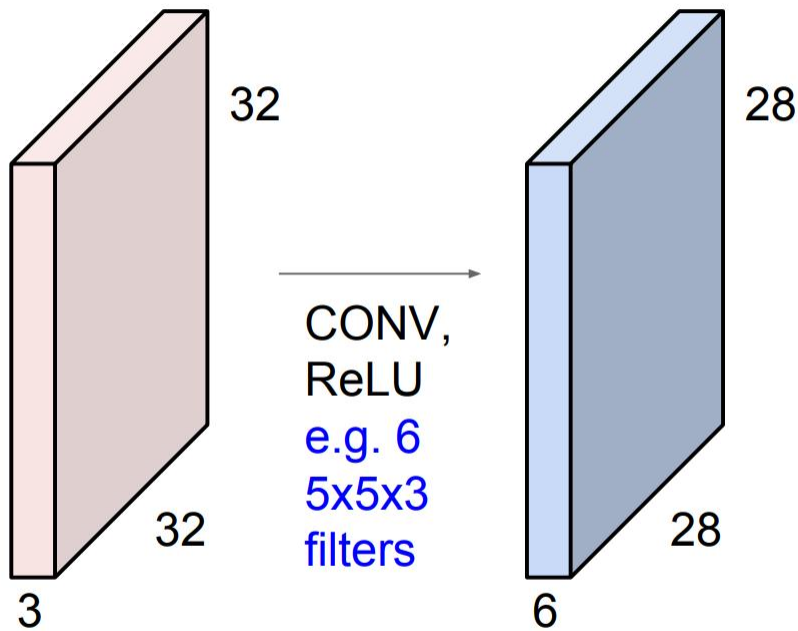


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

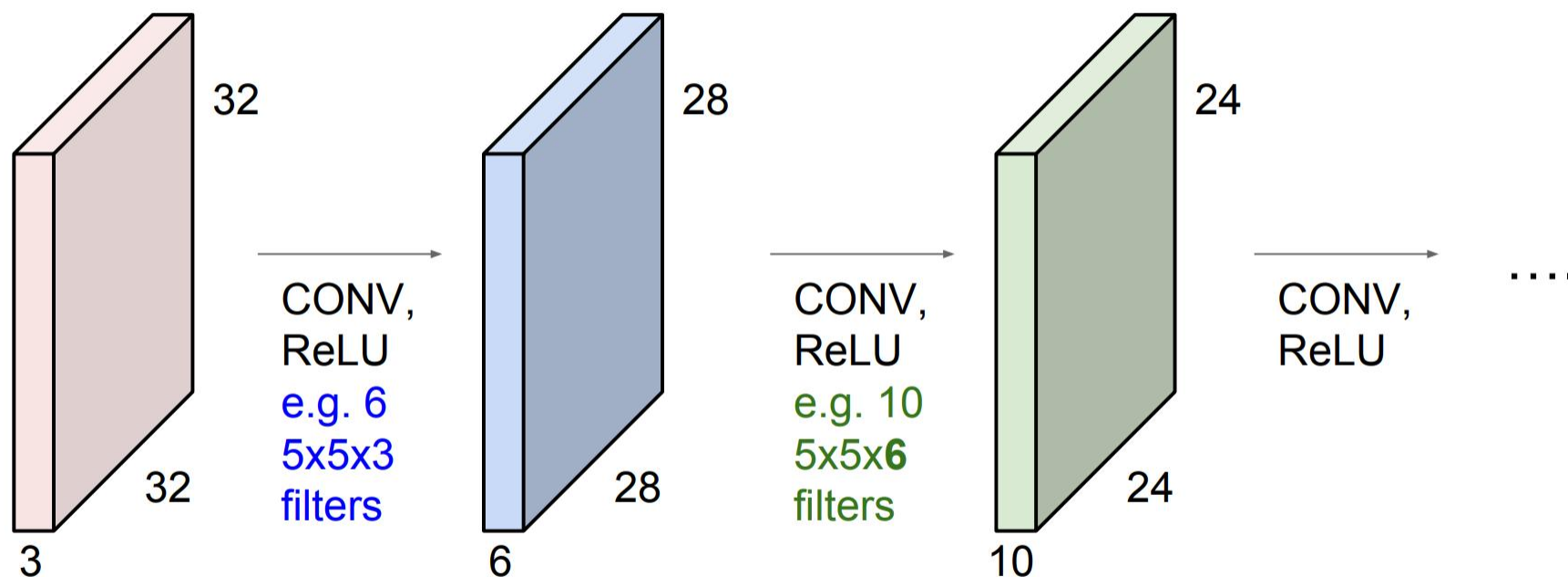


We stack these up to get a “new image” of size 28x28x6!

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



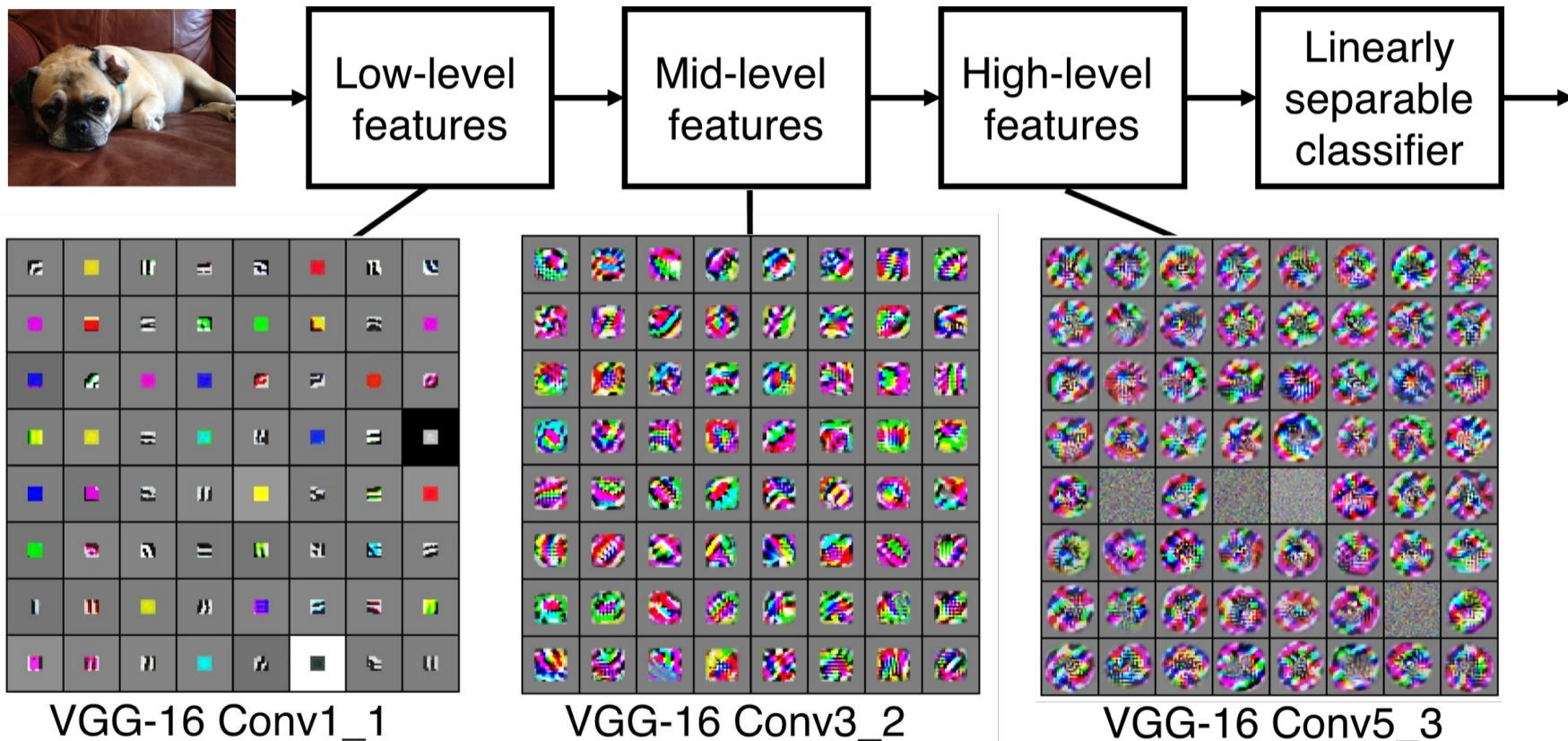
Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

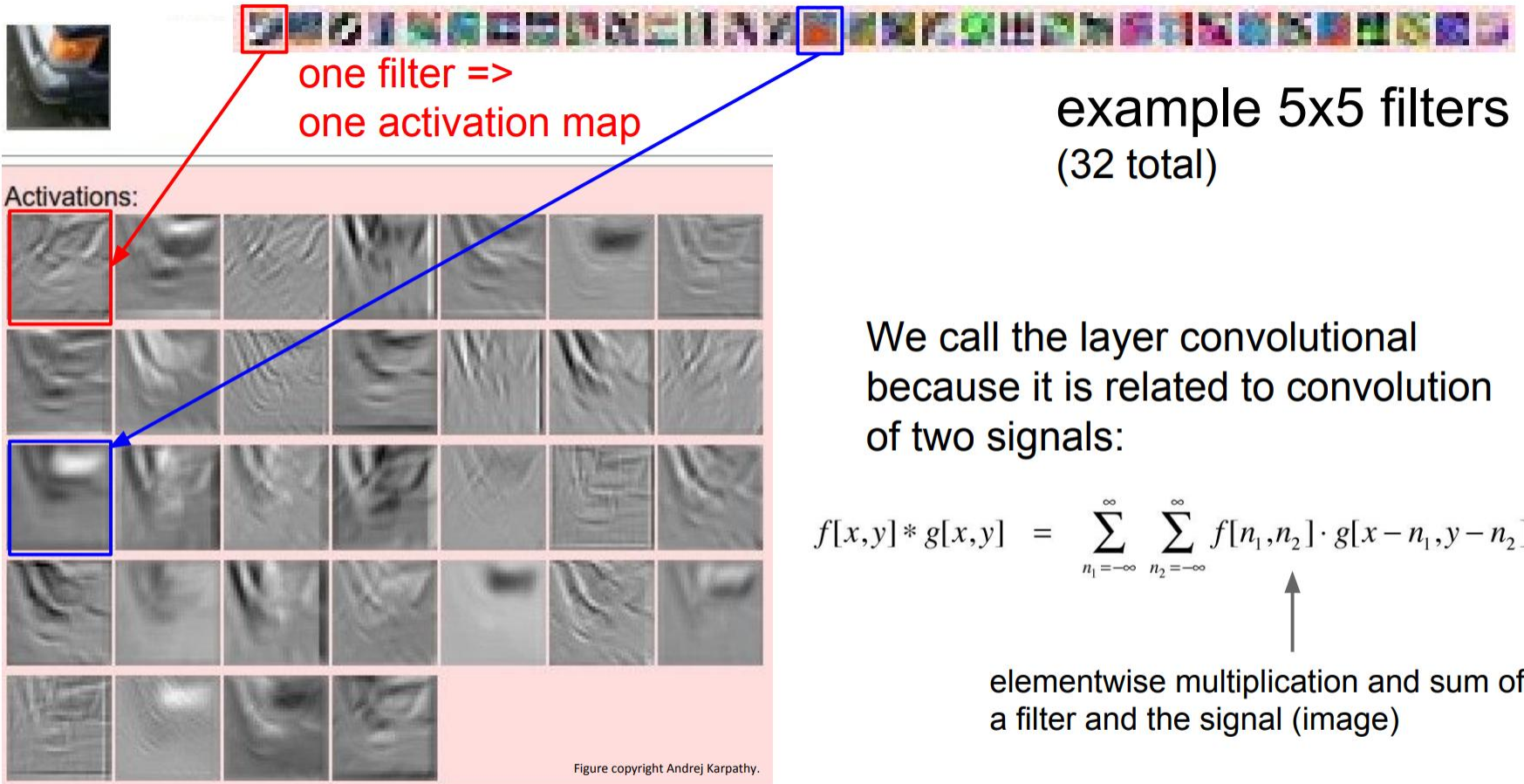


Preview

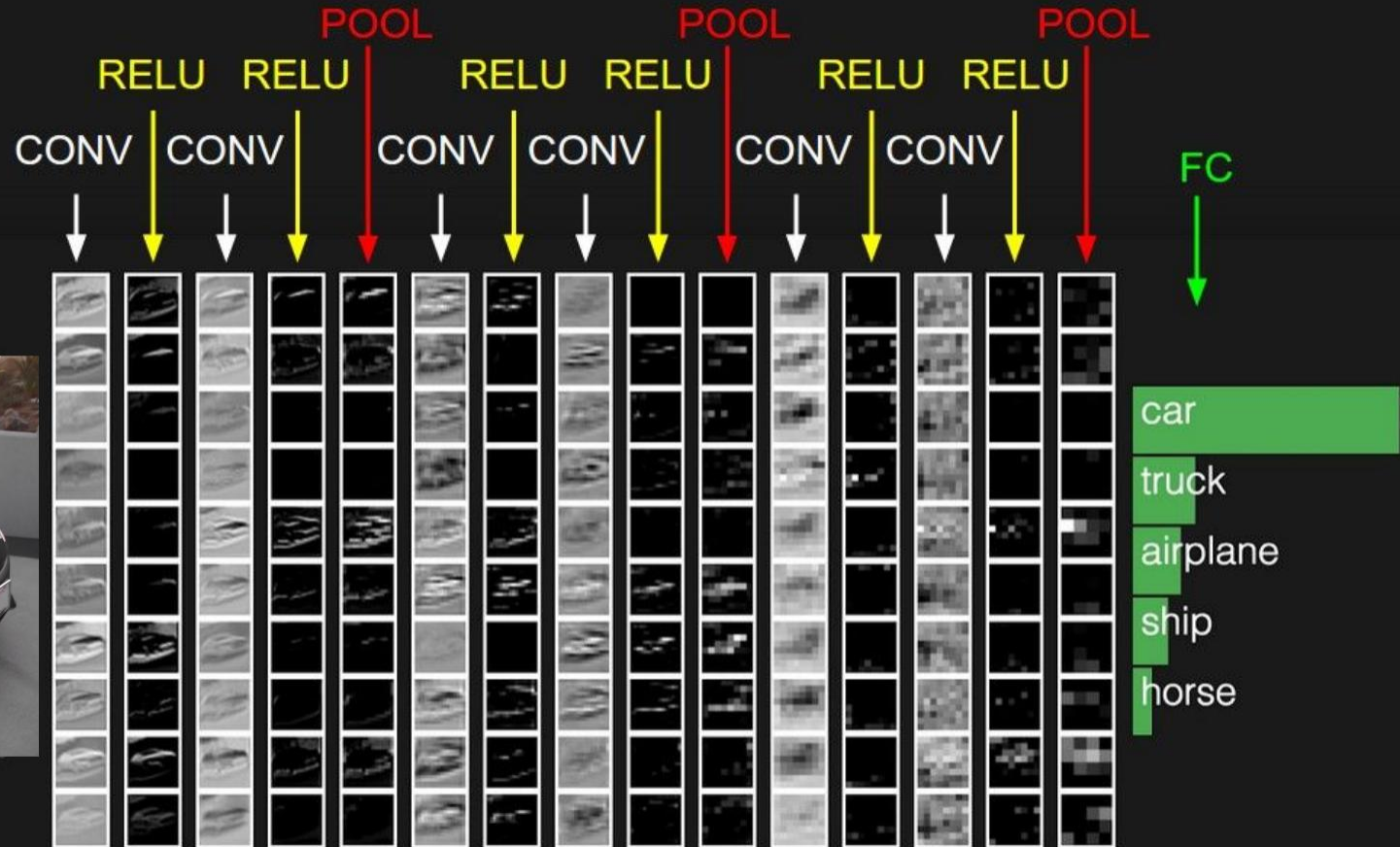
[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

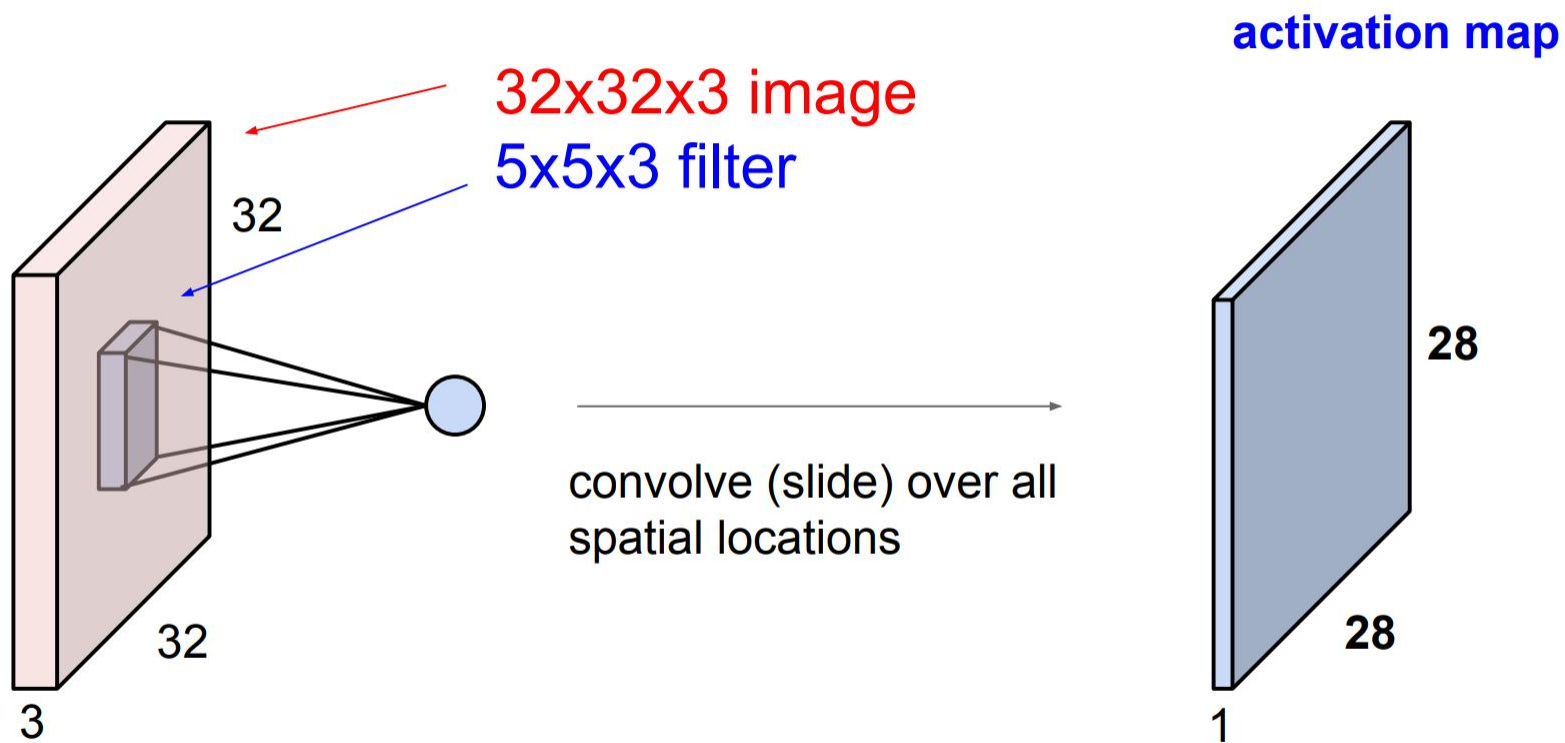




A black Mazda3 is parked on a rooftop terrace. The car is positioned at an angle, showing its front and side. The background features a concrete wall and a view of a city with hills and buildings under a clear sky.

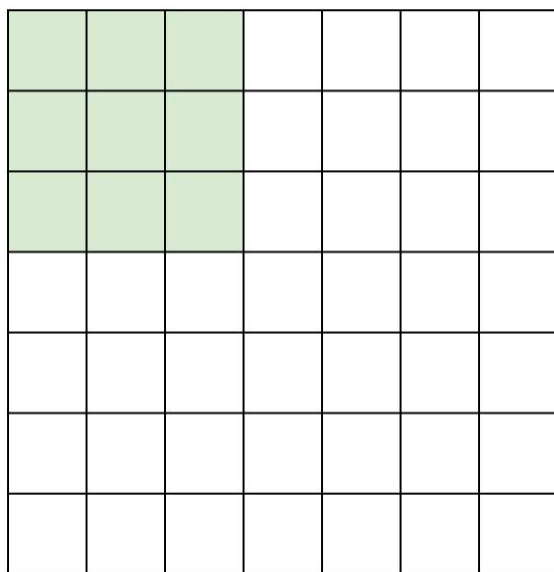


A closer look at spatial dimensions:



A closer look at spatial dimensions:

7

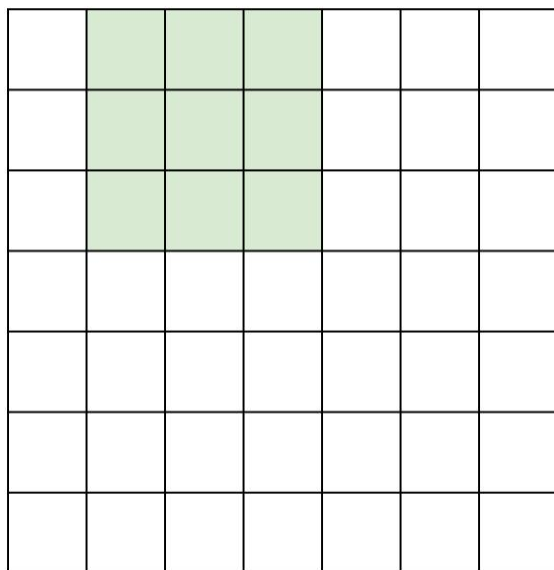


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

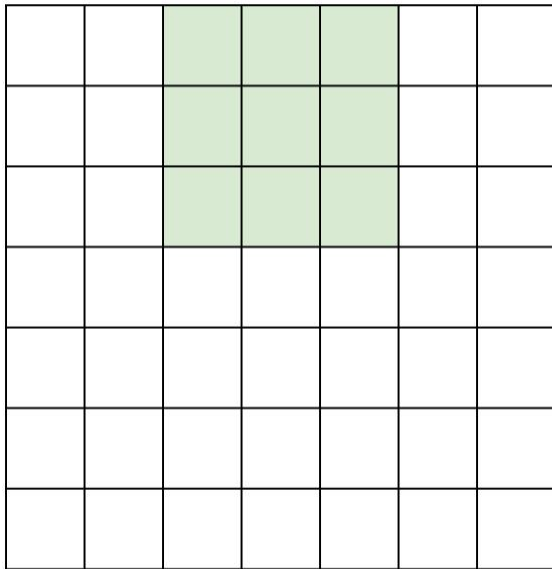


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

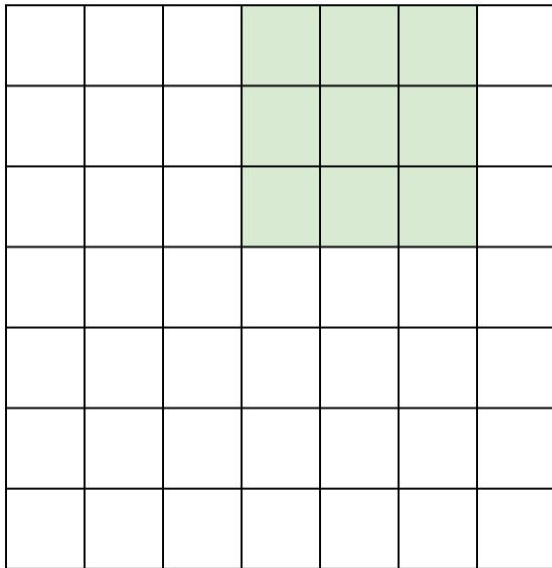


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

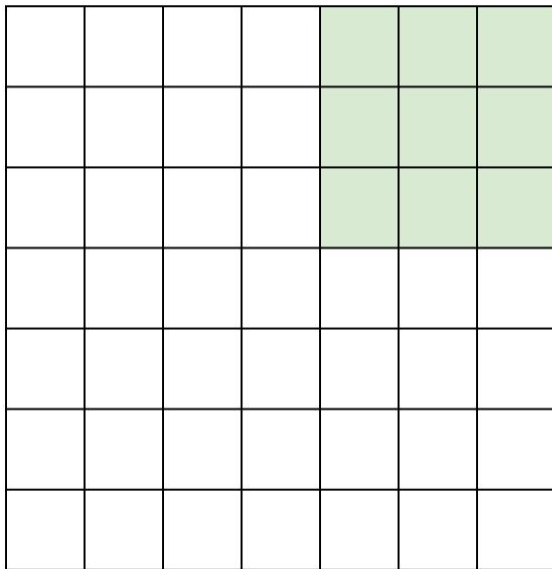


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions:

7

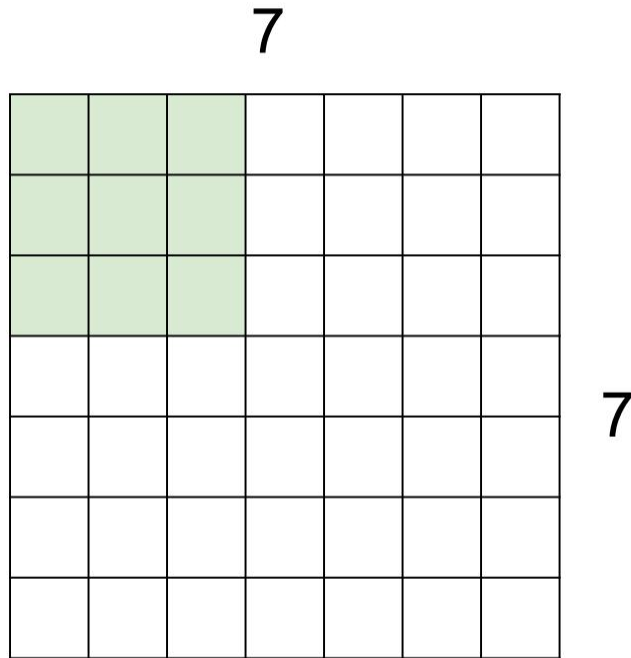


7

7x7 input (spatially)
assume 3x3 filter

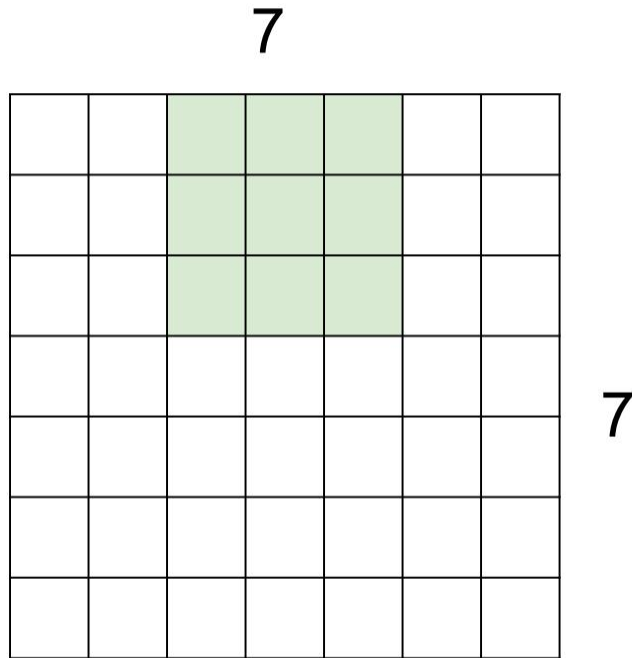
=> 5x5 output

A closer look at spatial dimensions:



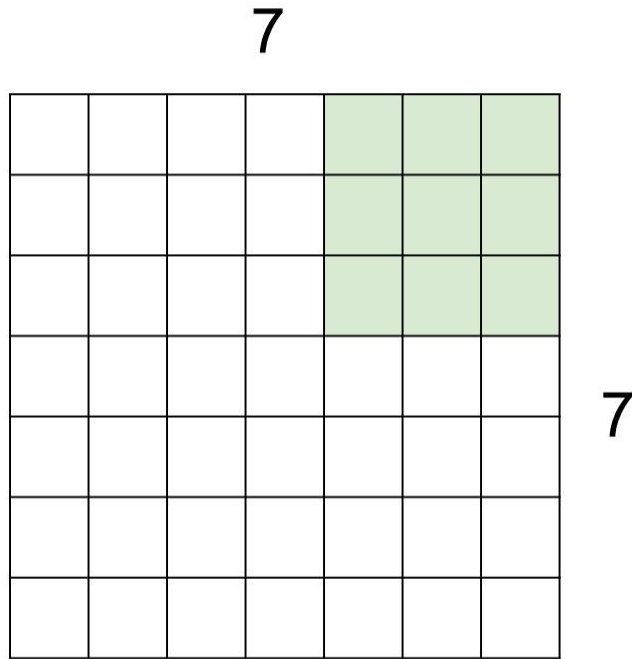
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



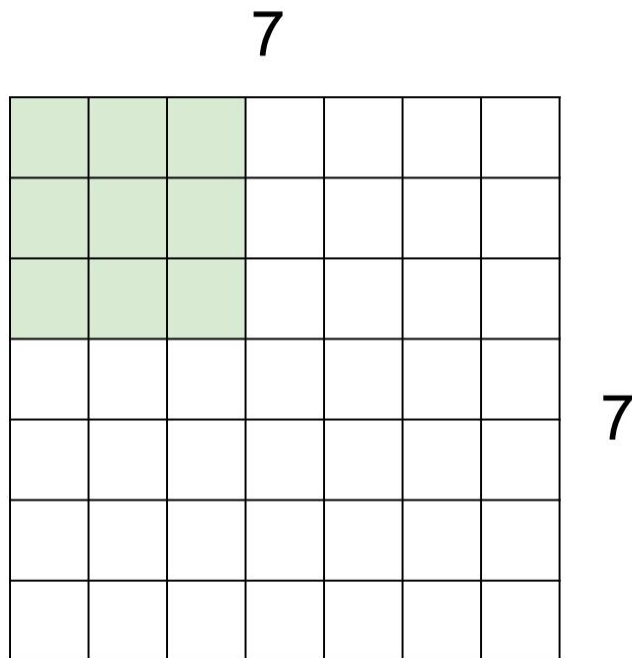
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



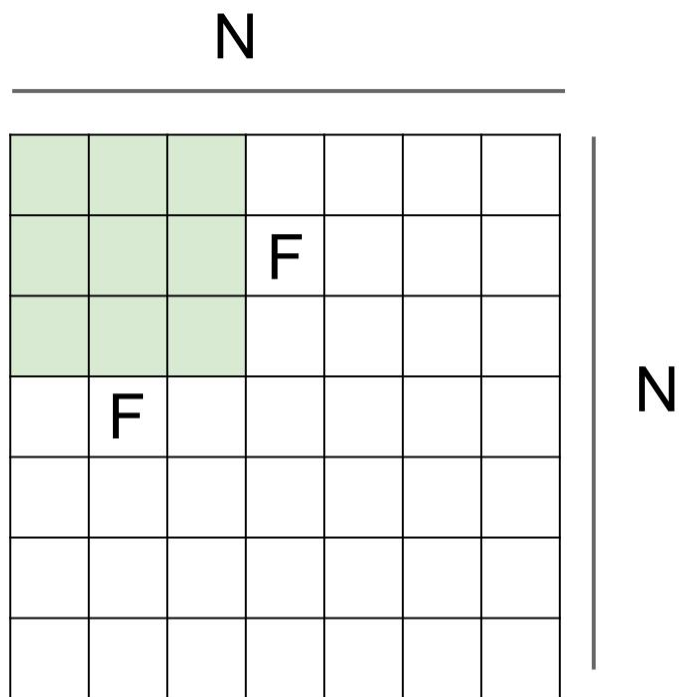
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

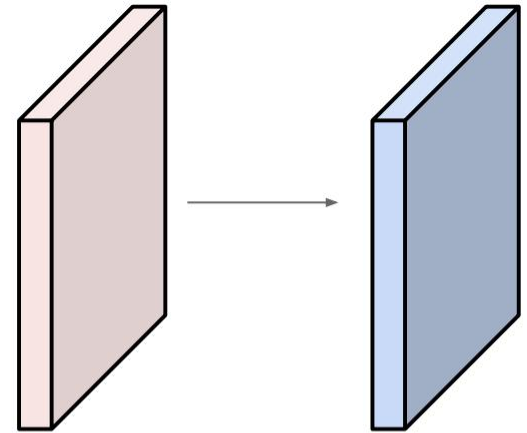
$F = 7 \Rightarrow$ zero pad with 3

Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

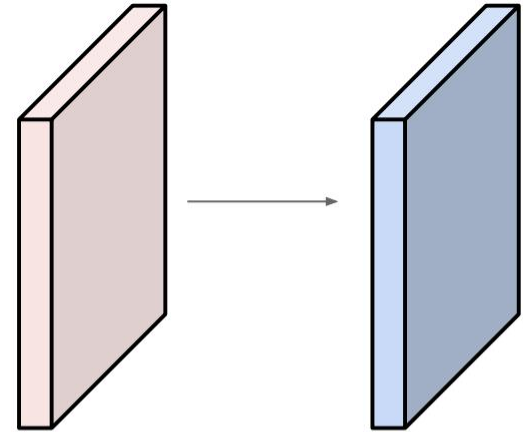
Output volume size: ?



Examples time:

Input volume: **32x32x3**
10 **5x5** filters with stride **1**, pad **2**

Output volume size:
 $(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so
32x32x10

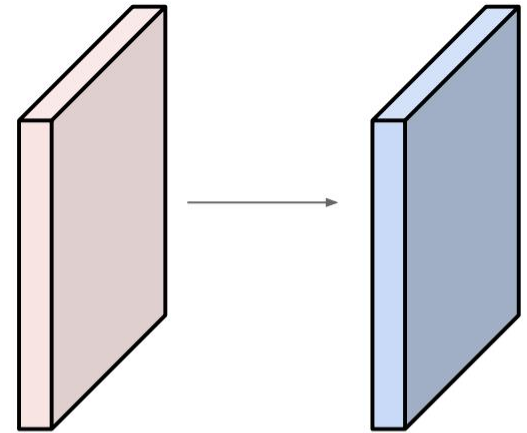


Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

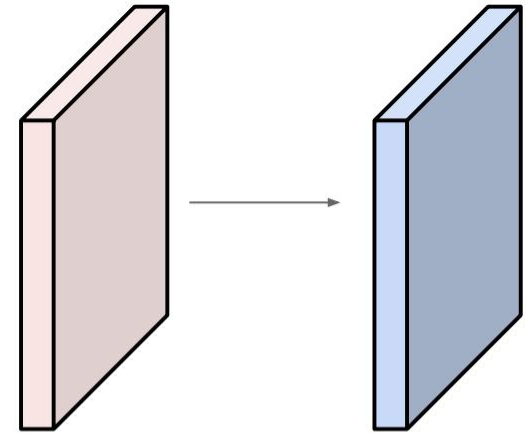
Number of parameters in this layer?



Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2

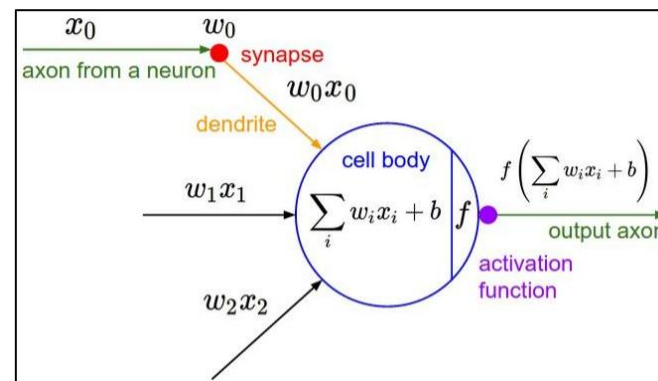
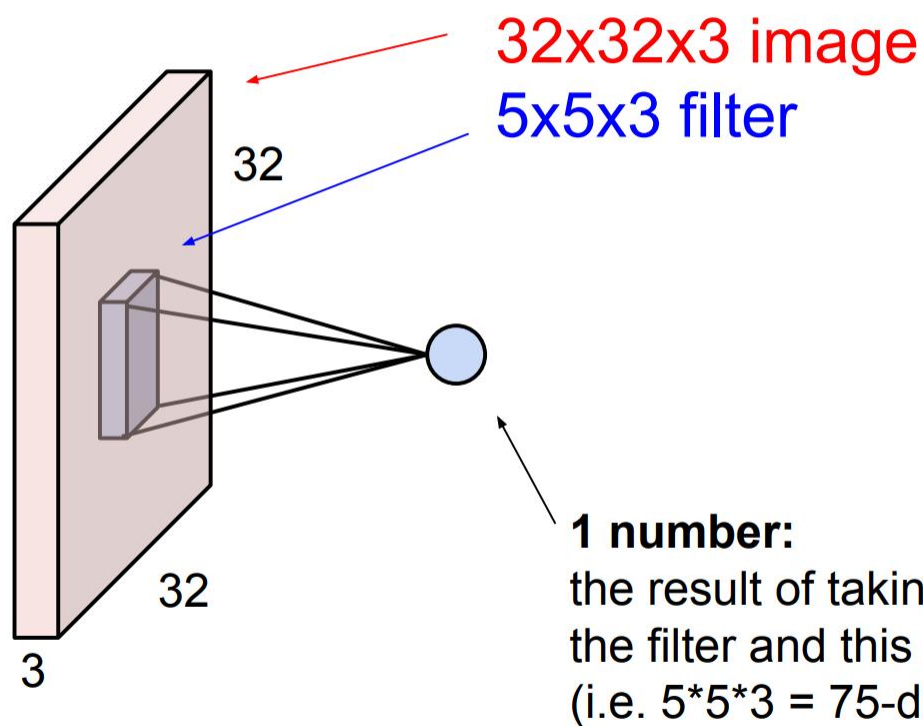


Number of parameters in this layer?

each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

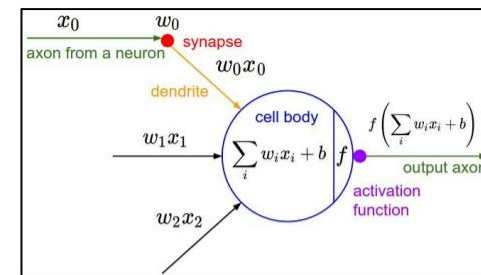
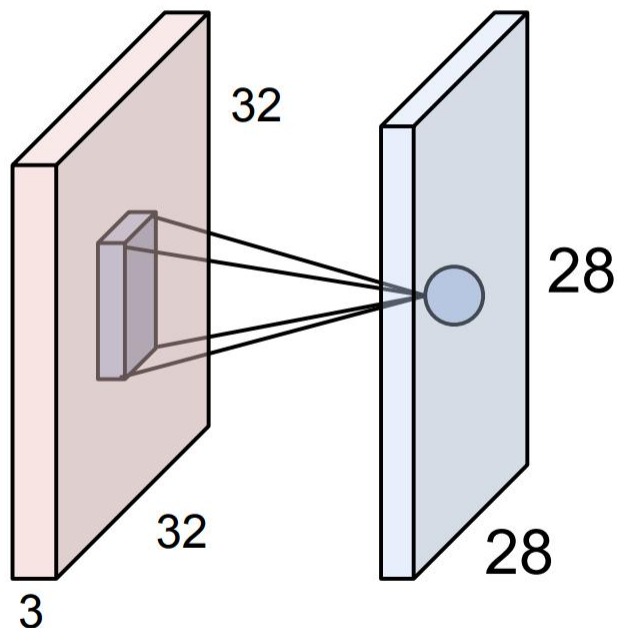
=> $76*10 = 760$

The brain/neuron view of CONV Layer



It's just a neuron with local connectivity...

The brain/neuron view of CONV Layer

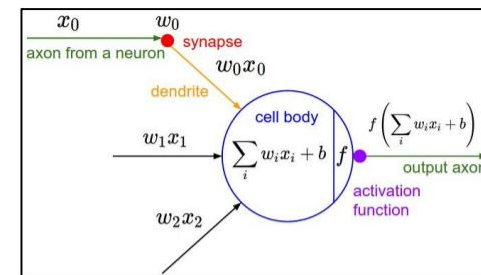
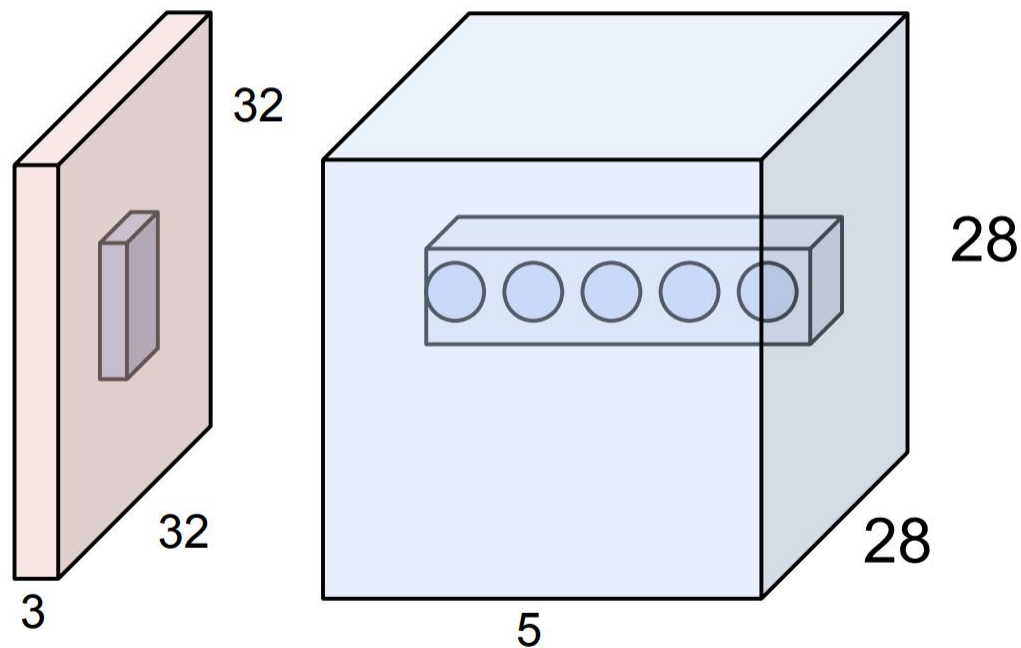


An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

“5x5 filter” -> “5x5 receptive field for each neuron”

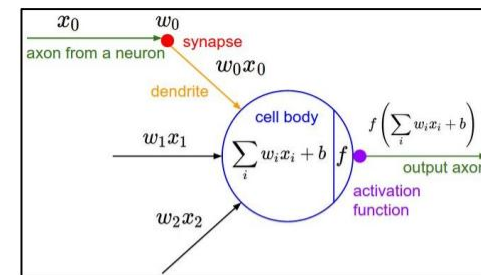
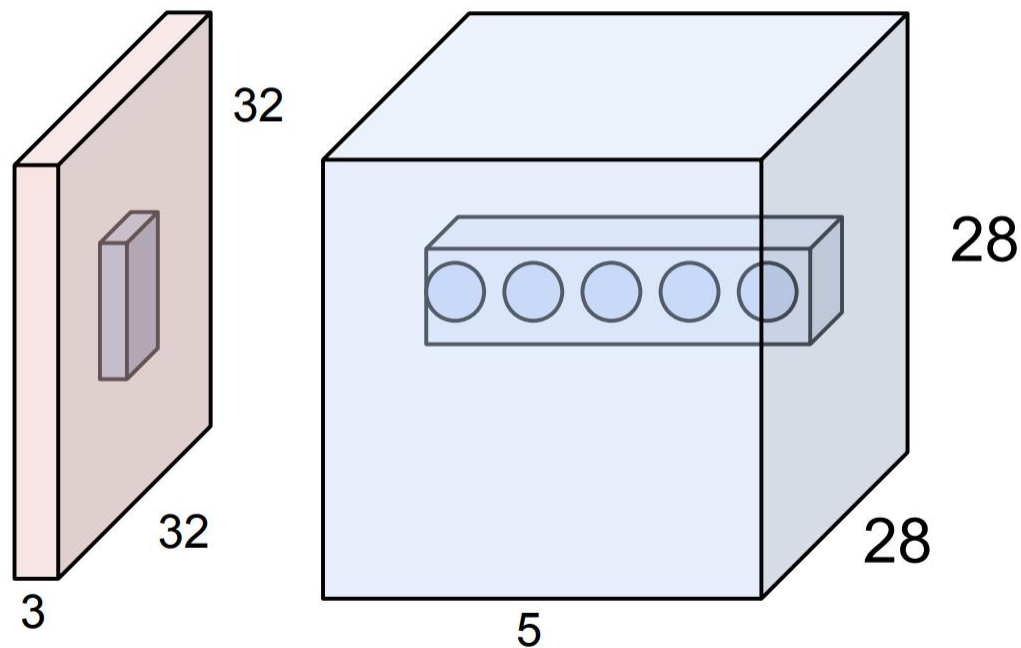
The brain/neuron view of CONV Layer



E.g. with 5 filters,
CONV layer consists of
neurons arranged in a 3D grid
(28x28x5)

There will be 5 different
neurons all looking at the same
region in the input volume

The brain/neuron view of CONV Layer

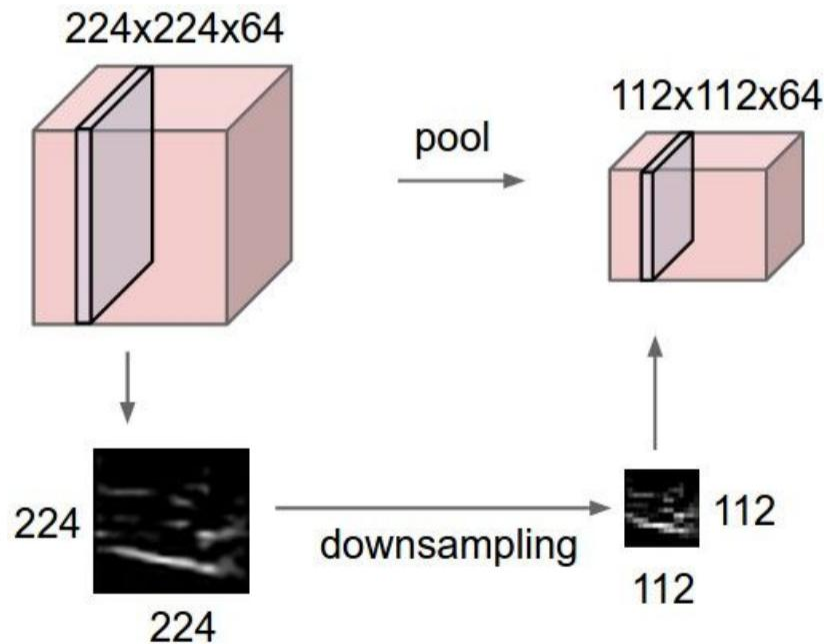


E.g. with 5 filters,
CONV layer consists of
neurons arranged in a 3D grid
(28x28x5)

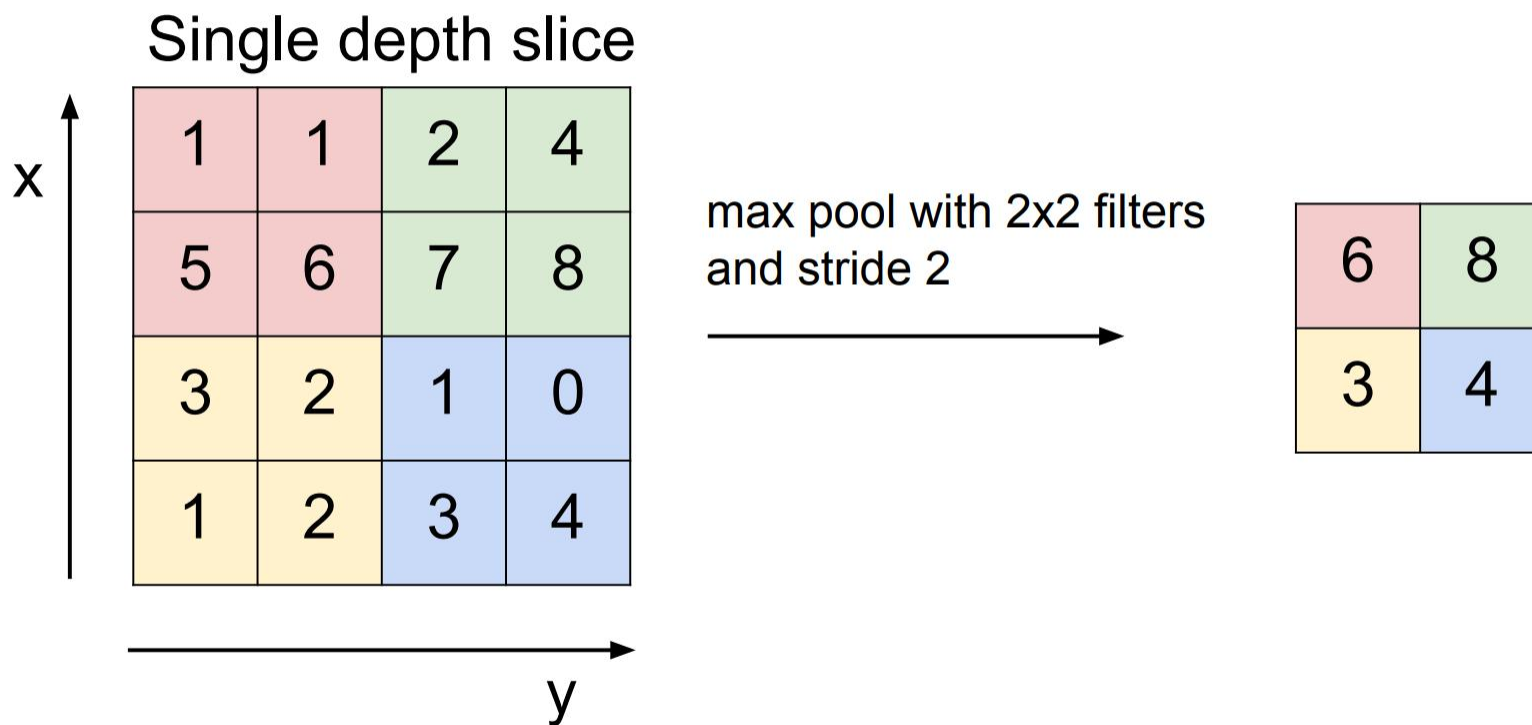
There will be 5 different
neurons all looking at the same
region in the input volume

Pooling layer

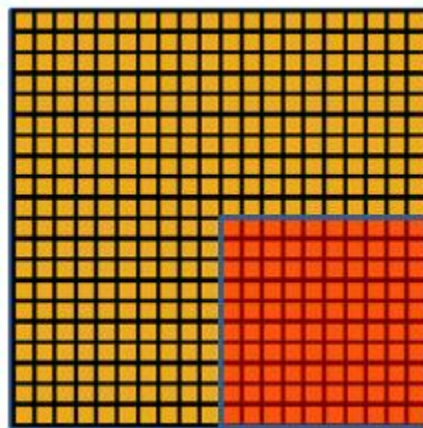
- makes the representations smaller and more manageable
- operates over each activation map independently:



MAX POOLING

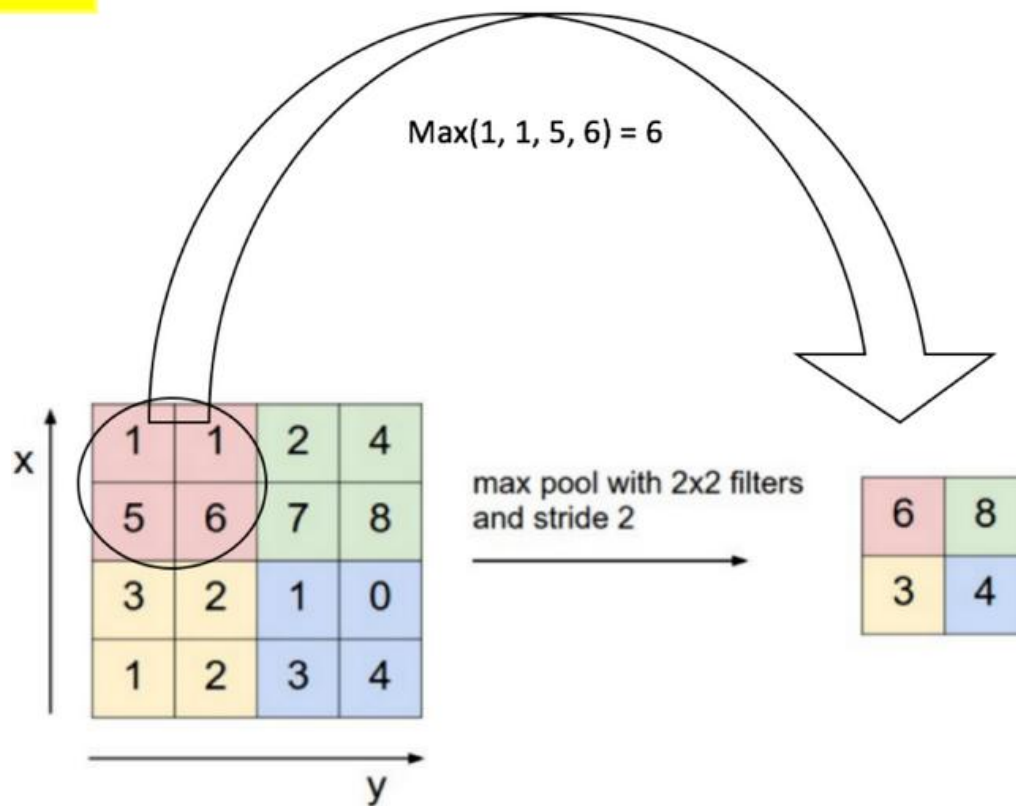
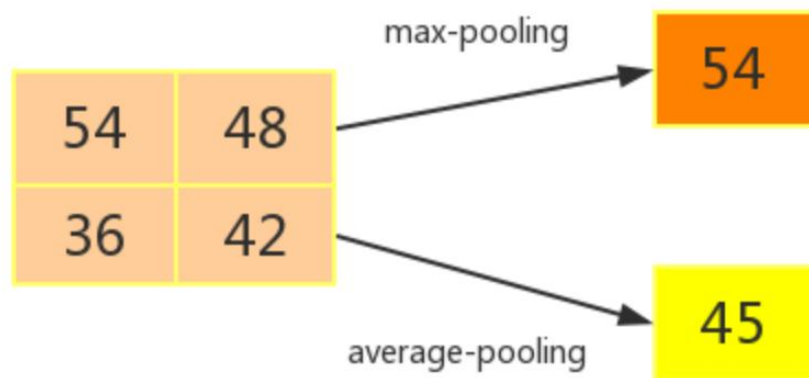


池化 (Pooling)



1	7
5	9

- 池化的过程如下图所示：
- 可以看到，原始图片是**20x20**的，对其进行采样，采样窗口为**10x10**，最终将其采样成为一个**2x2**大小的特征图。
- 之所以这么做，是因为即使做完了卷积，图像仍然很大（因为卷积核比较小），所以为了降低数据维度，就进行采样。
- 即使减少了许多数据，特征的统计属性仍能够描述图像，而且由于降低了数据维度，有效地避免了过拟合。
- 在实际应用中，分为最大值采样（**Max-Pooling**）与平均值采样（**Mean-Pooling**）。



其它常见层

⑩ Dropout

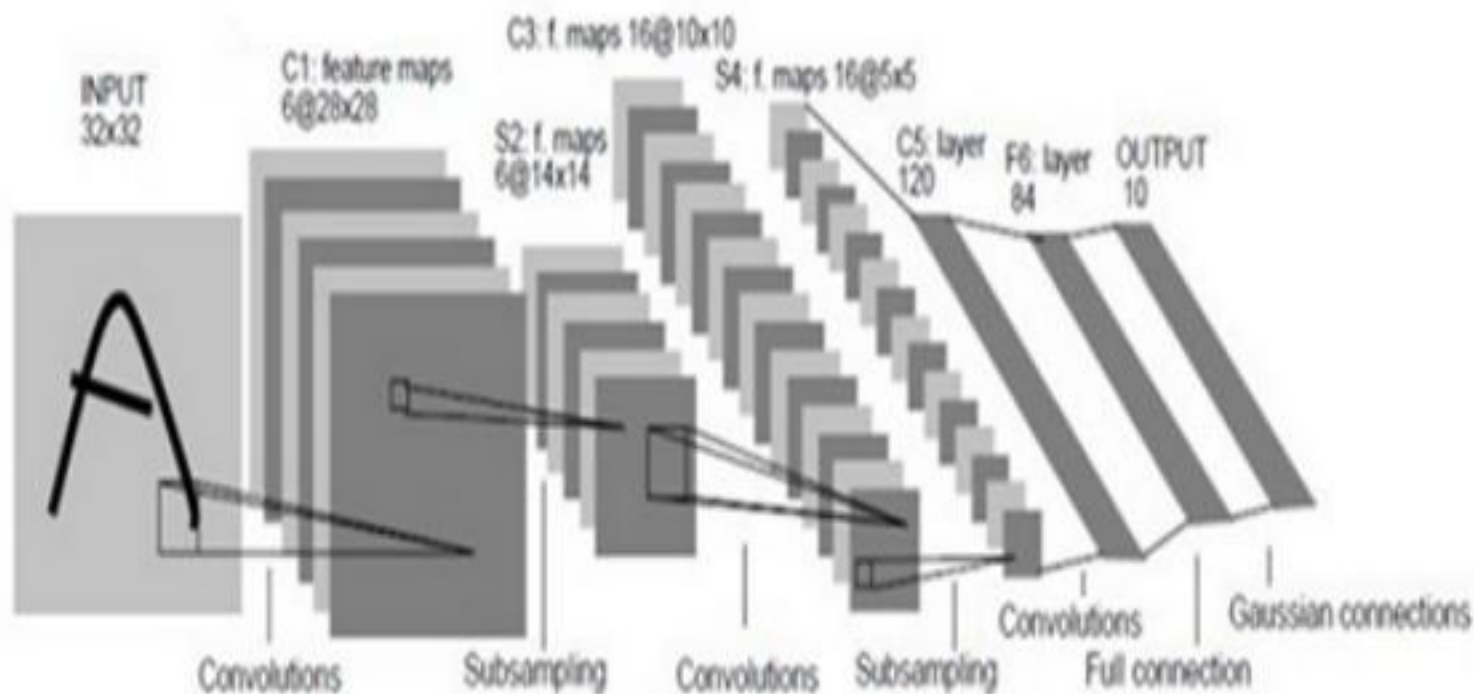
- 训练时，每次随机去掉全连接层中的一些神经元
- 防止或减轻过拟合

⑩ Batch normalization

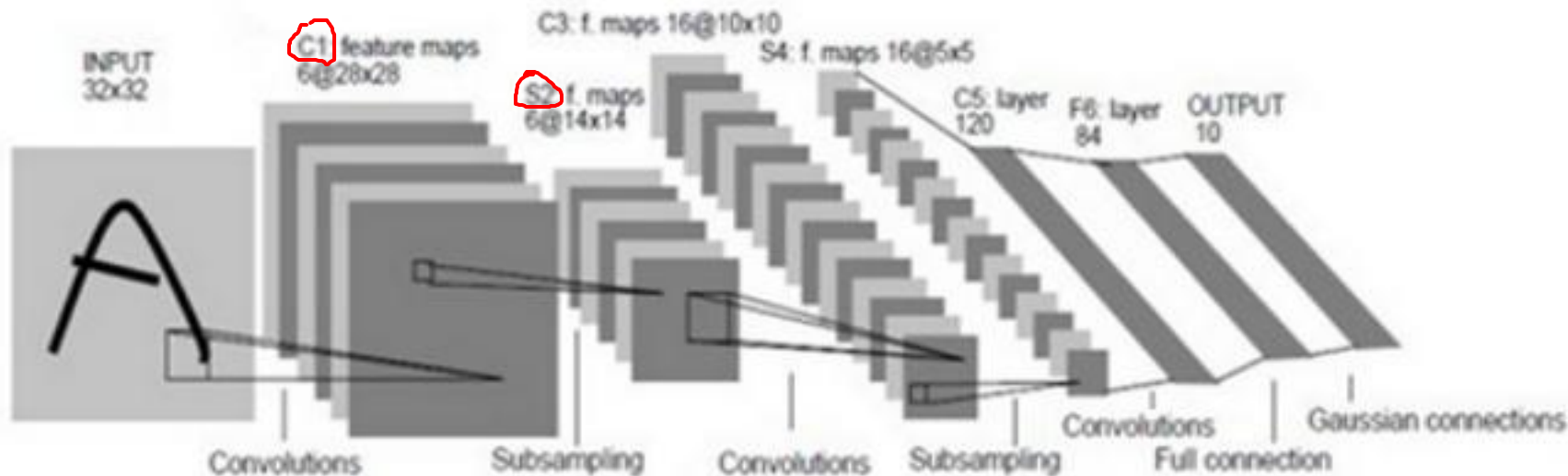
- 对输入数据进行减均值，除方差的操作
- 加快训练速度

LeNet介绍

下面再回到LeNet网络结构：

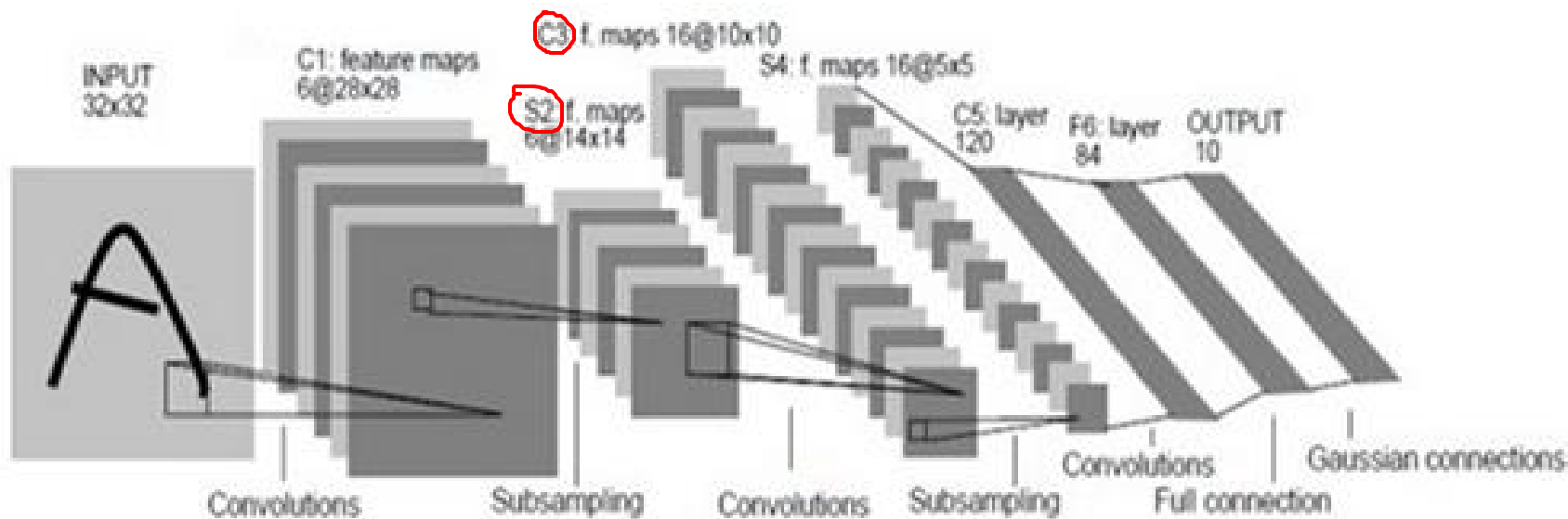


- 给定一个大小为 32×32 的图像，将其输入LeNet-5后，后续各层的处理如下：



1, 卷积层C1: 原始的 32×32 的图像输入进来以后, 先进入一个卷积层C1 (由6个 5×5 的卷积核组成), 卷积计算出6个 28×28 的图像。

2, 池化层S2: 然后用 2×2 的采样窗口进行下采样, 得到6个 14×14 的图像。

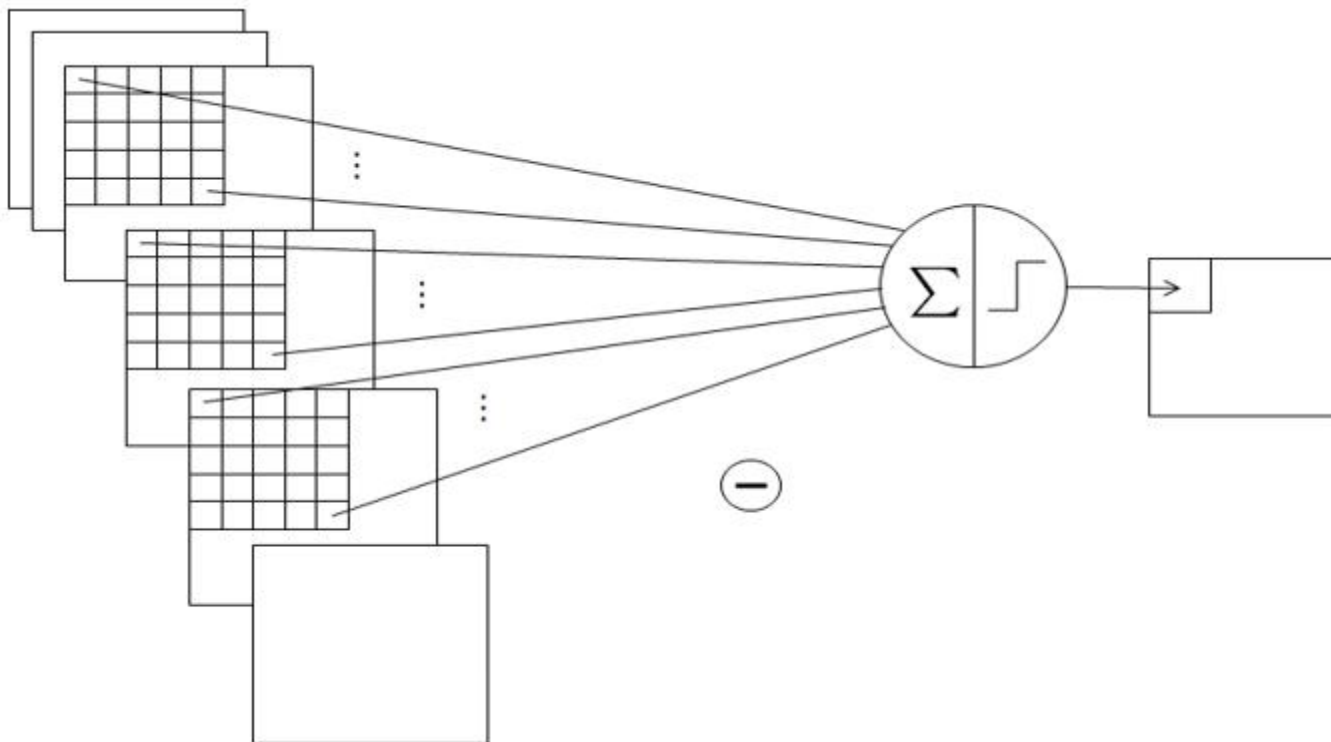


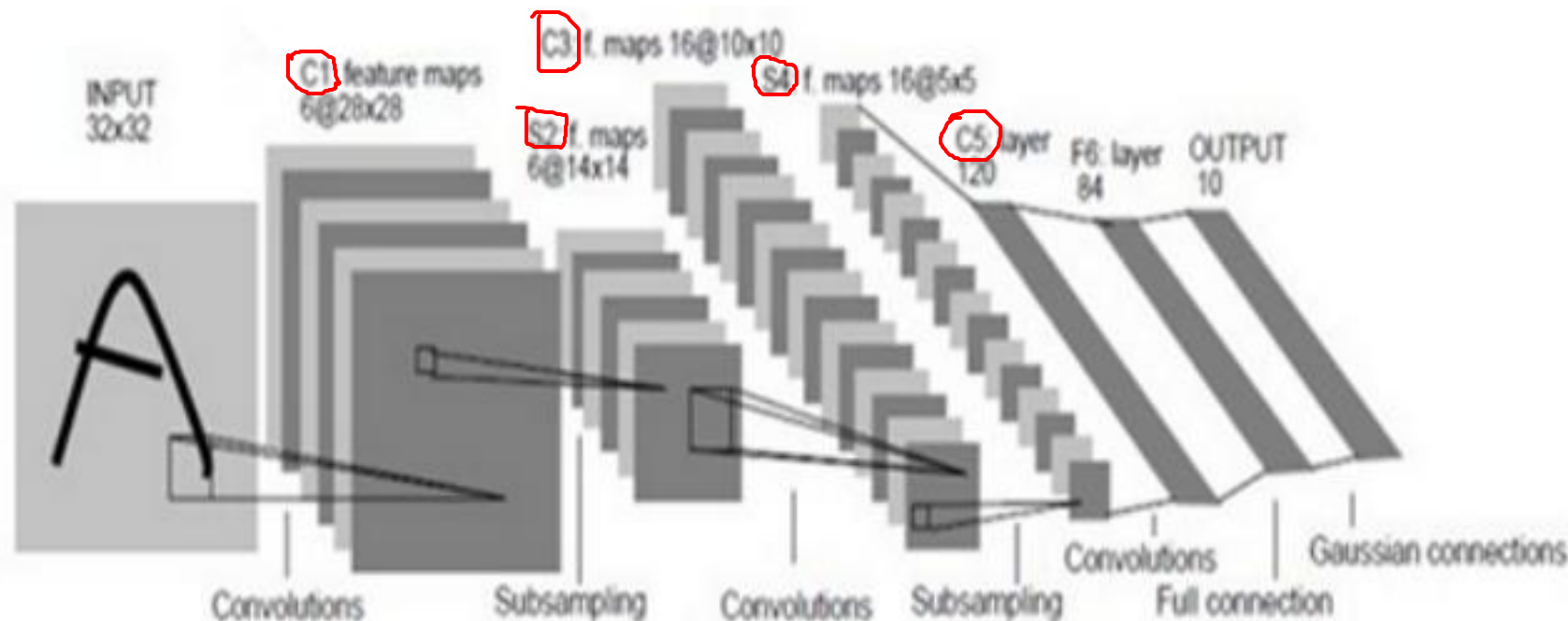
3, 卷积层C3: 由16个 5×5 的卷积核组成, 将6个 14×14 的图像进行卷积计算后, 得到16个 10×10 的图像。需要注意的是, C3中的每个特征图可以连接到S2中的所有6个或几个特征图, 表示本层的特征图是上一层提取到的特征图的不同组合。

一种可能的连接方式如图所示：**C3**的前**5**个特征图以**S2**中**3**个相邻的特征图子集为输入。接下来**5**个特征图以**S2**中**4**个相邻的特征图子集为输入。接下来的**5**个特征图以不相邻的**4**个特征图子集为输入。最后一个特征图将**S2**中所有特征图作为输入。之所以要采取这种方式，一是为了减少参数，二是这种不对称的组合连接的方式有利于提取多种组合特征。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0				X	X			X	X	X		X	X	X	X	X
1	X				X	X		X		X	X		X	X		X
2	X	X				X	X			X	X	X			X	X
3	X	X	X			X	X		X			X	X		X	X
4		X	X	X		X	X	X	X		X		X	X		X
5			X	X	X		X	X	X	X	X	X		X	X	X

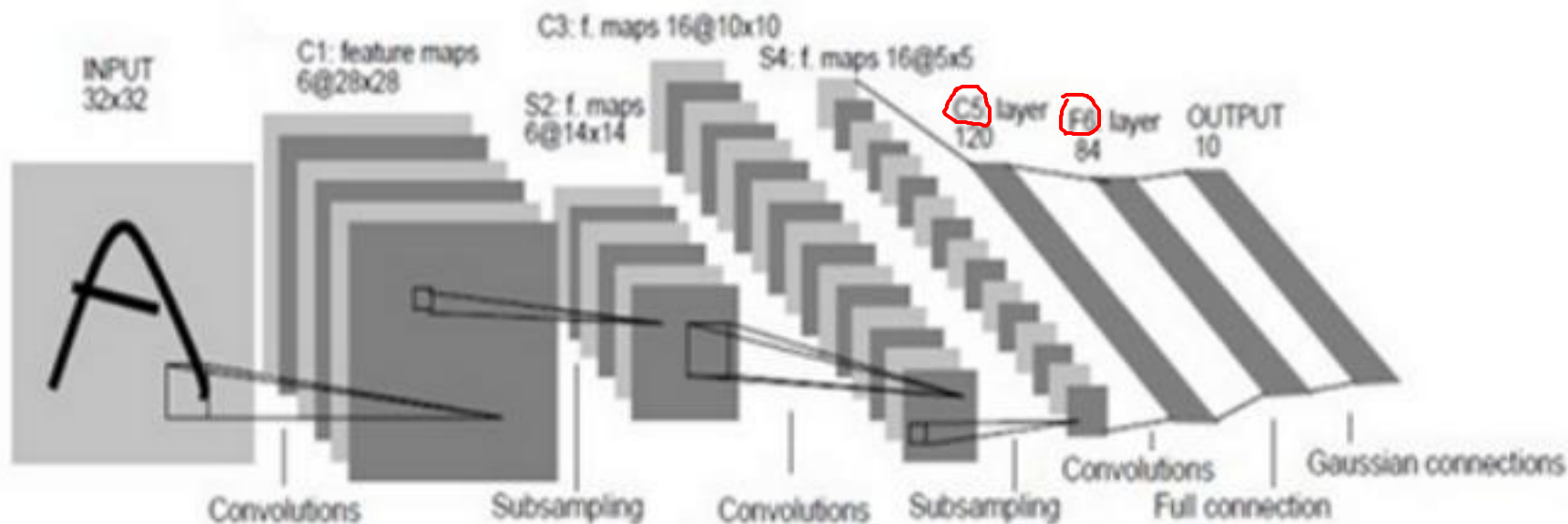
- C3与S2中的第1-3个图像相连的卷积结构如图所示:



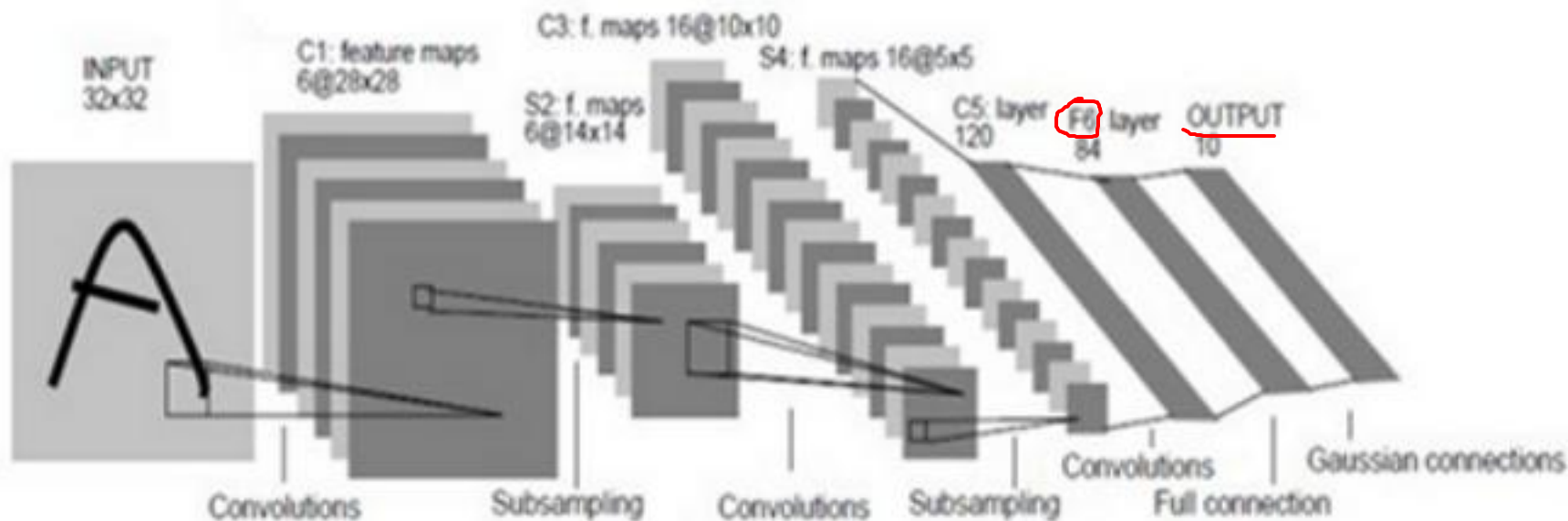


4，池化层S4：用 2×2 的采样窗口进行下采样，得到16个 5×5 的图像

5，卷积层C5：由120个 5×5 的卷积核组成，在16个 5×5 的图像上进行卷积操作后得到120个 1×1 的图像



6, 全连接层F6: 该层由 12×7 个神经元 (假设真实数字0-9的图像大小为 12×7) 组成, 每个神经元代表图像上的一个像素。用C5层得到的120维的向量作为输入, 和该层的84个神经元进行全连接。计算输入向量和连接权重向量之间的点积, 再加上一个偏置, 结果通过sigmoid函数输出。



7, 输出层 (OUTPUT) : 该层也是全连接层, 共包含10个神经元, 分别对应于0-9共10个数字。F6层的84个神经元与该层的10个神经元进行全连接。假设 x_j 对应F6层第j个单元的输出, w_{ij} 对应第i个数字的图像的第j个像素的bitmap编码, 则输出层10个神经元中第i个神经元的输出 y_i 由如下式子得出:

$$y_i = \sum_j (x_j - w_{ij})^2$$

该值越接近于0, 则原始图像越接近于第i个数字的图像 (越接近于其 bitmap 编码图), 则表示当前网络输入的识别结果就是数字i

-
- 卷积神经网络在使用时，往往使用多层卷积，然后再使用全连接层进行训练
 - 卷积和池化层只会提取特征，并减少原始图像带来的参数
 - 全连接层在整个卷积神经网络中起到“分类器”的作用
 - 之所以要多层卷积，是因为单层卷积学到的特征是局部的，卷积的层数越多，学到的特征就越全局化。当然，网络也就越复杂

训练过程

卷积神经网络的训练过程与传统神经网络类似，也是参照了反向传播算法

第一阶段，向前传播阶段：

- a) 从样本集中取一个样本(X, Y_p)，将 X 输入网络；
- b) 计算相应的实际输出 O_p

第二阶段，向后传播阶段

- a) 计算实际输出 O_p 与相应的理想输出 Y_p 的差；
- b) 按极小化误差的方法反向传播调整权矩阵。

-
- Stanford cs224n: Natural Language Processing with Deep Learning,
<http://web.stanford.edu/class/cs224n/>

常用的激活函数

- https://blog.csdn.net/tyhj_sf/article/details/79932893

Sigmoid和softmax的区别

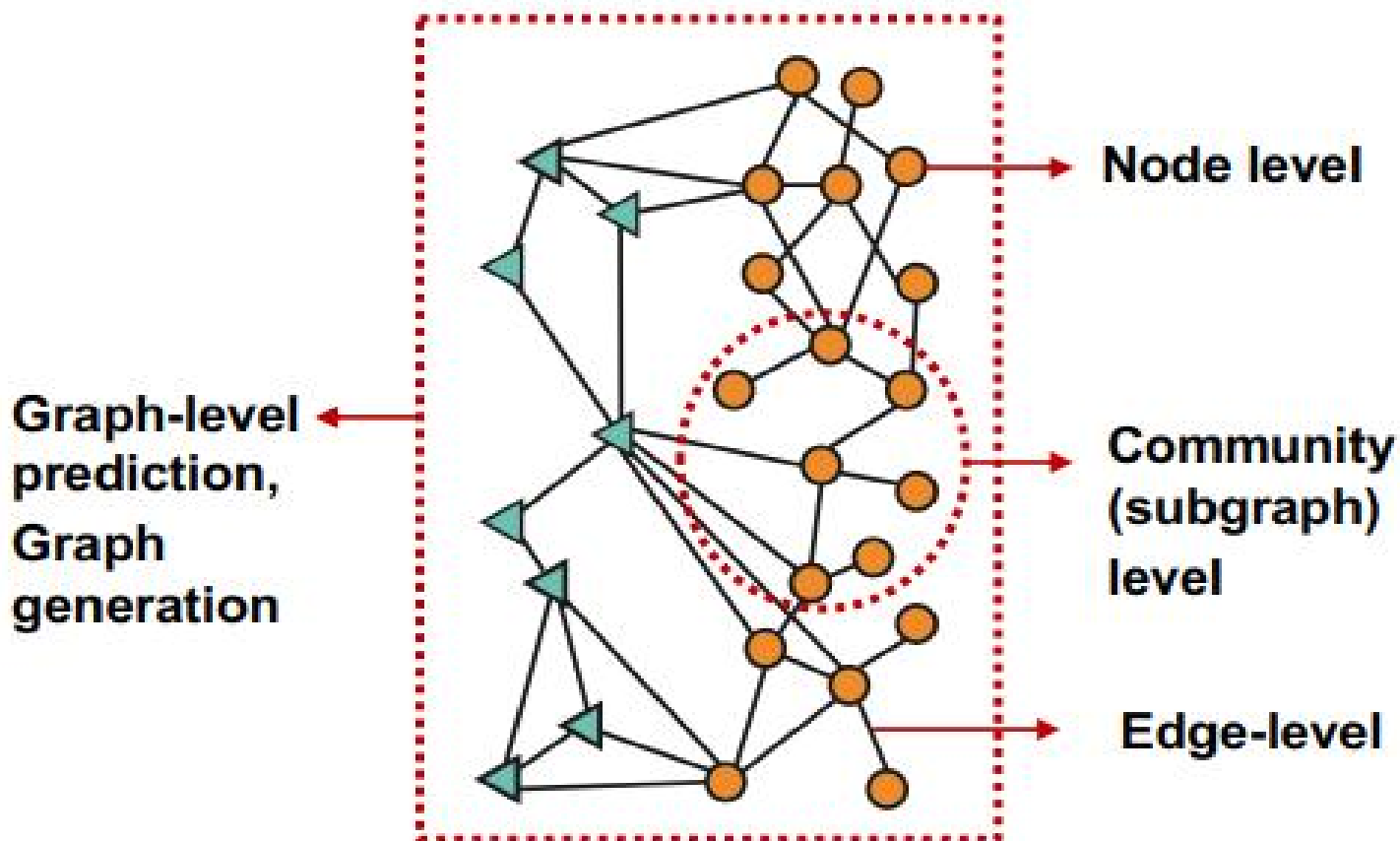
	Softmax	Sigmoid
公式	$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$	$S(x) = \frac{1}{1 + e^{-x}}$
本质	离散概率分布	非线性映射
任务	多分类	二分类
定义域	某个一维向量	单个数值
值域	$[0,1]$ 也是一个一维向量	$(0,1)$
结果之和	各个分量的值加起来为1	为某个正数

关于梯度消失或者爆炸

- 本质在于，求导算梯度时，前面的层上的梯度是来自后面的层上项的乘积。所以如果各项小于1，则在后向传播时会出现梯度随着层数减少越来越小的情况（消失）。如果各项的值大于1，则会出现随着层数减少梯度越来越大的现象（爆炸）
- 相关介绍：
<https://www.jianshu.com/p/917f71b06499>

图上的深度学习

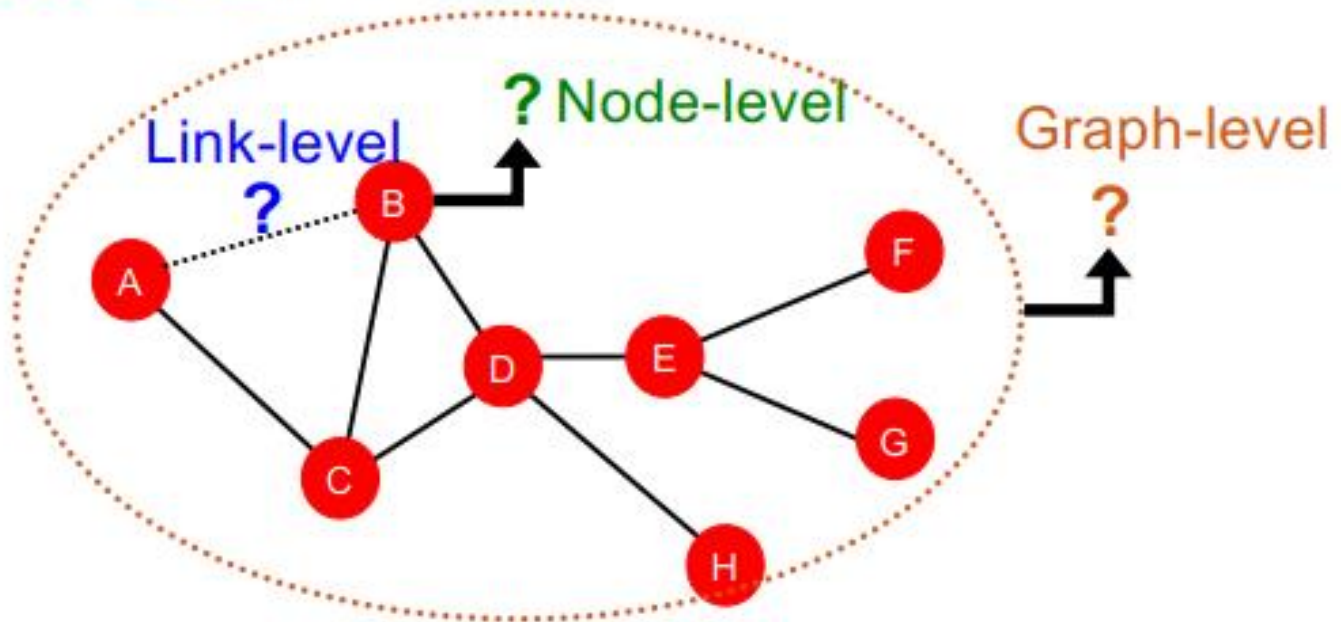
不同种类的图学习任务



图学习任务

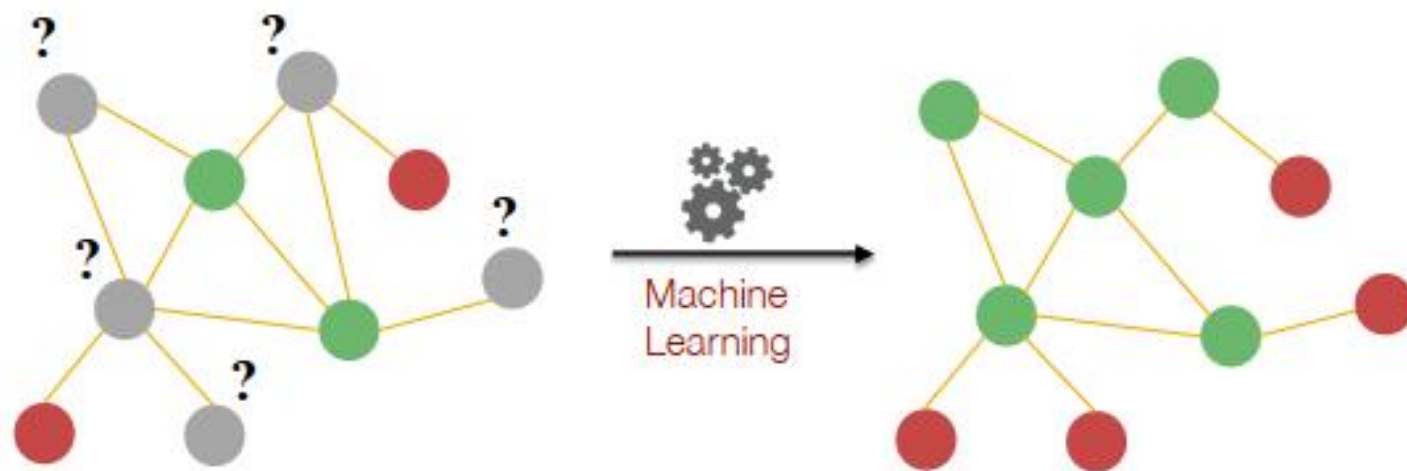
不同种类的图学习任务

- Node-level prediction
- Link-level prediction
- Graph-level prediction



图学习任务

Node-Level的图学习任务

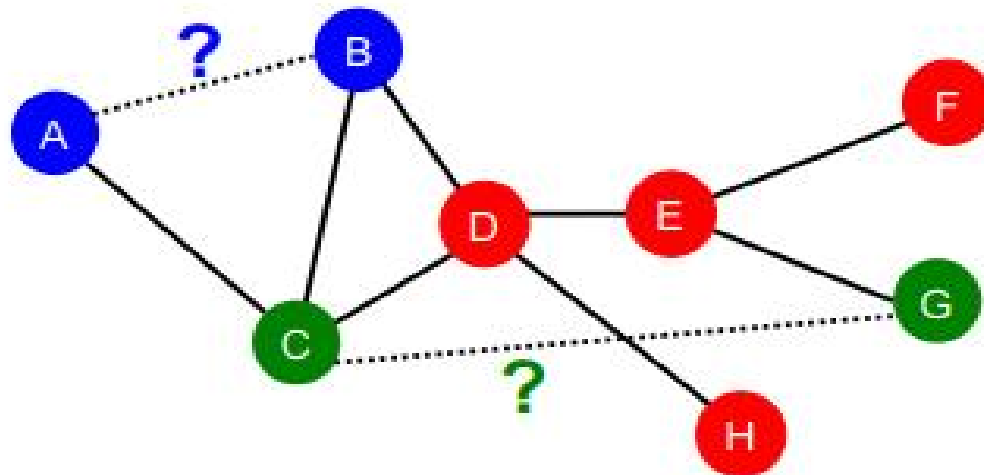


Node classification

图学习任务

Link-Level的图学习任务

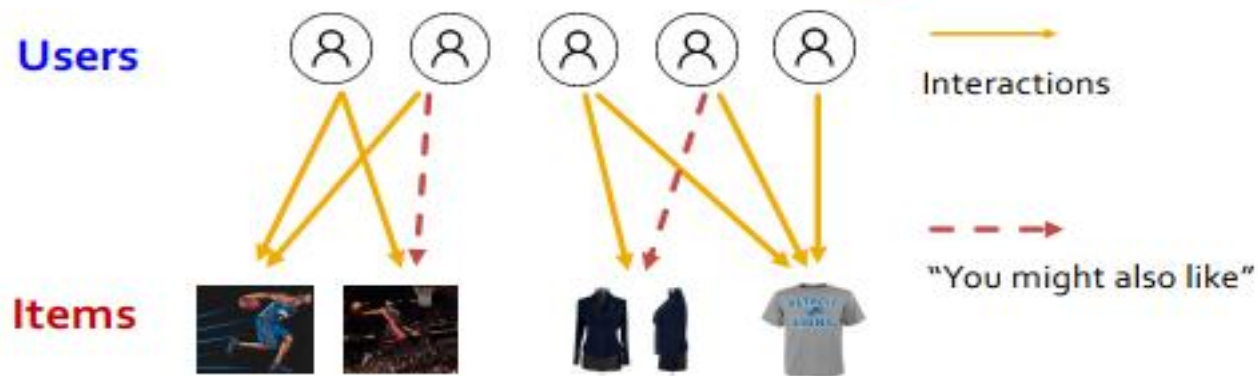
- The task is to predict **new/missing/unknown links** based on the existing links.
- At test time, node pairs (with no existing links) are ranked, and top K node pairs are predicted.
- Task: Make a prediction for a pair of nodes.



图学习任务

Link-Level的图学习任务：推荐系统

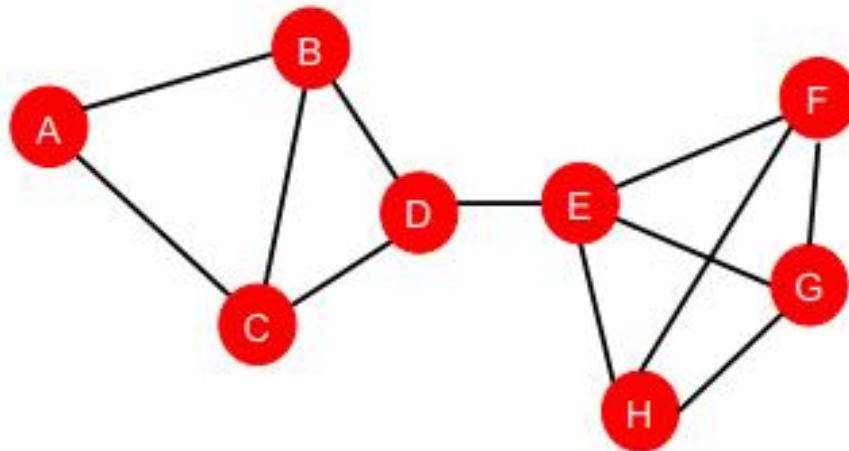
- **Users interacts with items**
 - Watch movies, buy merchandise, listen to music
 - **Nodes:** Users and items
 - **Edges:** User-item interactions
- **Goal: Recommend items users might like**



图学习任务

Graph-Level的图学习任务

- **Goal:** We want make a prediction for an entire graph or a subgraph of the graph.
- **For example:**



图学习任务

Graph-Level的图学习任务：交通预测

- **Nodes:** Road segments
- **Edges:** Connectivity between road segments
- **Prediction:** Time of Arrival (ETA)

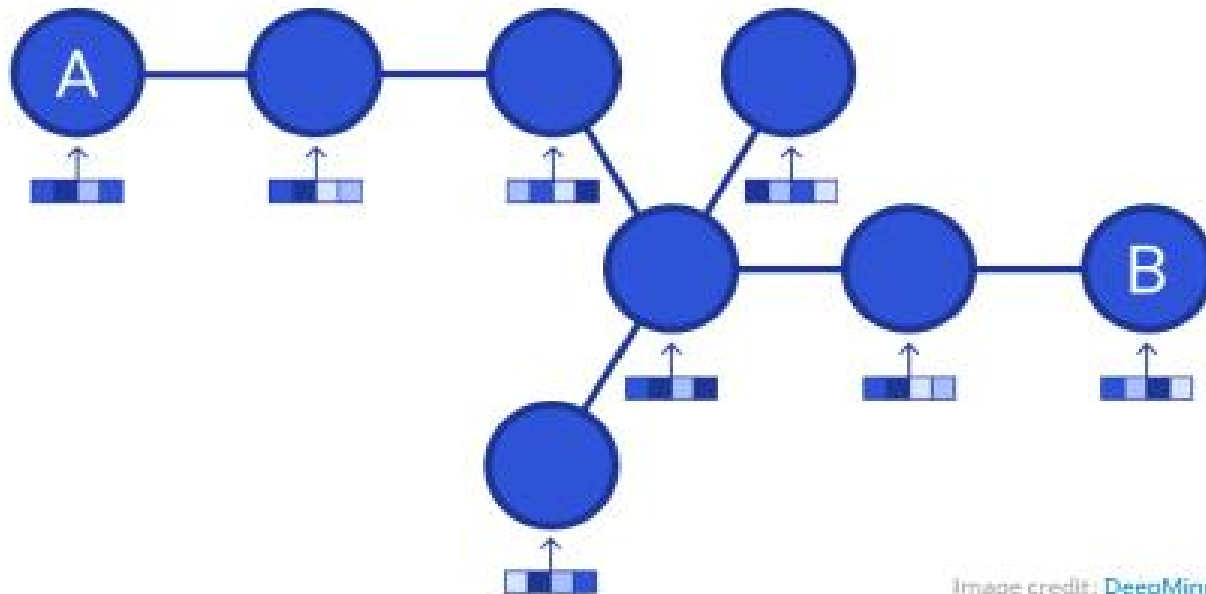
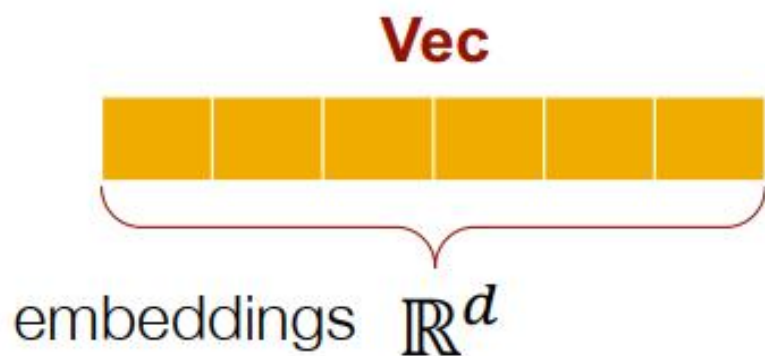
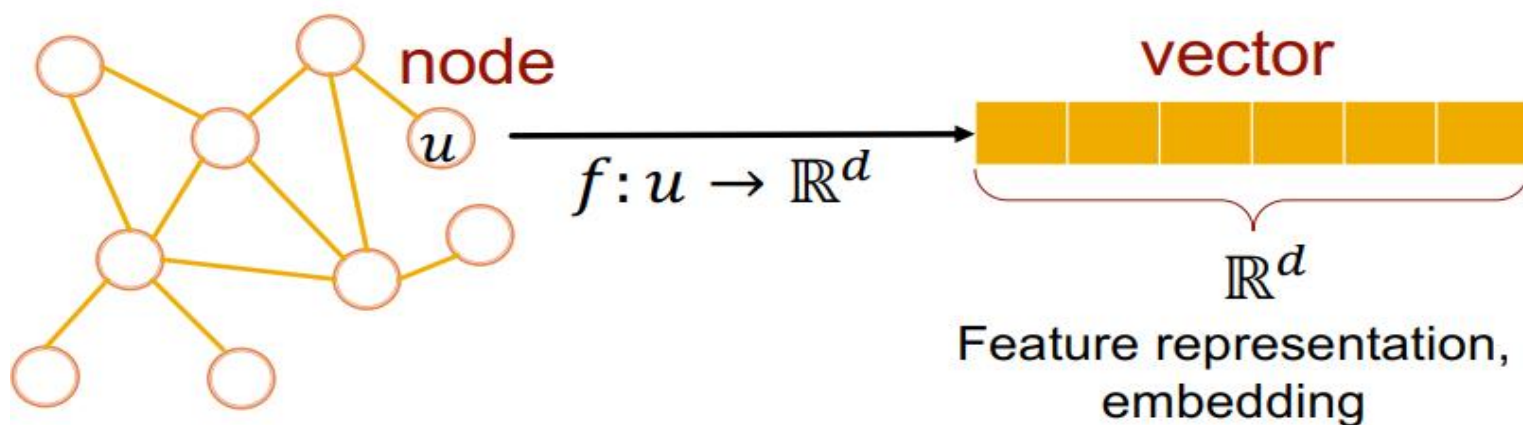


Image credit: [DeepMind](#)

图结点表示

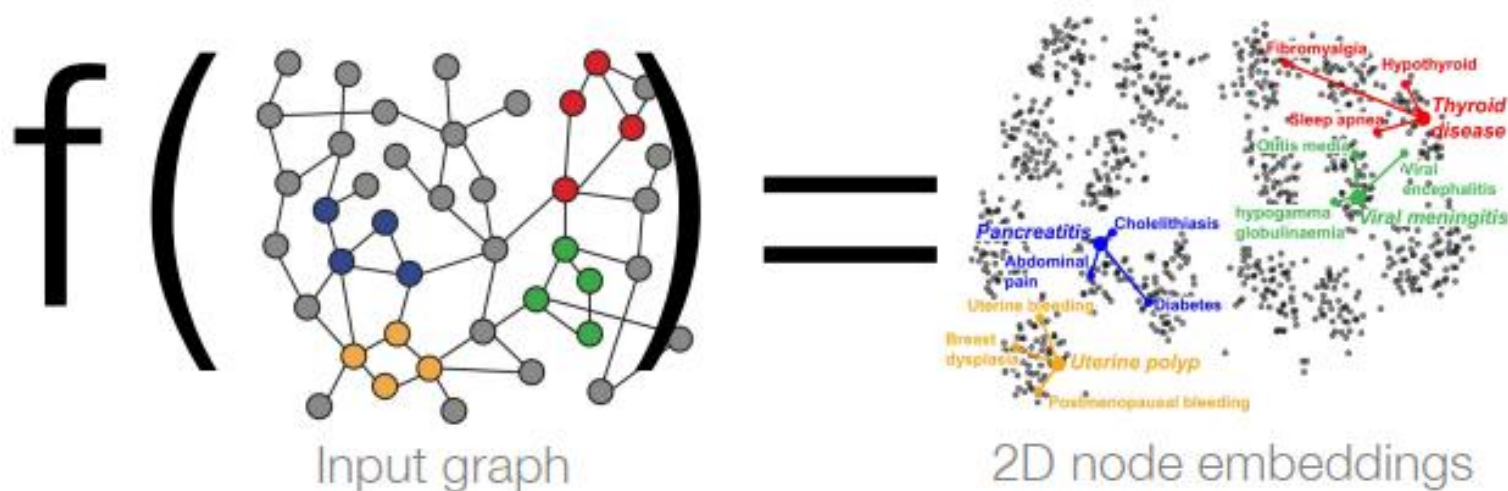


Tasks

- Node classification
- Link prediction
- Graph classification
- Anomalous node detection
- Clustering
-

图结点表示

图结点表示学习的目标是：对图结点进行编码，使得在向量空间中两个向量的相似度（如点积）近似等于图中两个结点的相似度

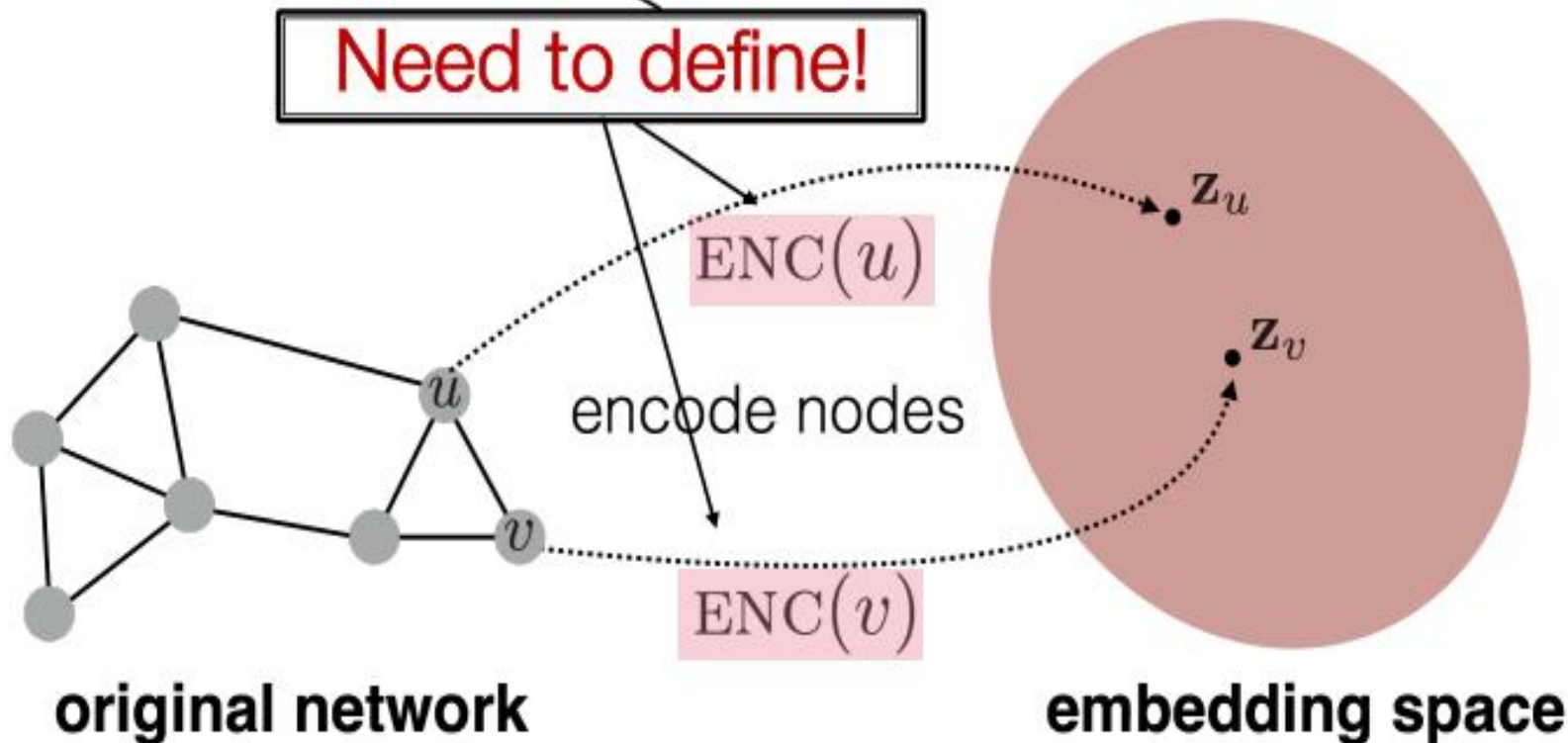


How to learn mapping function f ?

图结点表示

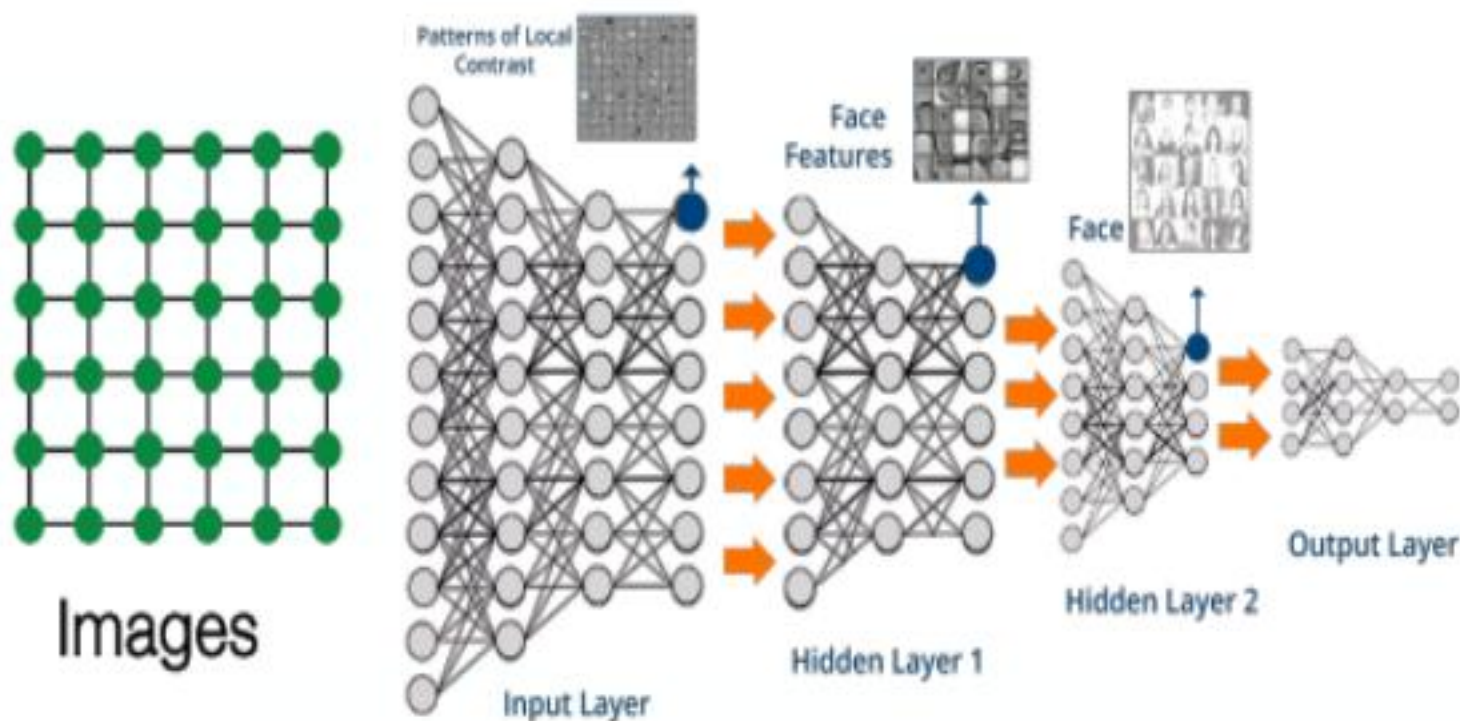
Goal: $\text{similarity}(u, v)$ in the original network $\approx \mathbf{z}_v^T \mathbf{z}_u$ Similarity of the embedding

Need to define!



图结点表示

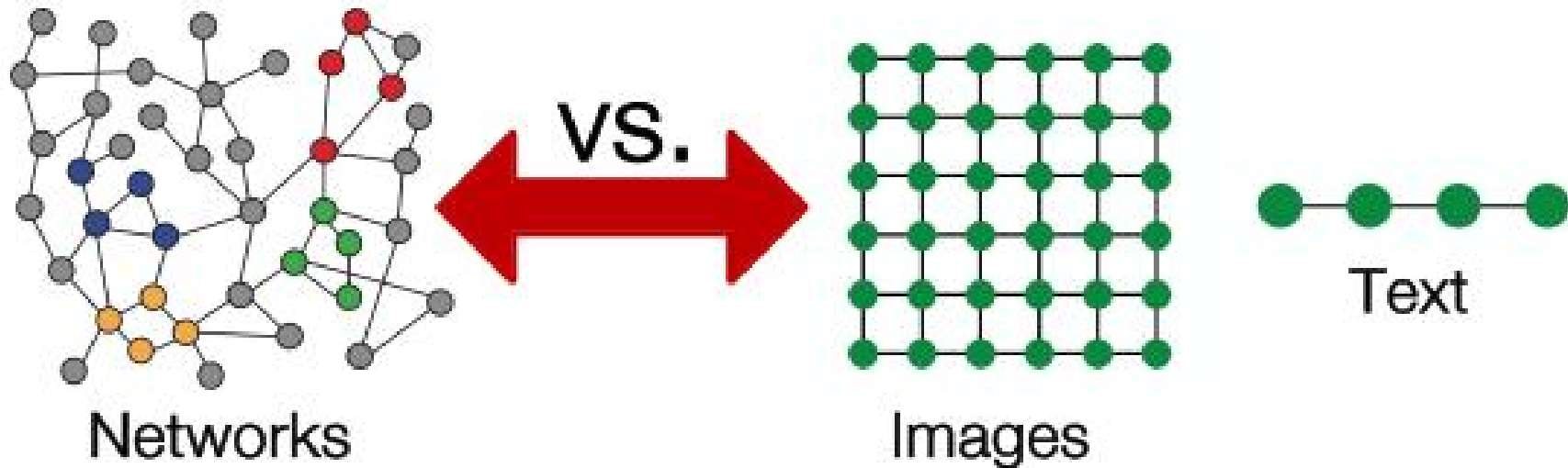
现有的CNN主要适用于规则的grid结构或者一维的序列



图结点表示

但图结构是**不规则**的!

图**没有**固定的结点顺序或者**参考点**
会**动态变化**，具有**多模态**的特征

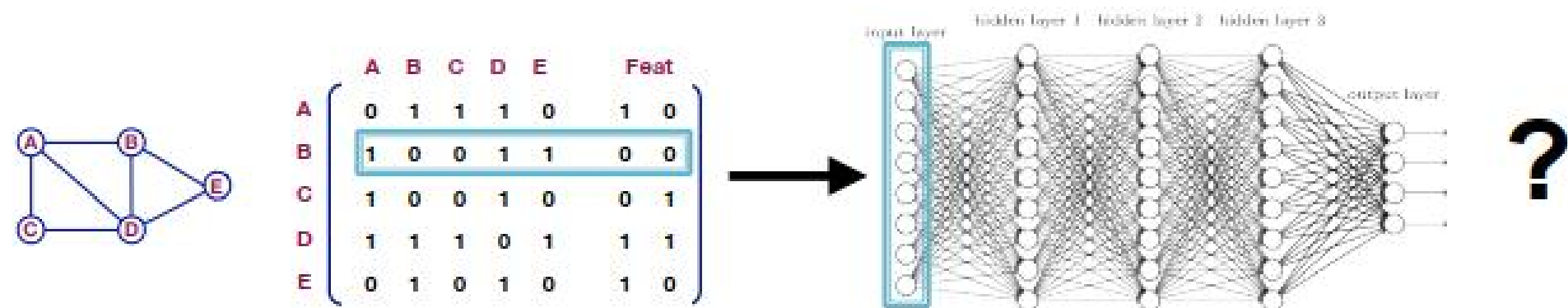


图结点表示

- 问题定义：给定一个图 G
 - 结点集合 V
 - 邻接矩阵 A
 - 结点 v 的邻居结点集合 $N(v)$
 - 结点的特征矩阵 X
 - 如，社交网络的结点特征有：用户画像、用户照片等

图结点表示

Naive方法： 将图的邻接矩阵和特征矩阵合并输入深度神经网络

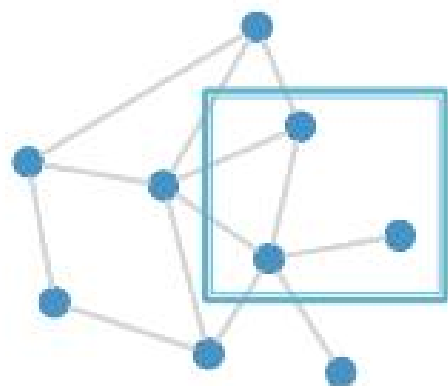


该方法的问题：

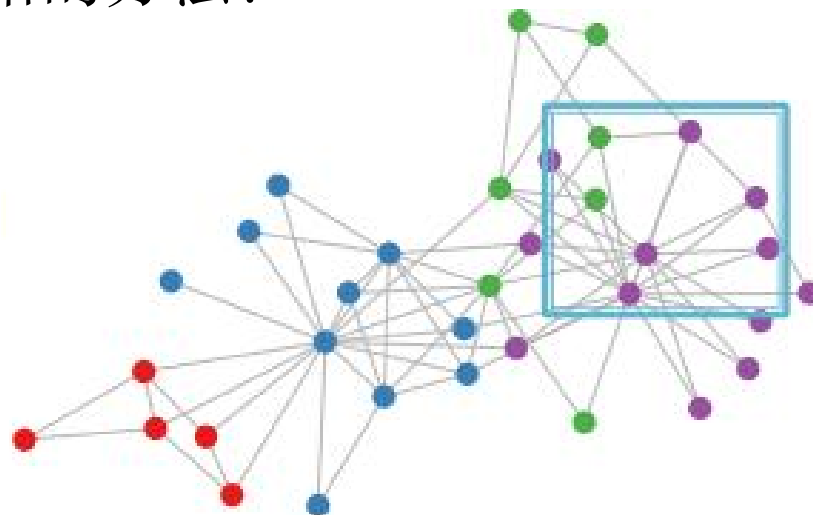
- 1, 参数的个数与结点个数相关
- 2, 不能适用于不同大小的图
- 3, 对结点的顺序敏感

图结点表示

能否用前面介绍的卷积神经网络的方法？



or this:



存在的问题:

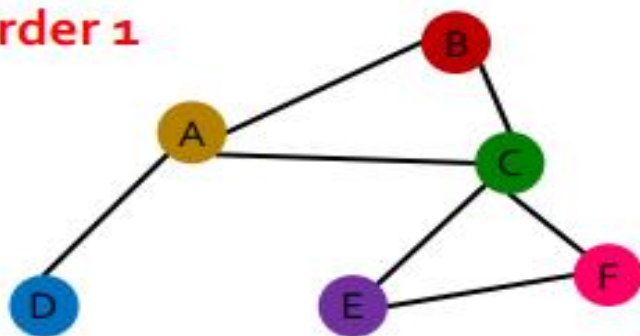
- 1, 在图上, 没有固定的**locality**或滑动窗口的概念
- 2, 图的结点没有一个固定的顺序。对于同一个图, 可以有多种结点排序方式。当对图进行表示学习时, 无论结点如何排序, 都希望能得到一个相同的结果表示 (Permutation Invariant) 或者等价的结果表示 (permutation equivariant)

Permutation Invariant (排列不变性)

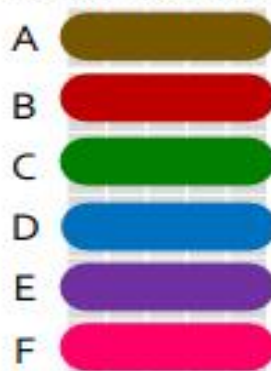
对Order1和Order2来说，图学习后得到的结点表示应该一样

- Graph does not have a canonical ordering of the nodes!

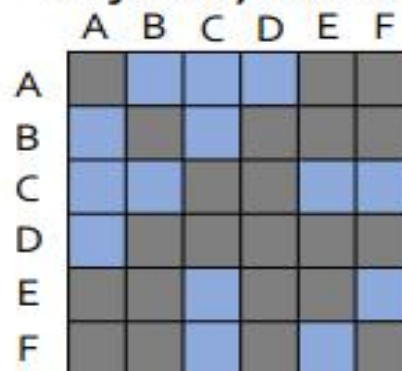
Order 1



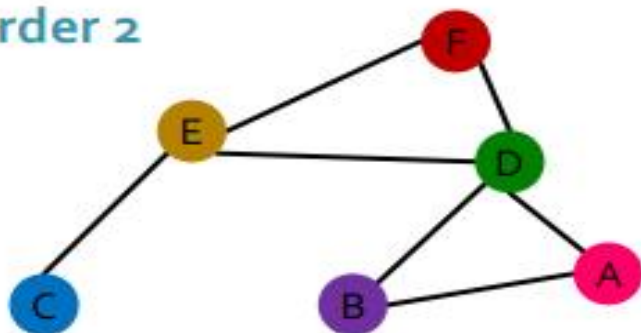
Node features X_1



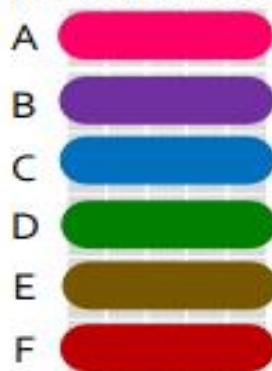
Adjacency matrix A_1



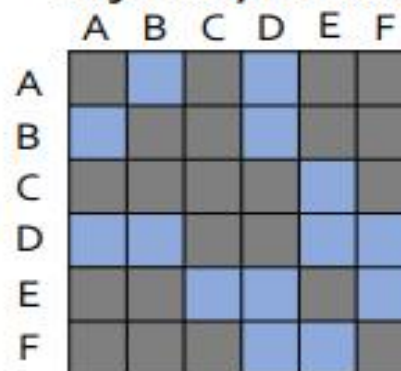
Order 2



Node features X_2



Adjacency matrix A_2



Permutation Invariant (排列不变性)

给定图G的两个顺序Order1和Order2:

Order1对应的邻接矩阵是A1, 特征矩阵是X1

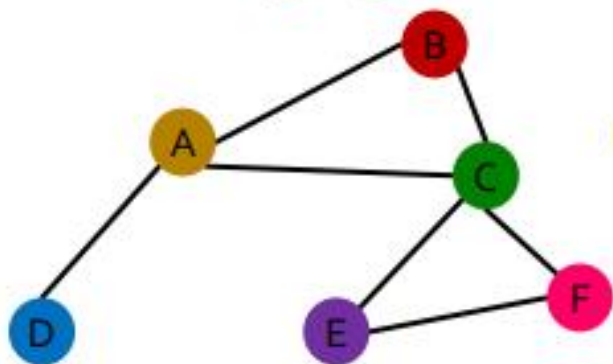
Order2对应的邻接矩阵是A2, 特征矩阵是X2

需要学习一个函数f, 将图G映射到向量d维向量R, 满足:

$$f(A_1, X_1) = f(A_2, X_2)$$

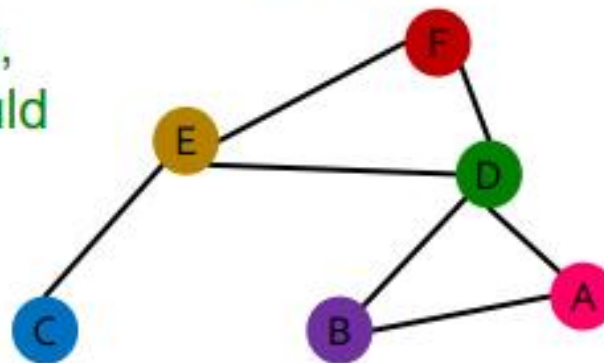
A is the adjacency matrix
X is the node feature matrix

Order 1: A_1, X_1



For two orders,
output of f should
be the same!

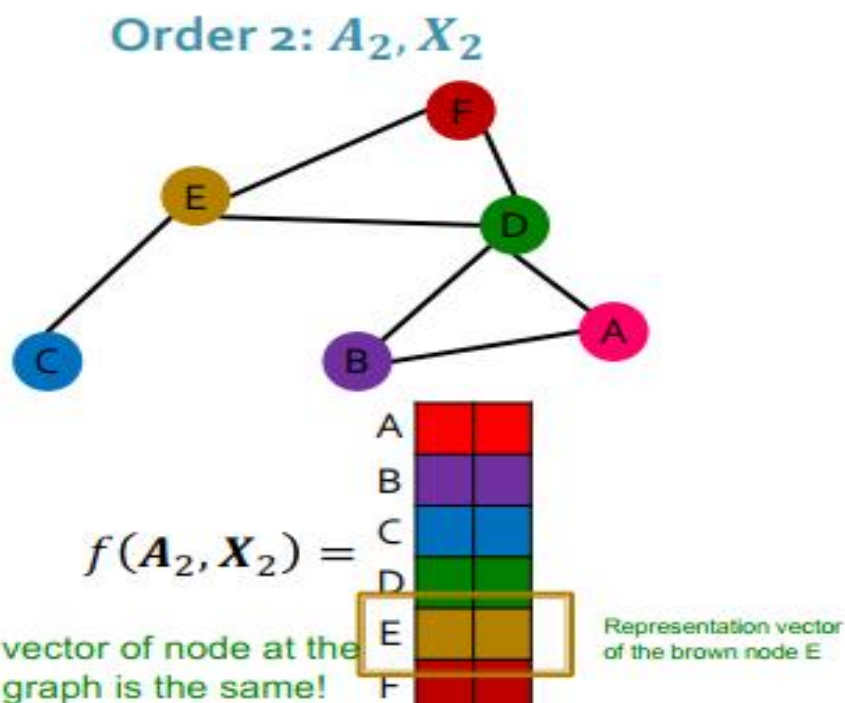
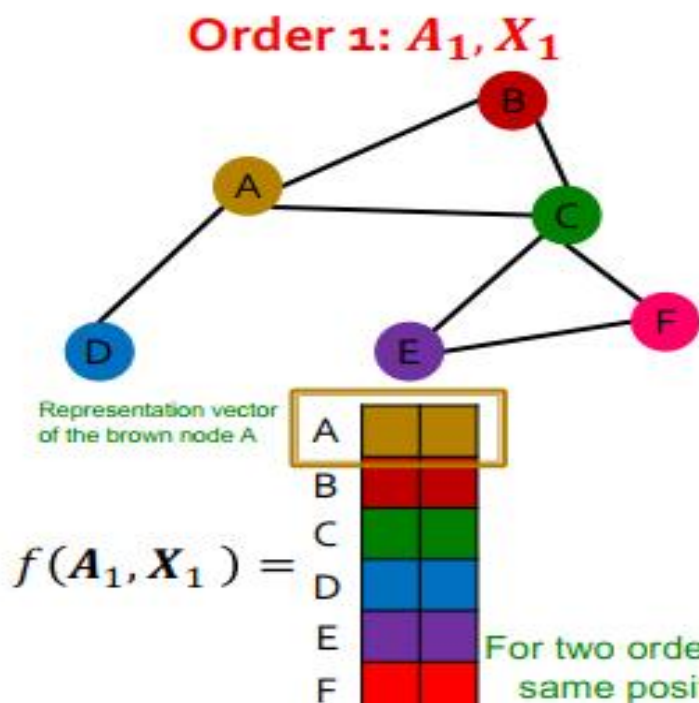
Order 2: A_2, X_2



permutation equivariant (排列等价性)

需要学习一个函数 f ，将图的结点映射为一个 $|V|*d$ 大小的矩阵
换句话说，需要为每个结点学习一个 d 维向量。

要求：不同Order顺序下，位于图上相同位置的结点对应的向量应该相同。例如，A和E在图上位于相同的位置，其向量该相同

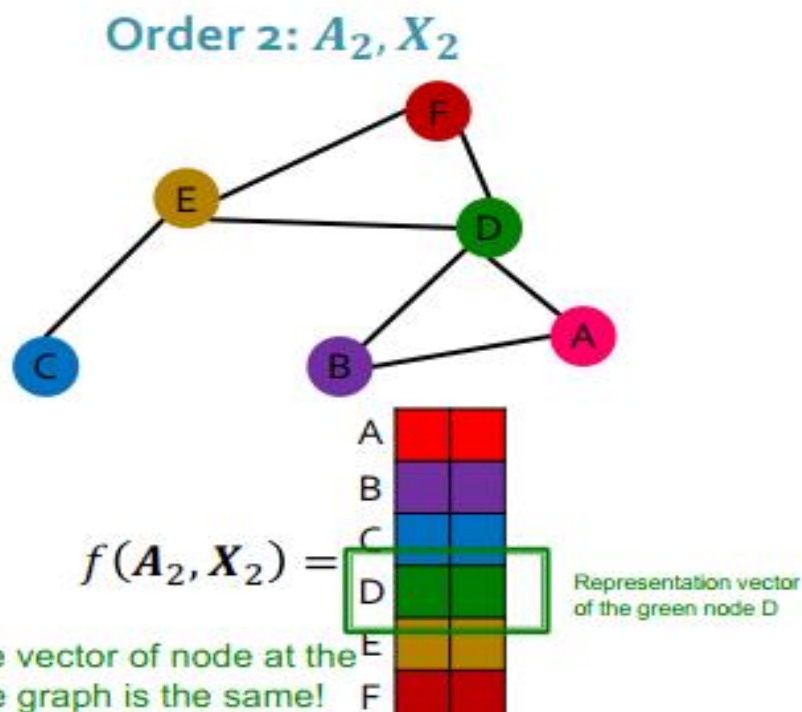
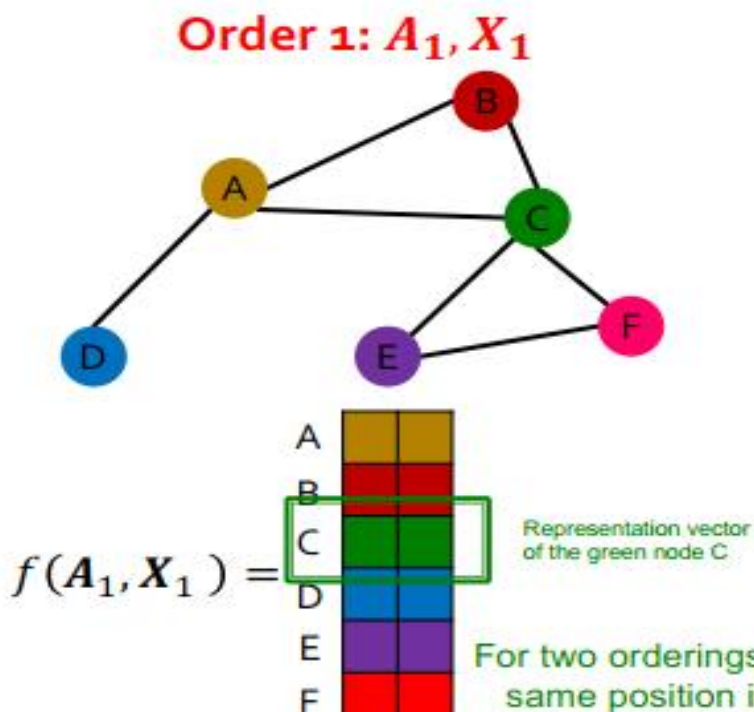


For two orderings, the vector of node at the same position in the graph is the same!

permutation equivariant (排列等价性)

需要学习一个函数 f ，将图的结点映射为一个 $|V|*d$ 大小的矩阵
换句话说，需要为每个结点学习一个 d 维向量。

要求：不同Order顺序下，位于图上相同位置的结点对应的向量应该相同。例如，C和D在图上位于相同的位置，其向量该相同

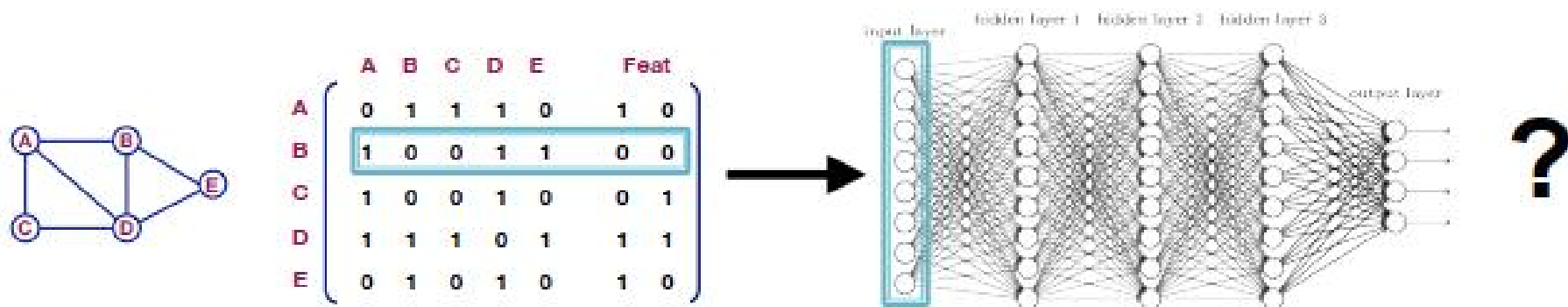


For two orderings, the vector of node at the same position in the graph is the same!

图神经网络

图神经网络由多个满足排列不变/等价性的函数组成

那下面的网络（MLP）是否满足排列不变/等价性呢？ 不!

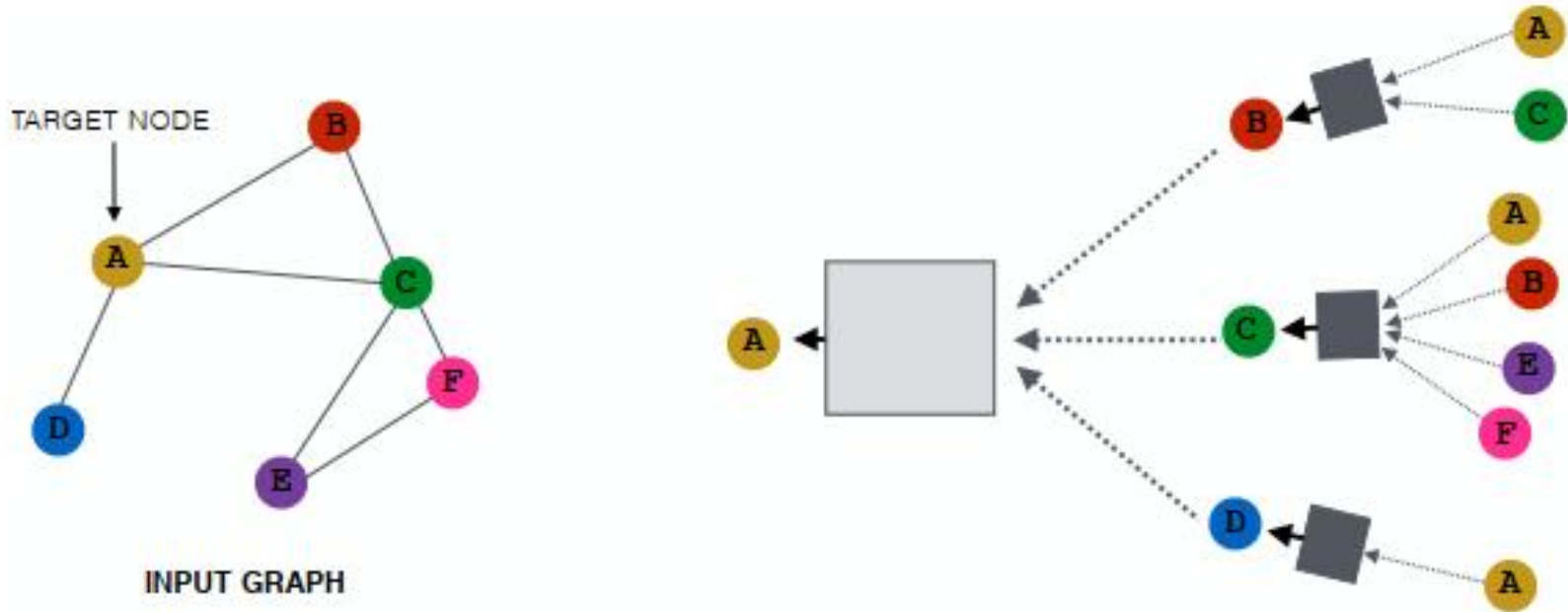


因此，前面提到的Naïve方法不能适用于图

需要设计新的能够满足排列不变/等价性的图神经网络

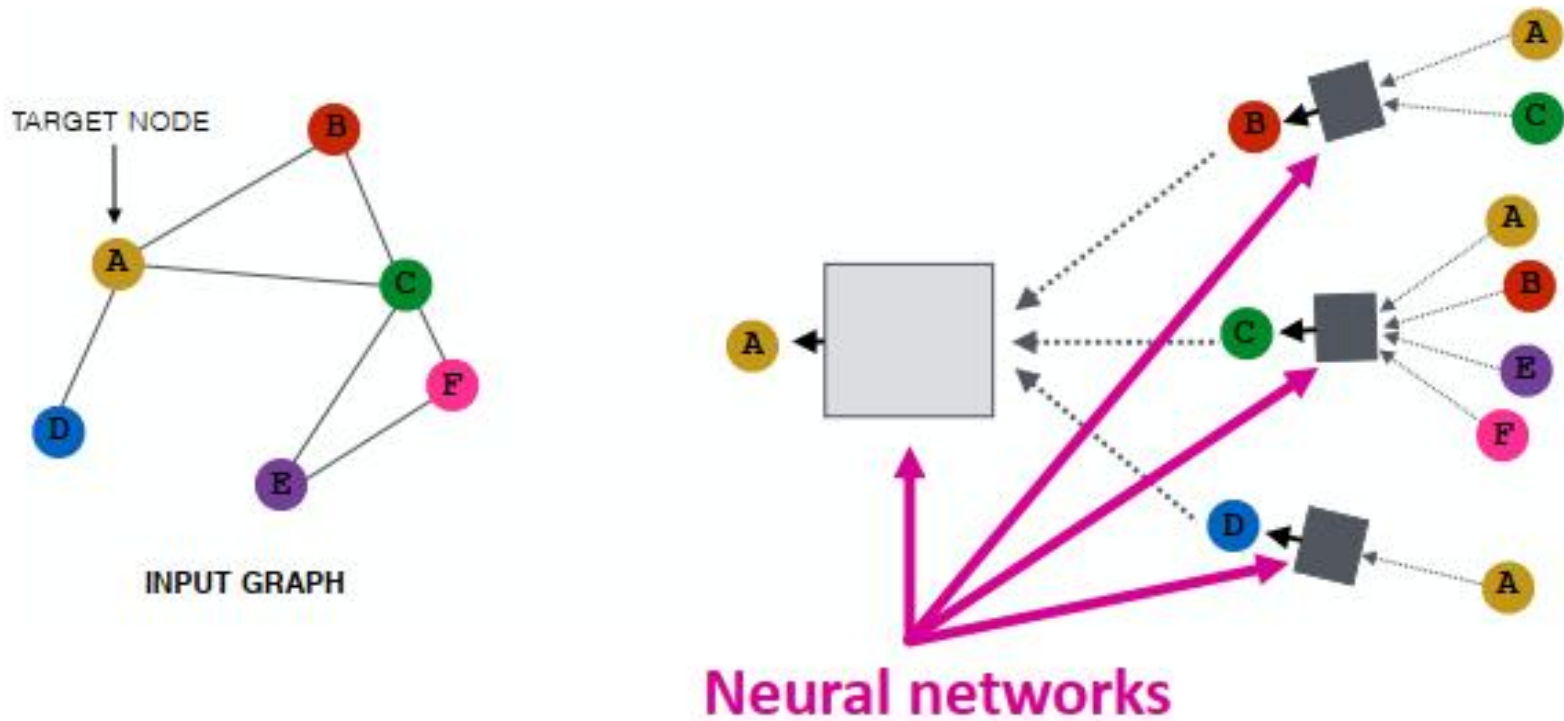
图神经网络

核心思想：根据结点的局部邻域 (Local neighborhoods) 信息来生成它的结点表示



图神经网络

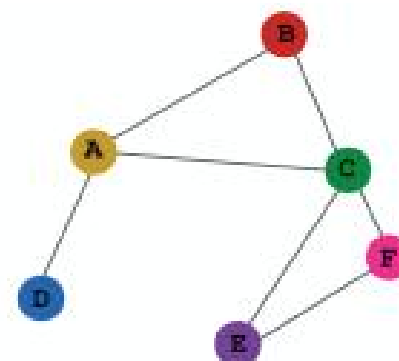
Intuition: 结点使用神经网络从它们的邻居聚集信息



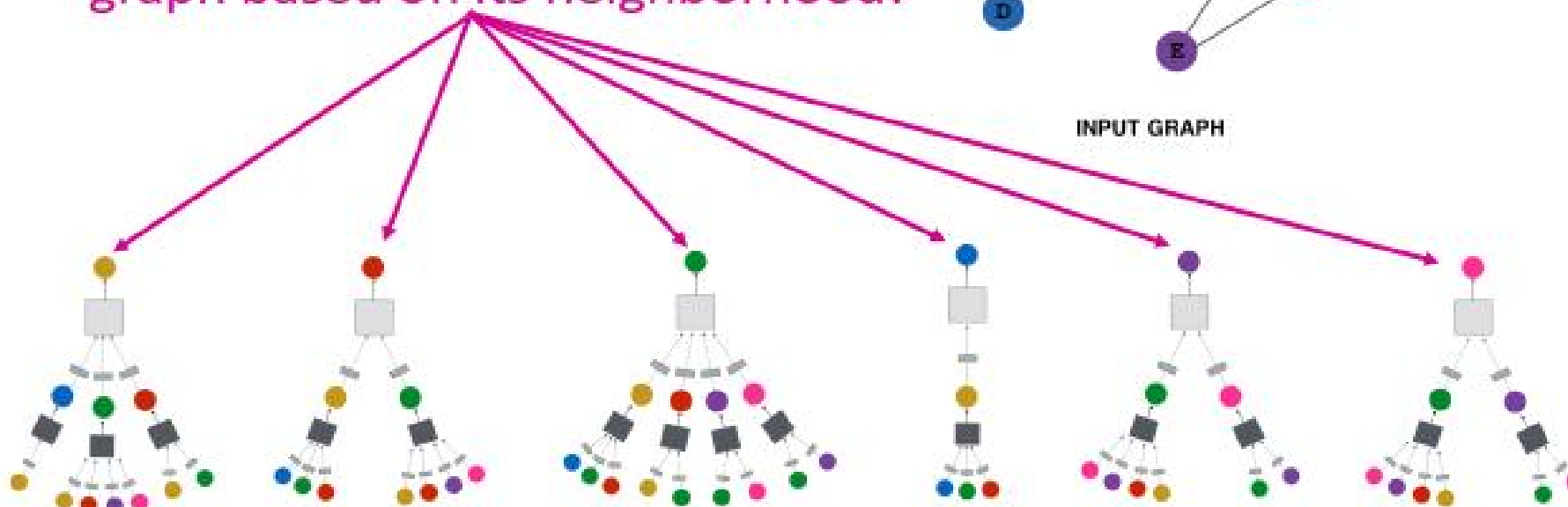
图神经网络

- **Intuition:** Network neighborhood defines a computation graph

Every node defines a computation graph based on its neighborhood!

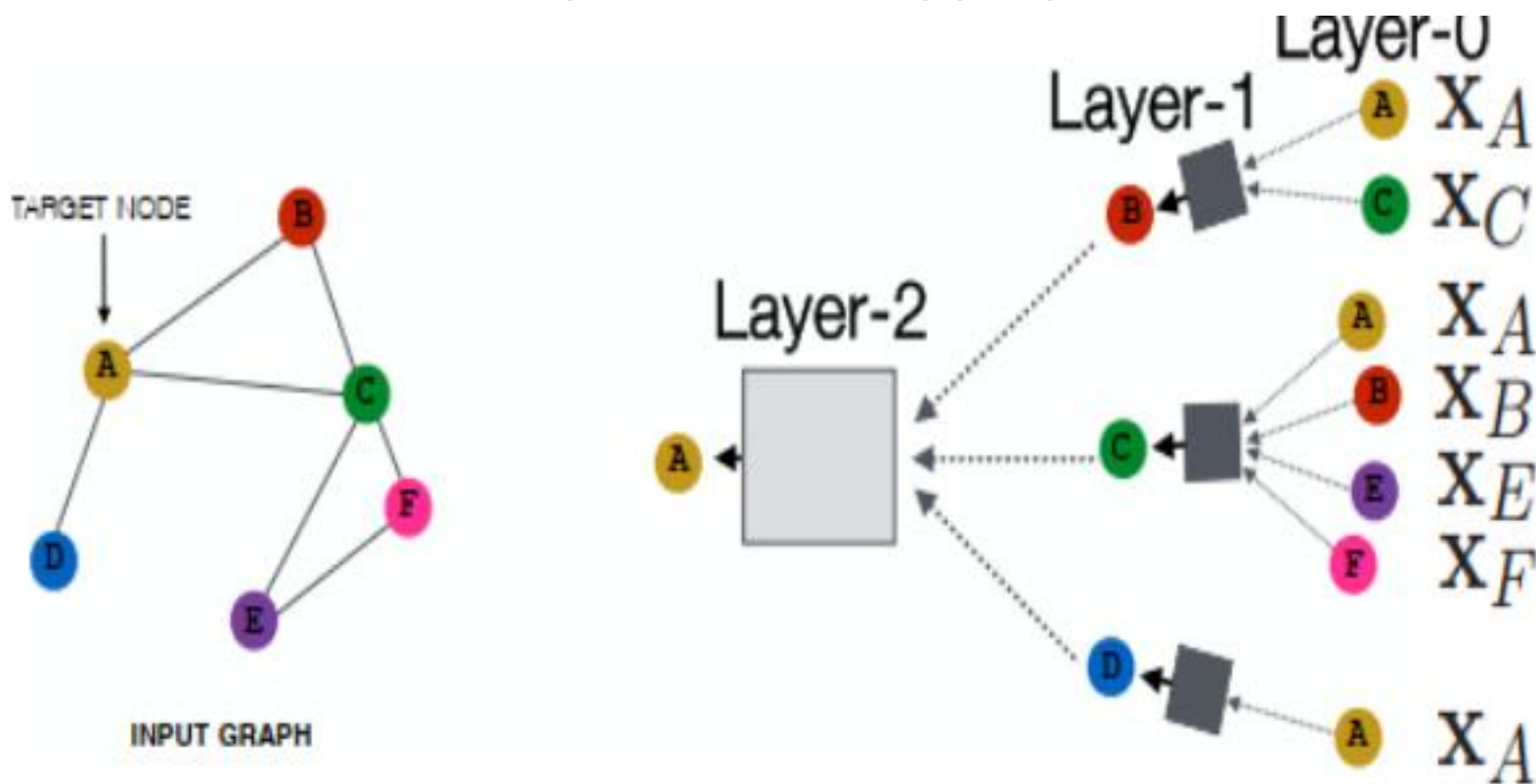


INPUT GRAPH



图神经网络

模型可以是任意深度。结点在每一层都有表示形式。结点 v 的第0层表示，就是 v 的特征 (X_v)；结点 v 的第 k 层表示，从 k -hops邻居聚集信息 (Neighborhood Aggregation)



基于Message Passing架构的GCN

一种简单而广泛使用的图卷积神经网络采用Message Passing架构
假设网络只有2层，采用ReLU激活函数，用于结点分类，则：

$$\mathbf{Z}_{\text{GCN}} = \text{softmax} \left(\hat{\mathbf{A}} \text{ReLU} \left(\hat{\mathbf{A}} \mathbf{X} \mathbf{W}_0 \right) \mathbf{W}_1 \right), \quad (1)$$

where $\mathbf{Z} \in \mathbb{R}^{n \times c}$ are the predicted node labels, $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ is the symmetrically normalized adjacency matrix with self-loops, with the diagonal degree matrix $\tilde{\mathbf{D}}_{ij} = \sum_k \tilde{\mathbf{A}}_{ik} \delta_{ij}$, and $\mathbf{W}_0$ and $\mathbf{W}_1$ are trainable weight matrices (Kipf & Welling, 2017).

更通用的基于Message Passing架构的GCN可表示如下

$$\mathbf{H}^{(\ell+1)} = \sigma(\tilde{\mathbf{P}} \cdot \mathbf{H}^{(\ell)} \cdot \mathbf{W}^{(\ell)})$$

$\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ 上一层的表示结果 待学习权重矩阵

图神经网络

图 $G = (V, E)$

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

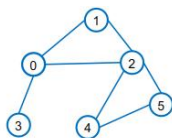


图 $G = (V, E)$

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{bmatrix}$$

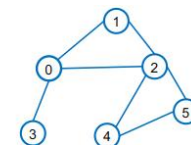


图 $G = (V, E)$

$$D^{-1/2} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$$

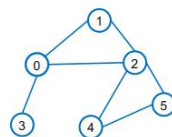
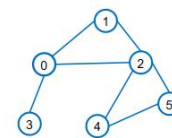


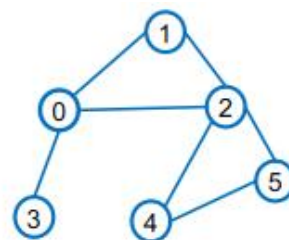
图 $G = (V, E)$

$$P = D^{-1/2} A D^{-1/2} = \begin{bmatrix} 0 & \frac{1}{\sqrt{3} \cdot \sqrt{2}} & \frac{1}{\sqrt{3} \cdot \sqrt{4}} & \frac{1}{\sqrt{3} \cdot \sqrt{1}} & 0 & 0 \\ \frac{1}{\sqrt{3} \cdot \sqrt{2}} & 0 & \frac{1}{\sqrt{2} \cdot \sqrt{4}} & 0 & 0 & 0 \\ \frac{1}{\sqrt{3} \cdot \sqrt{4}} & \frac{1}{\sqrt{2} \cdot \sqrt{4}} & 0 & 0 & \frac{1}{\sqrt{4} \cdot \sqrt{2}} & \frac{1}{\sqrt{4} \cdot \sqrt{2}} \\ \frac{1}{\sqrt{3} \cdot \sqrt{1}} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{\sqrt{4} \cdot \sqrt{2}} & 0 & \frac{1}{\sqrt{2} \cdot \sqrt{2}} \\ 0 & 0 & 0 & \frac{1}{\sqrt{4} \cdot \sqrt{2}} & \frac{1}{\sqrt{2} \cdot \sqrt{2}} & 0 \end{bmatrix}$$



图神经网络

节点特征矩阵 $X \in \mathcal{R}^{n \times f}$, f 代表特征维度。

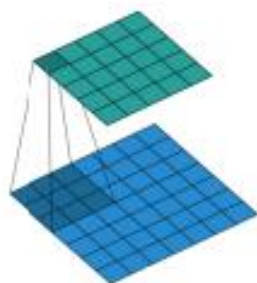


Node	Features1	Features2	Features3	Features4	Features5	Features6
$\mathbf{x}_0$	1	0	0	0	0	0
$\mathbf{x}_1$	0	1	0	0	0	0
$\mathbf{x}_2$	0	0	1	0	0	0
$\mathbf{x}_3$	0	0	0	1	0	0
$\mathbf{x}_4$	0	0	0	0	1	0
$\mathbf{x}_5$	0	0	0	0	0	1

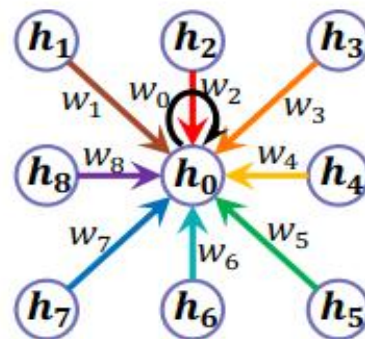
GCN vs. CNN

- CNN也可以看作是一种图卷积神经网络GCN

Convolutional neural network (CNN) layer with 3x3 filter:



Image



Graph

用3*3的卷积核进行卷积：可以看作是中心节点 h_0 聚合了周围8个邻居结点($h_1, h_2, \dots, h_8$)的信息和 h_0 自身的特征信息

-
- Stanford cs224w: Machine Learning with Graphs
<http://web.stanford.edu/class/cs224w/index.html#schedule>